

Augusto B. V. Silva, Vinicius B. Alvarenga

ArtEvoke

**AI-powered platform to trigger memories in Alzheimer's
patients using art imagery**

São Paulo, SP

2025

Augusto B. V. Silva, Vinicius B. Alvarenga

ArtEvoke

AI-powered platform to trigger memories in Alzheimer's patients using art imagery

Final undergraduate thesis submitted to the Department of Computer Engineering and Digital Systems of the Polytechnic School of the University of São Paulo in partial fulfillment of the requirements for the degree of Engineer.

University of São Paulo – USP

Escola Politécnica

Department of Computer Engineering and Digital Systems (PCS)

Supervisor: Prof. Jaime Sichman

Co-supervisor: Profa. Sonia Maria Dozzi Brucki

Co-supervisor: MD Raphael Ribeiro Spera

São Paulo, SP

2025

A arte existe porque a vida não basta.

Ferreira Gullar

Acknowledgements

The past six years, comprising both our bachelor's and master's studies, have been a journey of immense personal and academic growth. During this time, we have learned more about ourselves and refined our professional interests. This thesis marks the completion of our respective bachelor's and master's degrees: Augusto B. V. Silva in Computer Engineering, as part of a double degree program with UCLouvain in Belgium, and Vinicius Alvarenga in Computer Engineering, as part of a double degree program with Institut Polytechnique de Paris in Paris.

We would like to express our gratitude to our supervisor, Prof. Jaime Sichman, for his guidance and for allowing us to continue participating in a project that presents both a technical challenge and the potential for significant social impact. We also extend our thanks to Prof. Sonia Brucki and MD Raphael Spera for their support regarding the evaluation process and the medical aspects of the platform.

Regarding the initial phase of the project, we thank Prof. Benoît Macq from UCLouvain for the opportunity to work on this initiative. We are also grateful to Tiffanie Godelaine for her valuable feedback and help modeling of our platform, and especially to our colleague Marc Bejjani for his fundamental help in the early development stages.

We acknowledge the Center for Artificial Intelligence (C4AI) at USP for the technical support and infrastructure provided, represented by Renato Freire. Furthermore, we thank the entire team at the Museu do Ipiranga, specifically Vânia Carvalho, Laura Felicio, and Nicole Marziale, for providing access to their dataset and enabling the demonstration of the platform with patients in the museum.

Finally, we are deeply grateful to our families, friends, and God for their support throughout this thesis and our academic journey. Your encouragement, patience, and understanding, especially during the most stressful moments, kept us going. Thank you for always being there.

Abstract

This thesis explores how artificial intelligence can support non-drug interventions for people living with Alzheimer’s disease through the use of visual stimuli and art. We present the development and integration of ArtEvoke, a prototype platform that combines large multimodal models (LMMs), large language models (LLMs), and embedding-based vector search to connect personal stories with curated artworks.

Building upon the foundational research from a Master’s thesis (SILVA; BEJJANI, 2025), which benchmarked task-relevant LMMs and LLMs for image generation, description, and text-image retrieval, this work focuses on integrating these components into a functional prototype and introducing a novel clinical evaluation framework. Designed to assist memory recall and cognitive engagement, ArtEvoke generates personalized visual narratives or retrieves relevant artworks based on the user’s story. A key contribution of this thesis is the platform’s ability to support structured clinical sessions, enabling practitioners to capture patient responses and evaluate cognitive and memory abilities, in addition to proactively improve user engagement and overall interaction.

The prototype is intended for testing in future therapeutic and research settings as a scalable support tool. Our findings highlight the platform’s significant potential, suggesting that after further scientific testing and validation, it could become one of the first AI and art-based tools to offer a scalable and measurable therapeutic strategy for dementia care.

Keywords: LLM. Alzheimer’s Disease. Art. Retrieval Engine. AI. NLP.

Contents

1	Introduction	15
1.1	Context	15
1.2	Objectives and Approach	15
1.3	Thesis Organization	17
2	Conceptual Background	19
2.1	Alzheimer and dementia diseases	19
2.1.1	Overview	19
2.1.2	Non-drug interventions and prevention	20
2.1.3	Imagery use for intervention in dementia	21
2.1.4	Impact of the caregiver	22
2.1.5	Artificial Intelligence use for Alzheimer disease	22
2.1.6	Cognitive assessments	23
2.1.6.1	Overview and Role in Diagnosis	23
2.1.6.2	Classical Screening Assessments	25
2.1.6.3	Emerging and Digital Assessments	25
2.2	Large Multimodals Models (LMMs)	26
2.2.1	From Artificial Neurons to Transformers	27
2.2.2	Natural Language Processing (NLP)	29
2.2.3	LLM Architecture	30
2.2.4	LMM Architecture	31
2.2.5	Benchmarks	32
2.3	Embeddings	33
2.3.1	Overview	33
2.3.2	Architecture	34
2.3.3	Multi-Language Embeddings	34
2.3.4	Grammatical Errors	35
2.3.5	Text-First Embedding vs. Direct Image Embedding	35
2.4	Vector Database	36
2.4.1	FAISS	37
2.4.2	Qdrant	38
2.4.2.1	Overview and Architecture	38
2.4.2.2	Semantic Retrieval	38
2.4.2.3	Use Cases & Features	39
2.4.3	Comparison	39
3	Methodology	41
3.1	Art Features Extraction	41

3.2	User Interaction	43
3.2.1	Memory Reconstruction	43
3.2.2	Art Exploration	44
3.3	Clinical Evaluation Framework	45
3.3.1	Memory Reconstruction	46
3.3.2	Art Exploration	47
3.4	Models Selection	48
3.4.1	Benchmarks	48
3.4.2	Models - LMMs	49
3.4.3	Models - Embeddings	50
3.4.4	Models - LLM	51
3.4.5	Vector Database Selection	52
3.4.6	Text Segmentation	53
4	Requirements	55
4.1	Functional Requirements	55
4.2	Non-functional Requirements	57
5	Exploration and Evaluation of AI Models	59
5.1	Datasets	59
5.1.1	SemArt	59
5.1.2	WikiArt	61
5.1.3	Museu do Ipiranga	62
5.2	Evaluation of LMMs	62
5.2.1	Methodology	62
5.2.1.1	Evaluation Protocol	63
5.2.1.2	Quantitative Evaluation	63
5.2.1.3	Resolution Analysis Setup	64
5.2.2	Results and Analysis	64
5.2.2.1	Runtime and Resource Usage	64
5.2.2.2	Semantic Similarity Results	65
5.2.2.3	Statistical Significance	66
5.2.2.4	Resolution Impact Analysis	66
5.2.3	Final Considerations	67
5.3	Evaluation of Embedding Models	67
5.3.1	Methodology	67
5.3.1.1	Evaluation Setup and Protocol	67
5.3.1.2	Quantitative Metrics (Recall@K)	68
5.3.1.3	Multilingual Evaluation	68
5.3.1.4	Qualitative Evaluation	69
5.3.2	Results and Discussion	69

5.3.2.1	Performance and Efficiency	70
5.3.2.2	Text-Based Retrieval	70
5.3.2.3	Dual-Index Retrieval Strategy	71
5.3.2.4	Multilingual Performance	71
5.3.2.5	Subjective Evaluation	72
5.3.3	Final consideration	73
5.4	Evaluation of LLMs	73
5.4.1	Methodology	73
5.4.1.1	Evaluation Protocol	74
5.4.1.2	Evaluation Metrics	74
5.4.2	Results and Discussion	75
5.4.3	Final Considerations	76
5.5	Evaluation of Vector Database	76
5.5.1	FAISS	76
5.5.1.1	Methodology	76
5.5.1.2	Results and Discussion	77
5.5.1.3	CPU Performance	77
5.5.1.4	GPU Performance	80
5.5.1.5	FAISS Considerations	81
5.5.2	Qdrant	81
5.5.2.1	Evaluation Protocol	81
5.5.2.2	Recall and Latency	81
5.5.2.3	Memory Consumption	82
5.5.3	Final System Selection	83
5.6	User Input Pre-Processing	83
5.6.1	Methodology	83
5.6.1.1	Spell-Checking	83
5.6.1.2	Text Segmentation Setup	84
5.6.2	Results and Discussion	85
5.6.2.1	Quantitative Results	85
5.6.2.2	Diversity Criteria and Strategy Selection	85
5.6.3	Final Consideration	85
5.7	Discussion	87
6	System Architecture and Web Implementation	89
6.1	Overview	89
6.1.1	System Architecture	89
6.1.2	Technology Stack	89
6.1.3	Design Principles and Rationale	90
6.2	Front-end Architecture	90

6.2.1	General Architecture	91
6.2.2	Component Architecture and State Management	92
6.2.2.1	Local State Management	93
6.2.2.2	Custom Hooks	94
6.2.2.3	Component Hierarchy	94
6.2.3	Core Interface Flows	94
6.2.3.1	Memory Reconstruction Flow	95
6.2.3.2	Art Exploration Flow	97
6.2.3.3	Clinical Evaluation Flow	99
6.2.4	Accessibility	104
6.2.4.1	Overview	104
6.2.4.2	Perceivability	105
6.2.4.3	Operability	105
6.2.4.4	Readability	106
6.2.4.5	Understandability	107
6.2.4.6	Internationalization (i18n)	107
6.3	Backend Architecture	108
6.3.1	FastAPI Application Structure	108
6.3.2	Database Layer	109
6.3.3	External Services Integration	110
6.3.3.1	Vector Database (Qdrant)	111
6.3.3.2	AI Services	111
6.3.4	API Design and Endpoints	112
6.3.4.1	Request and Response Schema Architecture	112
6.3.4.2	Authentication and Authorization	112
6.4	Data Management	113
6.4.1	Embedding Generation and Storage	113
6.4.2	Art Catalog Structure	113
6.4.3	Interactive Modes Data Flow	115
6.4.4	Session Management and Evaluation Persistence	115
6.5	Infrastructure and Deployment	117
6.5.1	Containerization Strategy	117
6.5.2	Service Orchestration	118
6.5.3	Reverse Proxy Configuration	119
6.5.4	Database Administration	120
6.6	Platform Access	121
7	Conclusions and Further Work	123
7.1	Summary and Main Findings	123
7.2	Demonstration at Ipiranga Museum	123

7.2.1	Participant Demographics	124
7.2.2	Evaluation Scores	124
7.2.3	Qualitative Observations	124
7.2.4	General Outcome	125
7.3	Limitations and Improvement Areas	125
7.4	Future Work	126
	Bibliography	129
	Appendices	138
A	Models Overview and Benchmarks	139
A.1	LMM Model Specifications	139
A.2	Benchmark Results	139
A.3	Embedding Models	141
A.4	Embeddings Benchmark	141
A.5	LLM Models Specifications	142
A.6	LLM Models Benchmarks	142
B	Datasets	143
B.1	Sampled 2000 images from SemArt	143
B.2	Sampled 15,570 images from SemArt	144
C	Similarity scores	145
C.1	Statistical information between models	145
C.2	Statistical information between pixel max sizes	146
D	Survey answers	147
E	FAISS configurations	149
F	Database Schema	151

1 Introduction

1.1 Context

Alzheimer’s disease (AD) is a common neurological disorder that affects the memory and behavior of the affected people. In fact, it is the leading cause of dementia, which is the severe loss of mental function, enough to greatly affect daily life negatively. It is estimated that around 55 million people currently suffer from this syndrome, and that number is expected to rise to nearly 150 million by the year 2050, based on estimates from the World Health Organization (World Health Organization, 2017). Consequently, as the global population ages due to a number of different factors, there is an increasing demand to innovate and develop accessible intervention methods for the mentioned disease.

While pharmaceutical treatments do exist today, they possess a limited effectiveness and scope especially due to the fact that they are not able to successfully tackle the progressive or degenerative nature of the disease (PARDO-MORENO et al., 2022). In light of this, other approaches are being explored, such as cognitive stimulation (WOODS et al., 2023). The latter is a field studied by both clinical and technical researchers, and is aimed at improving the quality of life of the patients by providing them with activities that stimulate their thinking, concentration, and other brain activities.

In this context, and as further discussed in the next Chapter (Section 2.1.3), recent research has explored the use of visual imagery as a cognitive stimulation tool for individuals with AD. Studies suggest that exposure to artworks can help trigger memory and activate neural pathways, offering therapeutic benefits. However, selecting appropriate images that resonate with a patient’s experiences remains a non-trivial task. While expert input may enhance the relevance of selected stimuli, this manual process can be time-consuming and difficult to scale.

At the same time, the field of artificial intelligence (AI) has undergone rapid advancements, particularly in the development of models capable of interpreting and generating visual and textual information. These technologies can be leveraged to support and automate parts of the image selection and storytelling process, overcoming limitations of earlier systems and enabling more flexible and scalable therapeutic interventions.

1.2 Objectives and Approach

The objective of this thesis is to develop ArtEvoke, a prototype application that leverages artificial intelligence and visual stimuli to provide an accessible tool for individuals

with AD or other forms of dementia. By combining AI with curated artworks, ArtEvoke aims to stimulate memory and cognitive engagement through personalized visual narratives. This work contributes to the field of non-pharmacological interventions for dementia by proposing a scalable platform that not only delivers stimulation but also provides a framework for assessing its impact. The prototype is intended to serve both as a research tool and, potentially, as a practical solution in future clinical applications.

Beyond providing cognitive stimulation, a key objective of this work is to introduce a method for evaluating patient outcomes following interaction with the platform. To achieve this, ArtEvoke is designed to support clinical use by enabling doctors to create structured sessions. After a patient engages with an interaction mode, the system facilitates an evaluation to capture and measure the patient’s cognitive and emotional response. This feature provides physicians with a preliminary metric to track engagement and potential changes over time, while also laying the groundwork for future validation studies to determine the platform’s statistical effectiveness.

It is important to contextualize that this thesis builds upon foundational research conducted during a Master’s thesis at UCLouvain, co-authored by one of the authors, in (SILVA; BEJJANI, 2025). That initial work established the core AI exploration, including the selection and benchmarking of AI models and the initial feature extraction strategies. This thesis refines, integrates, and expands upon that foundation to develop the complete ArtEvoke prototype and, crucially, introduces its clinical evaluation framework.

To achieve these dual goals, the following approach was adopted:

- **Selection and evaluation of AI models:** Identify and benchmark the most suitable large multimodal models (LMMs), large language models (LLMs), and embedding strategies for description generation, storytelling, and similarity search.
- **Feature extraction from artworks:** Use AI-based tools to extract descriptive, emotional, and symbolic information from art datasets to enhance retrieval and interaction quality.
- **Design of interaction modes:** Develop and refine interaction modes that enable dynamic and engaging user experiences through visual content.
- **Development of an evaluation framework:** Design and integrate a module that allows physicians to create sessions and assess patient responses post-interaction, providing a quantifiable metric of engagement and cognitive response.
- **Prototype development:** Integrate all components (AI models, feature database, interaction modes, and evaluation module) into a functional and user-friendly prototype, with special attention to accessibility and cognitive usability for elderly users.

Figure 1 presents a simplified overview of the ArtEvoke platform. The process begins with two key inputs: **Feature Extraction**, where visual and semantic features are derived from artworks, and **Patient Input**, which may include keywords, memories, or short personal narratives. These inputs feed into the **Matching Engine**, responsible for retrieving the most relevant images, and into the **Story Engine**, which generates a personalized narrative based on the patient’s input and the extracted dataset features. The output consists of **Tailored Patient Stimuli**: a combination of visual and textual elements designed to support memory recall and enhance emotional engagement. Finally, **Patient Evaluation** captures the patient’s reactions to these stimuli, allowing clinicians to assess cognitive and emotional responses and determine the effectiveness of the generated content. The technical structure and implementation of each component will be detailed in Chapter 3.

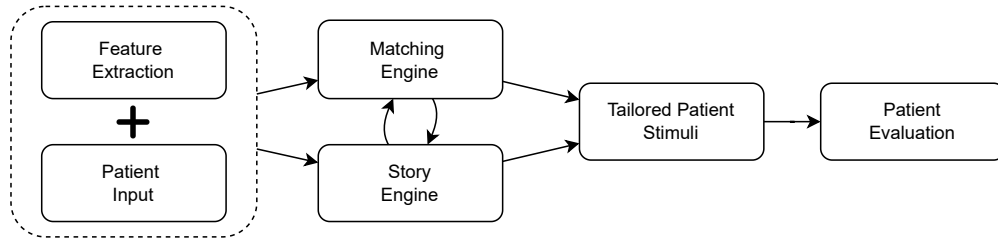


Figure 1 – Overview of the ArtEvoke platform.

1.3 Thesis Organization

Finally, this report is organized as follows:

- **Chapter 1 – Introduction** introduces the research context, main objectives, and the adopted methodology. It also presents a brief overview of the *ArtEvoke* platform and outlines the structure of the document.
- **Chapter 2 – Conceptual Background** reviews the state of the art concerning Alzheimer’s and dementia, including current interventions, the role of imagery and caregivers, and the use of Artificial Intelligence in this domain.
- **Chapter 3 – Methodology** describes the methodological approach and design of the proposed solution. It details the processes of art feature extraction, user interaction design, and the selection of AI models and supporting tools.
- **Chapter 4 – Requirements** specifies the functional and non-functional requirements that guided the development of the *ArtEvoke* platform.
- **Chapter 5 – Exploration and Evaluation of AI Models** reports the evaluation of various AI models, including LMMs, embeddings, and LLMs.

- **Chapter 6 – System Architecture and Web Implementation** presents the overall system architecture, backend and frontend implementation details, and the integration of AI services.
- **Chapter 7 – Final Considerations** concludes the thesis with a summary of the main findings, the identified limitations, and possible directions for future research and platform improvements.

To support the writing process, AI tools were used to assist in improving grammar coherence and organization in this report.

2 Conceptual Background

2.1 Alzheimer and dementia diseases

2.1.1 Overview

Dementia is the set of symptoms that are related to the decrease of memory, language and other logical capabilities. The tendency of this type of illness is to get worse over time, mainly with some risk factors, like age, hypertension, diabetes and other factors. The patients could include changes in mood and behavior, like personality changes and dislocation of the real world (World Health Organization, 2023).

With the growth in the elderly population, there is a tendency of certain types of diseases to increase, like the ones related to dementia symptoms, which includes problems with memory, language and other logical abilities (Alzheimer's Association, 2024). In 2021, the World Health Organization estimated the number of people living with dementia was to be around 55 million and it is expected to reach almost 150 million by 2050 (World Health Organization, 2021).

AD is one that causes dementia, it is caused by damaged nerve cells in the brain mainly in the areas responsible for thinking, language and memory. It is estimated that around 60-80% of dementia in the world is caused by AD (Centers for Disease Control and Prevention, 2024). As dementia, AD is progressive, affecting increasingly the person's ability to effectively do their work and day to day activities. After some time, the disease affects motor functions, like walking, grabbing and swallowing (World Health Organization, 2023; Alzheimer's Association, 2024). The progression of the disease is illustrated in Figure 2.

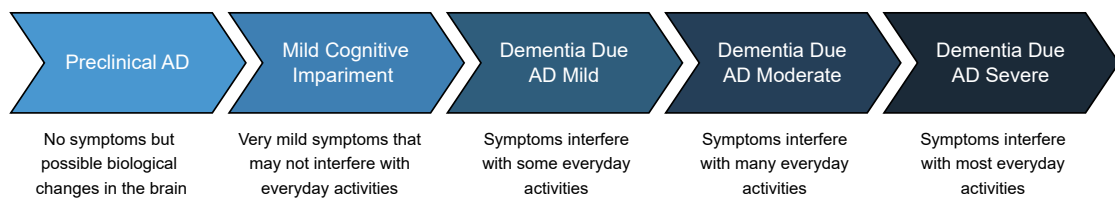


Figure 2 – Diagram illustrating the progression of AD, adapted from (Alzheimer's Association, 2024).

The cause of AD has been studied for decades and remains a major focus of research. One of the most recognized pathological remarks of AD is the progressive accumulation of amyloid- β ($A\beta$) peptides in the brain. These peptides aggregate to form senile plaques (SP), which are thought to contribute to synaptic dysfunction and neuronal loss. Although

the exact mechanisms remain unclear of this accumulation, approximately 95% of AD cases are sporadic, and many patients accumulate $A\beta$ in the absence of known genetic mutations (PARDO-MORENO et al., 2022).

A variety of factors influence the disease, including genetics and external influences such as behaviors and habits. A 2019 study evaluated several external risk factors (e.g., high blood pressure, obesity, diabetes and depression) and their relationship with subjective cognitive decline (SCD), which refers to self-reported mental confusion. The study revealed that individuals with SCD tended to have a higher number of risk factors (OMURA et al., 2022).

Patients with AD may have varying survival durations, depending on the age at diagnosis, the start of treatment, and the presence of comorbidities. The median survival time after diagnosis was 6.37 years for men and 8.81 years for women. However, when dividing by age and comorbidities, survival time ranged from 13.72 to 5.29 years for patients without comorbidities and from 6.43 to 2.98 years for those with three or more comorbidities (SUN et al., 2024; LARSON et al., 2004).

There is no current effective treatment for the progression of Alzheimer's dementia, however, some drugs have been developed to help with it. Until 2022, only six drugs had been approved by the US Food and Drug Administration. Among these, only one targets the mechanism of clearing $A\beta$ peptides, which are considered one of the causes of the disease, while the others are aimed at alleviating its symptoms (PARDO-MORENO et al., 2022). Additionally, as already mentioned, many risk factors are correlated with AD, and controlling them can reduce or delay the development of it (Centers for Disease Control and Prevention, 2024; OMURA et al., 2022).

2.1.2 Non-drug interventions and prevention

Alternative non-pharmacological interventions has been studied for years for dementia diseases. Most notably, diets, physical activities, and cognitive training has shown in some studies promising advantages in adopting them (PARDO-MORENO et al., 2022; LI et al., 2023a).

The Mediterranean diet offers a rich eating routine, with fruits, vegetables, nuts, olive oil, and lean proteins like fish. It is well-known for its neuroprotective properties, helping to preserve cognitive functions (PARDO-MORENO et al., 2022). Additionally, it addresses major risk factors for developing AD. For instance, it reduces insulin resistance, which is a primary cause of diabetes, a significant risk factor for AD (DOVE et al., 2021).

As with diets, physical activities attack main risk factors of AD, such as obesity and sedentarism. The literature has shown that physical activity helps reduce the chance of developing the disease and delays its progression in the early stages (PARDO-MORENO et al., 2022). In addition, physical activity, mainly aerobic exercises, such as sports and

daily tasks, improves patients' cognitive functions, leading to enhancements in their daily lives (HÖTTING; RÖDER, 2013).

Cognitive training is a type of treatment that focuses on cognitive functions that are usually degraded by the progression of age or disease, it includes memory, language, and attention functions (LI et al., 2023a). After completing the cognitive training, patients had small to moderate effects on cognition, overall and specific such as verbal improvements, which lasted up to one year (LIVINGSTON et al., 2020). Adaptive chunking training has led patients with mild AD to significant improvements in verbal memory performance (HUNTLEY et al., 2017).

On the other hand, Cognitive Stimulation Therapy (CST) is a more generalized approach that engages people in activities designed to stimulate thinking, concentration, and memory (LI et al., 2023a). When compared to no cognitive stimulation in people with dementia (DA), all measured parameters showed improvement (in group or not), including cognitive function, quality of life, communication and social interaction, mood (as reported by both the patient and the interviewer), activities of daily living, and behaviors that challenge (WOODS et al., 2023).

Reminiscence therapy is a form of CST that uses all the senses to stimulate memory triggers, helping people suffering from Alzheimer's disease (AD) recall their past to better cope with their present (LI et al., 2023a). This sensory stimulation is typically achieved through familiar stimuli such as old photographs, music, scents, recorded voices, or objects from the person's past, which are used to evoke positive memories in either individual or group sessions. A three-month trial using reminiscence therapy with patients with mild to moderate AD showed positive results, including an increase in cognitive levels and a decrease in depressive symptoms (AŞİRET; KAPUCU, 2015).

2.1.3 Imagery use for intervention in dementia

With the growing interest in non-pharmacological interventions, visual arts have become a subject of study and research. While language is processed in the left hemisphere of the brain, art engages widespread neural networks due to their complexity. As a result, visual art stimulates broader brain regions, which are remarkably resilient to the brain due to aging or diseases (ZAIDEL, 2010).

In one study, a Cognitive Activation Therapy (CAT) was developed using art imagery, primarily aimed at addressing age-related decline and worsening cognitive function. CAT is a protocol designed to stimulate patients and support improvements in daily activities (GOLLIN et al., 2012). The study revealed that the length of leukocyte telomeres, a biological marker associated with cellular aging, increased significantly after the full course of CAT, suggesting a beneficial effect at the cellular level. In addition, cognitive domains such as memory, spatial perception, and communication skills showed improvement

after the intervention (CAMPISI et al., 2024).

Similarly, a culture-specific, picture-based Cognitive Training Intervention for Alzheimer’s Disease demonstrated significant cognitive improvements in patients with early-stage AD. The training focused on structured visual tasks targeting key cognitive domains such as episodic and semantic memory, attention, and language, using non-verbal materials to accommodate low literacy levels. Participants showed enhanced performance in memory, attention, and language-related tasks (BAJPAI et al., 2020). These findings reinforce the potential of visual arts as a therapeutic tool, not only for preserving cognitive function but also for promoting neural resilience and adaptation in neuro-degenerative conditions.

2.1.4 Impact of the caregiver

Caregivers play a crucial role in the lives of people living with dementia (PLwD), often acting as a source of emotional and physical support. As studies have shown, the presence of a caregiver is associated with improved quality of life, increased safety for PLwD, and better adherence to treatment (WEIZENBAUM, 1966).

However, the burden that caregivers often carry comes at a significant personal cost, especially with dementia patients. One study shows that caregivers of PLwD report higher levels of stress, depressive symptoms, and health issues compared to other caregivers, highlighting the dual impact of dementia not only on patients but also on those who care for them (CHENG, 2017). In addition, it is mentioned that caregiving is often provided by close family members, who are usually not professionals in the field.

One of the most challenging aspects for caregivers of PLwD is dealing with neuropsychiatric symptoms (NPS), such as agitation, aggression, and emotional instability. These symptoms are often considered more disturbing than memory or functional impairments, significantly increasing caregiver stress levels (NEMCIKOVA; KATRENIKOVA; NAGYOVA, 2023). In fact, NPS are frequently cited as the primary source of caregiver distress, given their emotional toll they take on those providing care.

2.1.5 Artificial Intelligence use for Alzheimer disease

In recent years, AI has increasingly been applied in healthcare, offering new tools for the diagnosis, prediction, and treatment of AD. Traditionally, diagnosis relied on clinical evaluations, cognitive assessments, as mentioned in 2.1.6, and imaging techniques such as MRI or PET scans. However, these methods have limitations, including insufficient sensitivity in early stages of the disease (ZHANG et al., 2023; ANGELUCCI et al., 2024).

AI has emerged as a promising alternative to support AD research. With Machine Learning (ML), scientists are able to detect early indicators of AD through features extracted from MRI and clinical data (ZHANG et al., 2023). In contrast to ML, Deep

Learning (DL) models can enhance this process by handling complex data, combining different sources of information, and providing interpretable visualizations of disease probability (ANGELUCCI et al., 2024).

Moreover, DL techniques have demonstrated the ability to predict AD up to six years before clinical diagnosis based on PET scans (ANGELUCCI et al., 2024). AI has also been applied in unconventional diagnostic methods, such as analyzing retinal photographs, speech patterns, and sleep behavior, all showing high accuracy in distinguishing AD from healthy individuals or other cognitive impairments (ANGELUCCI et al., 2024).

Furthermore, AI is being used to personalize treatments and enhance patient care. AI-powered cognitive stimulation tools, such as personalized brain training and voice assistants, have shown potential in slowing disease progression and supporting daily activities. These systems can adapt to the patient’s cognitive level, promoting engagement and communication (ANGELUCCI et al., 2024).

In addition, chatbots and virtual assistants can contribute to an improved quality of life for patients and their families (ANGELUCCI et al., 2024; TREDER; LEE; TSVETANOV, 2024). Recently, the use of LLMs has been explored in dementia care with promising results. LLMs can support meaningful conversation, offer cognitive stimulation, and help users with day to day activities. When integrated into easy-to-use apps, these models can adapt to individual needs, provide companionship, and reduce caregiver burden. Users and caregivers have expressed optimism about these tools, especially when designed with transparency and accessibility (TREDER; LEE; TSVETANOV, 2024).

2.1.6 Cognitive assessments

2.1.6.1 Overview and Role in Diagnosis

Mental status examination (MSE) is a important tool for physicians to help them diagnose their patients, even compared with physical examination as a reliable method of investigation of the patient (NETO et al., 2019). The MSE considers both information gathered during the patient and the doctor encounter, as well as, information from the patient daily routine and previous assessments. The examination can help assess between different kinds of disorders, like mood or thought, and cognitive impairment (SNYDERMAN; ROVNER, 2009; BARRAGAN et al., 2025).

According to (SNYDERMAN; ROVNER, 2009), the MSEs have ten main components they are trying to evaluate (not at the same time), and the main ones are selected and shown below:

- *Appearance and general behavior*: body habitus, appearance, eye contact, and general behavior;

- *General behavior*: posture, catatonia, psychomotor agitation and retardation;
- *Speech*: quantitative, rate, volume, fluency and coherence;
- *Thought*
- *Sensorium and Cognition*

For the last one, *Sensorium and Cognition* is an essential part of the MSE. The first evaluate the patients ability to sustain consciousness, indicating delirium otherwise. This one is evaluated exploring some key aspects, like alertness, somnolence, comatose, and others. For the cognitive part, one of the most important for this study, includes attention, concentration and memory. This part can be also called Cognitive Assessment and have some well-known and systematic approaches that will be explored in the section 2.1.6.2.

Cognitive screening plays a central role in the early detection of AD and related dementias. Its primary purpose is to identify individuals who may be exhibiting early signs of cognitive decline and who therefore require more comprehensive diagnostic evaluation. Early screening is essential because the initial phases of AD often present with subtle impairments that can be mistaken for normal aging or overlooked during routine clinical encounters. Detecting these early signs enables clinicians to initiate supportive interventions and plan follow-up assessments appropriately. Moreover, screening helps differentiate cognitive impairment related to neurodegenerative processes from conditions such as depression which require different prognosis (BARRAGAN et al., 2025).

It is important to differentiate screening, diagnosis, and monitoring, as these processes address distinct clinical objectives, described below:

- **Screening** involves quick and standardized tests used to flag possible impairments but does not confirm a specific disorder;
- **Diagnosis** requires a more comprehensive evaluation that may include detailed testing, imaging, laboratory tests, and clinical history to determine the underlying motive of cognitive decline;
- **Monitoring**, in turn, assesses disease progression and the effectiveness of treatment or supportive strategies over time.

Barragan et al. (2025) highlight that thorough cognitive assessments can reveal different trajectories of decline in individuals diagnosed with AD, helping differentiate true AD from conditions that mimic it. This reinforces the importance of repeated cognitive evaluations not only for diagnostic clarification but also for ongoing clinical management.

2.1.6.2 Classical Screening Assessments

One of the main cognitive screening instruments is the Mini-Mental State Examination (MMSE). The MMSE is a brief, standardized test composed of eleven items designed to assess various cognitive domains, including orientation, attention, memory, language, and visuoconstructional skills. It takes approximately 5–10 minutes to administer and focuses exclusively on cognitive functions, excluding aspects such as mood or thought form (FOLSTEIN; FOLSTEIN; MCHUGH, 1975).

The MMSE has shown high reliability and correlation with the Wechsler Adult Intelligence Scale (WAIS) after the study. It can distinguish individuals with cognitive impairment from those without and track cognitive changes over time, making it suitable for serial clinical assessments. Its simplicity, and quantitative scoring make it one of the most widely used tools for screening and monitoring dementia and related disorders (FOLSTEIN; FOLSTEIN; MCHUGH, 1975).

Another important test is The Montreal Cognitive Assessment (MoCA) (NASREDDINE et al., 2005). It was developed to overcome the limited sensitivity of the MMSE in identifying early cognitive decline, particularly mild cognitive impairment (MCI). The MMSE often fails to detect subtle deficits, as most individuals with MCI score within the normal range.

The MoCA is a brief screening tool designed for patients with mild cognitive complaints. It assesses executive function, memory, attention, and language abilities using more complex and demanding tasks than the MMSE. In validation studies with patients diagnosed with MCI, AD, and healthy controls, the MoCA demonstrated excellent sensitivity (90% for MCI, 100% for AD) and high specificity (87%), outperforming the MMSE, particularly for detecting MCI (NASREDDINE et al., 2005).

Its strong properties, like reliability, internal consistency, and cross-language equivalence, make the MoCA one of the most effective tools for early detection of cognitive impairment. Its use helps clinicians rapidly and reliably evaluate patients cognitive function (NASREDDINE et al., 2005).

2.1.6.3 Emerging and Digital Assessments

Digital and Information and Communication Technology (ICT) based cognitive assessments have increasingly gained attention as tools to complement or extend classical screening methods in dementia care. A recent review covering studies from 2015 to 2025 showed that ICT-based cognitive interventions can produce moderate but significant improvements in cognitive performance, particularly in attention, memory, and executive functioning (ALAM; RABBANI; PRYBUTOK, 2025). These findings suggest that digital solutions are not only feasible but clinically meaningful when appropriately designed.

Importantly, the review emphasizes that ICT is most effective when implemented as part of a structured and sustained cognitive support strategy rather than as isolated training exercises.

Despite this potential, the real-world adoption of digital cognitive tools remains limited. Although more than half of older adults with MCI or dementia own smartphones or tablets, fewer than 10% engage with programs specifically intended to support memory or cognition (ALAM; RABBANI; PRYBUTOK, 2025). The review attributes this gap not to lack of interest, but primarily to usability barriers and the absence of guided support during tool use. Individuals in early stages of cognitive decline, in particular, report willingness to use digital tools when these are perceived as personally meaningful, easy to operate, and integrated into daily routines.

In this context, caregivers play a decisive role, as already mentioned in the overall scenario. The review notes that sustained digital engagement increases substantially when caregivers or clinicians help orient users, personalize task difficulty, and integrate the platform into day to day activities. Additionally, interventions that incorporate socially and emotionally relevant content, such as personal photos, familiar music, or meaningful aesthetic experiences, show higher adherence and stronger cognitive outcomes than patterned exercises (ALAM; RABBANI; PRYBUTOK, 2025).

This evidence is particularly relevant for approaches that aim to embed cognitive assessments within familiar or meaningful activities. Digital platforms that present cognitively engaging tasks through contexts such as artistic exploration, visual interpretation, or guided reflection may help maintain user attention and reduce task-related anxiety, especially in individuals experiencing early cognitive changes. In such settings, the interaction is not primarily framed as a test but as a structured activity that elicits memory, attention, and interpretive reasoning. This may support both cognitive engagement and a more comfortable assessment environment, while still allowing clinicians or researchers to monitor key cognitive domains in a systematic manner.

Therefore, emerging digital assessment frameworks suggest that the effectiveness of cognitive screening tools depends not only on the cognitive domains they measure, but also on how they are experienced by the user. By situating cognitive evaluation within an art-based interaction environment, the platform developed in this study aligns with contemporary recommendations for meaningful, engaging, and person-centered digital dementia care, as highlighted in (ALAM; RABBANI; PRYBUTOK, 2025).

2.2 Large Multimodal Models (LMMs)

With the public release of ChatGPT at the end of 2022, a platform with access to the chat bot powered by the OpenAI Generative Pre-training Transformer (GPT), LLMs

became widely available for users to help with their tasks. Since then, these models have been used in both professional and personal life and started a run worldwide for achieving the best results with the lowest cost. Those models have the ability of processing and generating text with human-like communication, generating a text output according to the user's input (ARCAS, 2022). Building on this foundation, multimodal models extend LLM capabilities by processing and generating content across multiple modalities, such as text, images, audio, and video. They incorporate additional components to manage and integrate diverse modalities efficiently (HUANG et al., 2024).

2.2.1 From Artificial Neurons to Transformers

One of the most influential papers in the development of Artificial Intelligence was published in 1958, when Frank Rosenblatt introduced the **Perceptron** model in his seminal paper "*The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*" (ROSENBLATT, 1958). Inspired by the structure of biological neurons, the Perceptron was designed to mimic how interconnected neurons in the brain might produce intelligent behavior. In essence, it is a binary classifier that computes a weighted sum of input features and applies a threshold to decide the output: if the sum exceeds the threshold, the output is 1, otherwise, it is 0. Each input is associated with a weight and threshold which affects the decision (ROSENBLATT, 1958). Although limited to solving linearly separable problems, the Perceptron laid the groundwork for more complex models and is considered the first formal idea of an artificial neuron.

Utilizing the idea of the Perceptron, more sophisticated neural network architectures emerged by combining multiple artificial neurons into **structured layers**. These networks introduced activation functions, which define how the weights and bias are interpreted. Functions such as the sigmoid, tanh, and later ReLU enabled models to learn complex, non-linear mappings (AGARAP, 2019; GABRIÉ et al., 2023). A basic feedforward neural network consists of an input layer, one or more hidden layers, and an output layer, where data flows unidirectionally, the simplest one with multiple layers is known as the multi-layer perceptron (**MLP**) (GABRIÉ et al., 2023). The depth of a network, defined by the number of hidden layers, allows the network to learn representations, where successive layers increasingly capture features. This concept forms the foundation of DL.

Recurrent neural networks (RNNs) extend conventional feedforward neural networks by incorporating a recurrent hidden state, allowing them to process long sequences of data where the activation at each time step depends on the previous one. However, RNNs often struggle to learn long-term dependencies due to vanishing or exploding gradients (CHUNG et al., 2014). To mitigate this issue, Long Short-Term Memory (LSTM) networks were introduced, incorporating nonlinear and more complex control mechanisms into each cell to preserve the gradient and enable effective learning across long sequences

(HOCHREITER; SCHMIDHUBER, 1997; SHERSTINSKY, 2020). Gated Recurrent Units, a simpler alternative to LSTMs, were proposed to capture dependencies over varying time scales without using separate memory cells, relying instead on gating mechanisms to control information flow (CHUNG et al., 2014).

However, although these recurrent architectures were designed to capture long-range dependencies, they still suffered from limitations such as vanishing gradients and long path lengths between distant tokens. To address these issues, the **self-attention** mechanism was introduced (VASWANI et al., 2023). Also known as intra-attention, it relates different positions within a single sequence to compute contextualized representations, allowing each token to consider all others in the sequence without relying on a hidden state, as in LSTM networks. This mechanism has proven effective in a variety of tasks, including reading comprehension and textual summarization (VASWANI et al., 2023).

At its core, an attention function maps a query and a set of key/value pairs to an output, computed as a weighted sum of the values, with weights determined by a compatibility function between the query and each key. As illustrated in Figure 3, this mechanism is implemented in the multi-head attention blocks of the Transformer architecture, where each token can attend to all others in the sequence. Additionally, since self-attention does not capture the order of tokens at first, positional encodings are added to the input embeddings to provide a sense of sequence. These encodings introduce information about each token's position, enabling the model to distinguish between inputs like "Augusto likes Vinícius" and "Vinícius likes Augusto" (VASWANI et al., 2023).

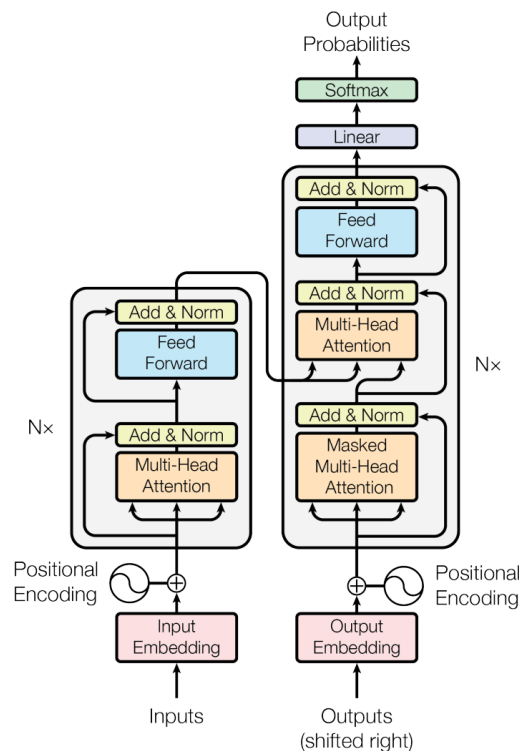


Figure 3 – Transformer architecture. Source: (VASWANI et al., 2023).

Beyond being more efficient than recurrent networks, self-attention also allows for better parallel processing and makes it easier for the model to learn relationships between distant words in a sequence. Instead of passing information step by step like in RNNs, self-attention connects all words directly, shortening the "path" between them (VASWANI et al., 2023). This makes the mechanism especially powerful for natural language processing tasks, where understanding the full context of a sentence is essential, even when key words are far apart.

2.2.2 Natural Language Processing (NLP)

NLP is the field that focuses on enabling machines to understand, interpret, and generate human-like language. It all began in 1966 with the development of ELIZA at MIT, a system that mimicked a psychotherapist using a rule-based approach (WEIZENBAUM, 1966). While rule-based and symbolic methods were the foundation of early systems, the field gradually shifted toward statistical approaches in later years. Today, modern models have advanced, employing DL techniques and Transformer-based architectures to extract information from vast amounts of data (SUPRIYONO et al., 2024). Since language is inherently ambiguous and context-dependent, these models must process input in a way that captures both structure and meaning, forming the foundation for tasks such as question answering, and text summarization (SUPRIYONO et al., 2024).

Before any language model can make sense of text, the input must be broken down into smaller components called **tokens**. Tokenization is the process of segmenting raw text into these units. Modern NLP systems address this with subword tokenization, splitting words into more frequent fragments. Techniques such as Byte Pair Encoding (ZOUHAR et al., 2024) and WordPiece (DEVLIN et al., 2019) build a flexible vocabulary of subword units that balance efficiency and coverage. For example, the word “unbelievable” might be split into “un”, “believ”, and “able”. This allows the model to handle new or rare words by combining familiar parts. Tokenization is thus a key step in preparing text for DL models (MIELKE et al., 2021).

After tokenization, additional preprocessing steps are often applied to prepare the text for more efficient and meaningful analysis. One common step is the removal of stopwords, such as “the,” “is,” and “and” that often carry little semantic weight on their own (SUPRIYONO et al., 2024). This step is helpful in simpler NLP tasks such as document classification, where common function words do not contribute significantly.

Another common pre-processing technique involves reducing words to their root or canonical form. Stemming does that by removing prefixes and suffixes to reduce a word to its base form (e.g., "running" becomes "run"). Lemmatization, on the other hand, transforms words into their dictionary forms, considering context and part of speech (e.g., “better” becomes “good”). Both approaches aim to normalize word forms, allowing the

model to treat different variations for the same word (SUPRIYONO et al., 2024; MIELKE et al., 2021). In Figure 4, a simple example of tokenization with stopwords removal and stemming can be seen.

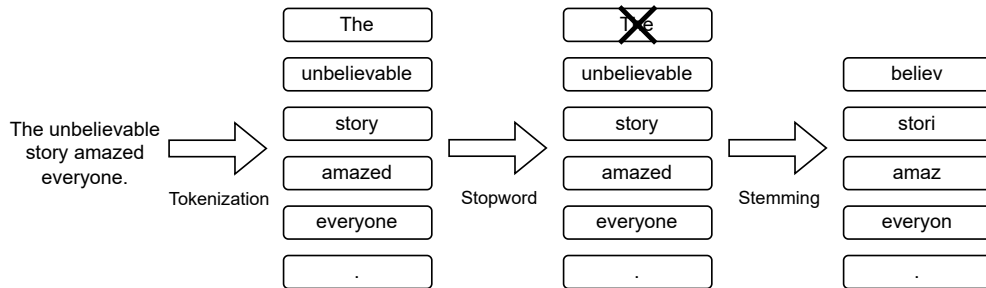


Figure 4 – Example of techniques used in NLP.

2.2.3 LLM Architecture

The architecture is an important feature of LLMs, as it helps us understand how they process and generate text. There are three main architectures commonly used in LLMs: Encoder, Decoder, and Encoder-Decoder (HUANG et al., 2024).

- **Encoder:** intended for scenarios where input processing is the most relevant part of the model, like text categorization, it uses self-attention mechanism to process the context (RAFFEL et al., 2023).
- **Decoder:** divided into two, Causal and Prefix Decoder, both focus on the predicting of the next output token in a sequence. The first uses only previous tokens for predicting and, the second, uses bidirectional attention mechanism (RAFFEL et al., 2023). This architecture framework is being used in most text generation models.
- **Encoder-Decoder:** originally used in sequence-to-sequence (Seq2Seq) models based on RNNs and LSTM (HOCHREITER; SCHMIDHUBER, 1997), but modern architectures, such as the Transformer (VASWANI et al., 2023), have become the standard for this structure. The encoder module extracts the main features and semantic attributes from the input text, while the decoder module sequentially generates output based on the context processed by the encoder.

Several commonly used models illustrate these three architectures. BERT (Bidirectional Encoder Representations from Transformers) is a pure encoder model designed for tasks like sentence classification, named entity recognition, and question answering (DEVLIN et al., 2019). GPT-3, on the other hand, is a causal decoder model specialized in autoregressive text generation tasks such as summarization, dialogue, and code completion

(RADFORD et al., 2019). Finally, BART (Bidirectional and Auto-Regressive Transformers) is an example of encoder-decoder model that perform well in Seq2Seq tasks, including translation and summarization (LEWIS et al., 2019).

Additionally, LLMs are typically pretrained on large-scale corpora in an unsupervised or self-supervised fashion. They are then fine-tuned on more specific datasets to adapt to particular downstream tasks. For instance, in (SINGHAL et al., 2023), the PaLM model was fine-tuned on medical data to create Med-PaLM, which was evaluated on the MultiMedQA benchmark. As shown in Figure 5, the fine-tuned model significantly reduced the performance gap between the raw LLM and real physicians in medical question answering. To highlight the impact of domain-specific fine-tuning, Flan-PaLM, a general purpose, instruction-tuned variant of PaLM, was included as a baseline in the evaluation.

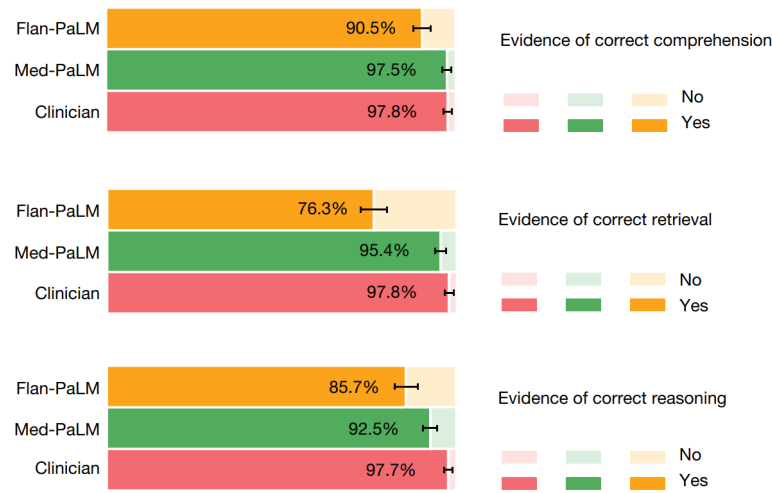


Figure 5 – Evaluation of Flan-PaLM, Med-PaLM, and human clinicians across three metrics: comprehension, retrieval, and reasoning. Source: (SINGHAL et al., 2023).

2.2.4 LMM Architecture

Large Multimodal Models (LMMs) require different technical approaches, as they need to process not only text but also images, audio, and video (HUANG et al., 2024). Currently, the architecture of most LMMs incorporates some, though not necessarily all, of the following five components (see Figure 6):

1. **Multimodal Encoder:** Extracts information from various input sources supported by the model. Examples include image encoders (e.g., ViT (DOSOVITSKIY et al., 2021), CLIP ViT (RADFORD et al., 2021), OpenCLIP (CHERTI et al., 2023)) as well as video and audio encoders;
2. **Input Modal Aligner:** Bridges the gap between non-text features and textual tokens, facilitating the fusion of different modalities (CHEN et al., 2024a), like

Q-Former (LI et al., 2023), Cross-Attention Layer, and Multilayer Perceptron (MLP) (ALAYRAC et al., 2022);

3. **Pre-trained LLM:** Serves as the backbone of the model, leveraging the capabilities of pretrained language models to interpret and generate information;
4. **Output Modal Aligner:** Performs the inverse function of the input aligner, converting the unified representation from the LLM back into a format suitable for decoding into the target modality (CHEN et al., 2024a);
5. **Multimodal Decoder:** Generates the final output decoding features into the requested modality, such as images, or video.

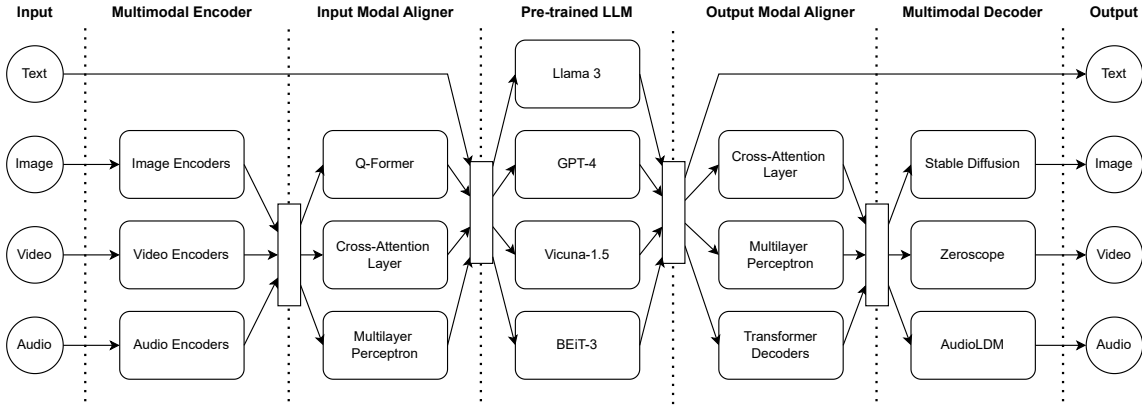


Figure 6 – Architecture of a typical Large Multimodal Model (LMM), including components such as encoders, aligners, and decoders. Adapted from (HUANG et al., 2024; CHEN et al., 2024a).

Not all LMMs follow this exact architecture, it varies depending on the chosen model design or even the goal for the model. For instance, BLIP-2 (LI et al., 2023), for example, has the capability of only generating text, so it does not have the fourth nor fifth step of the architecture mentioned. Also, Generative Adversarial Networks (GOODFELLOW et al., 2014), which consist of a generator and a discriminator trained in opposition to one another, do not adopt the structure described above.

2.2.5 Benchmarks

Benchmarks are synthetic tools used to evaluate model performance under specific conditions. They allow for the comparison of models with different architectures and help reveal the strengths and weaknesses of LLMs (LI et al., 2024). Currently, benchmarks for LMMs cover various aspects of reasoning, understanding, and perception, enabling users to objectively choose models that best suit their needs.

While benchmarks can provide valuable insights into model capabilities, it is important to recognize that they often diverge from real-world behavior. In particular,

recent studies have highlighted the issue of Benchmark Data Contamination (BDC), where benchmark items unintentionally appear in the training data of LLMs. This contamination introduces biases that compromise the evaluation results, especially in scenarios that are intended to reflect zero-shot performance (XU et al., 2024). Consequently, relying on a single or solely on traditional benchmarks may yield unrealistic assessments, highlighting the need for diverse and more robust evaluation methods.

2.3 Embeddings

In this section, we will define what embeddings are, and then we will discuss how the correctness of grammar and language can affect the use of embeddings.

2.3.1 Overview

Embeddings are a concept that is mainly used in machine learning tasks relating to natural language processing (NLP). They are a way to represent items like words, sentences, documents, or even images as a set of real vectors. Thus, they project objects from a high-dimension space to a lower-dimensional one, while trying to keep the semantic information and relations intact. This allows capturing semantic relationships more easily between different items both in formats or sizes (PATIL et al., 2023).

Modern embeddings utilize dense vectors, which compress information into a lower-dimensional space consisting mostly of non-zero real numbers. This dense format is efficient and allows semantic relationships to be distributed across dimensions, enabling the capture of nuanced similarities that sparse representations often miss. In contrast, sparse vectors, typical of traditional techniques like TF-IDF (AIZAWA, 2003), are high-dimensional and dominated by zero values, where each dimension often corresponds to a specific word in the vocabulary.

Figure 7 illustrates how both text and images can be embedded into the same vector space using dense representation. As shown, similar concepts (e.g., the word “king” and its emoji) are closer in values, while semantically distant items (e.g., a smiley emoji or a dog) appear farther apart. This visual proximity reflects their semantic similarity within the embedding space.

In this particular example, we could interpret each dimension of the output vectors as a specific semantic feature. For instance, one could imagine that the first dimension corresponds to “royalty”, the second to “social status”. While in real models these axes are not directly interpretable, this helps to conceptually understand how different types of items can be embedded and compared in the same space.

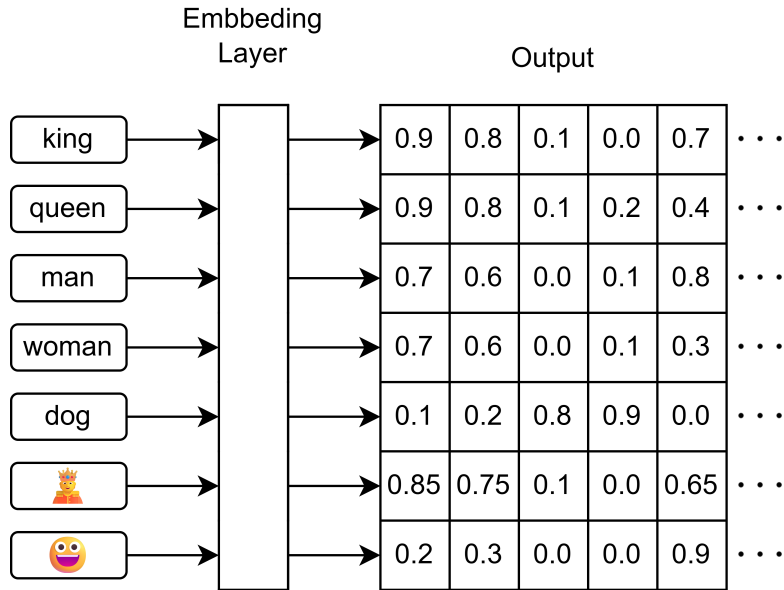


Figure 7 – Representation of text and image embeddings in a shared vector space

2.3.2 Architecture

Early embedding approaches were based on statistical or rule-based techniques, such as co-occurrence matrices or dimensionality reduction methods like Latent Semantic Analysis (PATIL et al., 2023). These methods captured basic word associations but lacked contextual understanding.

Modern approaches use neural networks to automatically learn embeddings from large text corpora. These can be broadly categorized into feature-based models, which generate reusable representations (e.g., Word2Vec (MIKOLOV et al., 2013)), and fine-tuning-based models (e.g., BERT (DEVLIN et al., 2019), GPT (OPENAI, 2024)), which are pre-trained using self-supervised tasks and then adapted to specific downstream applications (PATIL et al., 2023).

In particular, transformer-based models such as BERT produce contextual embeddings, where the same word may have different vector representations depending on the surrounding text (DEVLIN et al., 2019). This has proven to be significantly more effective for capturing nuance and semantic relationships than static embeddings. More recent models, like CLIP (RADFORD et al., 2021), go further by embedding entire sentences or text-image pairs into a shared vector space using contrastive learning objectives. These approaches are especially relevant for tasks requiring cross-modal alignment or semantic similarity at the sentence level, which are core to this project.

2.3.3 Multi-Language Embeddings

Most modern embedding models are designed to be able to understand multiple languages. However, their effectiveness can vary depending on how they were trained

(KAKARLA; VENKATA; GADDAM, 2023). Models can either be trained on the same text translated to different languages, which would allow them to share an embedding space where words with similar meanings are close together no matter the language, or they can be trained by using pre-trained models for each language, and combining them together. The factors that affect the quality of multilingual models are the number of languages and the relatedness of these models (XU, 2025), the training data, and the model architecture.

2.3.4 Grammatical Errors

Word embeddings are not immune when it comes to dealing with grammatical errors in the input text. In fact, it is shown that BERT’s sentence embeddings can capture syntactic relationships (MERCHANT; LI; HOCKENMAIER, 2020). Knowing this, it is apparent that any sort of grammatical error would affect the output of the embeddings. Thus, any application that requires manually written text to be embedded should employ some sort of grammar check to ensure correctness.

There are many tools available that can correct the spelling and grammar of a text. In Python for example, there are packages such as *language_tool_python*, or *pyspellchecker*. These are rule-based and statistical approaches respectively. They can detect spelling mistakes in the input string, and provide suggestions for corrections.

However, tools like these are not that powerful when compared to ML approaches. For example, there are Seq2Seq models, which are usually based on RNN (MELIS; DYER; BLUNSOM, 2017). It can effectively correct a set of input strings. Furthermore, Neural Language Models can also be used for this task. They work by identifying words that have a low probability of appearing in a given context, which would usually mean that they contain some sort of error. Known uses of Neural Language Models include BERT and GPT models.

2.3.5 Text-First Embedding vs. Direct Image Embedding

While both images and text can be embedded into a shared vector space, the choice of modality preprocessing has significant implications on performance, especially in tasks involving semantic similarity, like the one in this project. In particular, recent studies have shown that generating textual representations (via image captioning, OCR, or scene description) and then embedding that text can be more effective than directly embedding the raw image (LI et al., 2022b; LI et al., 2023; GORDO; LARLUS, 2017).

There are several key reasons why this “text-first” approach is advantageous:

- **Semantic Precision:** Generated text captures high-level abstract concepts and

relations present in an image (e.g., “a dog sitting on a sofa next to a window”), whereas image embeddings may focus more on features such as texture, or color.

- **Alignment with NLP Tasks:** When the goal is semantic comparison based on meaning, using a textual intermediary ensures better compatibility with language models and text-based embeddings such as BERT, GPT, or Sentence Transformers (DEVLIN et al., 2019; RADFORD et al., 2021).
- **Multimodal Learning Efficiency:** In cross-modal settings, contrastive models like CLIP (RADFORD et al., 2021) and BLIP (LI et al., 2023) have shown improved performance when textual captions are explicitly used in training, allowing better generalization across domains.

According to Gordo & Larlus (2017), embeddings guided by human-generated or auto-generated captions achieve significantly higher retrieval accuracy than those based solely on vision. Similarly, caption-enhanced models have shown improved performance in zero-shot and few-shot scenarios (FANG et al., 2022; LI et al., 2022a).

Therefore, for semantically rich tasks generating text from visual input and then embedding that text is generally have higher performance than using vision encoders alone.

2.4 Vector Database

A vector database is a specialized system designed to store and retrieve high-dimensional vectors efficiently. These vectors, typically embeddings generated by neural networks, represent data such as text, images in a way that similar items are close together in the vector space. Vector databases enable similarity search, allowing to retrieve the top most similar to a given query based on metrics like cosine similarity, Euclidean distance or inner product (JOHNSON; DOUZE; JéGOU, 2017).

Unlike traditional databases, which perform exact matches on structured fields, vector databases focus on approximate nearest neighbor search (ANNS). This is essential in real-world applications where semantic similarity is more valuable than exact match. For instance, to retrieve artworks thematically similar to a query sentence, ANNS provides an efficient and relatively accurate tool, depending on the database structure (DOUZE et al., 2025).

One of the most widely used tools for vector search is Facebook AI Similarity Search (FAISS). It is a C++ library with Python bindings, supporting indexing methods, including brute-force, clustering-based, and graph-based techniques. FAISS is not a database per se, it does not support concurrent writes, or metadata management, but serves as the core

vector indexing and search engine behind many modern vector database systems (DOUZE et al., 2025).

2.4.1 FAISS

For smaller datasets, brute-force search is a simple and effective baseline. This method guarantees exact results and is computationally feasible on CPUs for small to medium datasets. FAISS optimizes this through efficient distance computations (matrix multiplication for batched queries) and heap structures for top- k selection (DOUZE et al., 2025).

However, as the number of queries or search throughput increases, non-exhaustive methods become more interesting. FAISS offers (Facebook AI Research, 2024):

- **IndexFlat**: Brute-force exact search and it is ideal for small datasets and CPU-only environments.
- **IndexIVF (Inverted File Index)**: This method splits the dataset into several clusters using a rough quantizer (a simple way to group vectors). At search time, instead of comparing the query with every vector in the dataset, FAISS only looks into the most relevant clusters, reducing the number of comparisons.
- **IndexHNSW**: As described by Malkov & Yashunin (2018), this graph-based structure offers strong speed–accuracy trade-offs for medium-scale datasets and is known as Hierarchical Navigable Small World.
- **IndexPQ / IndexIVFPQ**: Compresses vectors via Product Quantization for memory-efficient approximate search.

Index considerations

FAISS provides both CPU and GPU implementations to accommodate various dataset sizes and performance requirements. The choice between CPU and GPU depends on factors such as dataset size, query batch size, latency requirements, and available hardware resources (DOUZE et al., 2025).

The CPU version of FAISS is versatile and supports a wide range of index types, including brute-force search (IndexFlat), inverted file systems (IndexIVF), and quantized indexes like IndexIVFPQ (DOUZE et al., 2025; Facebook AI Research, 2024). It is particularly well-suited for small to medium-sized datasets, especially when the dataset fits in main memory, performance is not a strict requirement, or GPU resources are unavailable.

FAISS’s GPU implementation uses the parallel processing capabilities of GPUs to accelerate similarity search, making it effective for large-scale datasets and high-throughput

applications (DOUZE et al., 2025; Facebook AI Research, 2024; JOHNSON; DOUZE; JéGOU, 2019). It often achieves speedups of 3x to 10x compared to CPU implementations, especially when using large query batches that utilize the GPU’s parallel architecture. However, transferring data between CPU and GPU memory can introduce latency, so it’s more efficient to transfer the index once and issue many queries to help with this overhead.

In summary, the CPU implementation is more interesting for smaller datasets or resource-constrained environments, offering simplicity and lower overhead. The GPU implementation, on the other hand, is better suited for large-scale, performance-critical scenarios, provided that its memory and transfer constraints are taken into account.

2.4.2 Qdrant

Qdrant is a modern, open-source vector search engine and full-featured vector database built for scalable. Unlike libraries such as FAISS that focus only on vector indexing, Qdrant integrates storage, indexing, filtering, and retrieval into a single system.

2.4.2.1 Overview and Architecture

Qdrant is written in Rust for high performance and offers REST APIs for integration with modern ML pipelines. It stores both vector embeddings and structured metadata (payloads), allowing combined filtering and semantic search in real-time.

Its architecture centers on efficient in-memory and persistent storage of high-dimensional vectors HNSW graphs as the default indexing method. This makes it highly performant for large-scale ANNS, especially in high-dimensional spaces (MALKOV; YASHUNIN, 2018).

2.4.2.2 Semantic Retrieval

Qdrant supports many features that make it ideal for applications where semantic meaning, context, and metadata play critical roles:

- **Filtering with Metadata:** One of Qdrant’s features is its support for metadata-aware filtering. You can query vectors not only by similarity but also using constraints like tags, categories, or timestamps.
- **Updates:** Vectors and payloads can be inserted, updated, or deleted without reindexing the full collection, allowing for dynamic, applications such as personalized recommendations or adaptive search.
- **Multilingual and Custom Scoring:** Enables custom similarity scoring strategies and supports multilingual embedding spaces through integration with language models.
- **Ease of Use:** Comes with pre-built client SDKs for Python, Rust, JavaScript, and more, as well as an intuitive dashboard and Docker-based deployment.

2.4.2.3 Use Cases & Features

Qdrant has been widely adopted in both production systems and research contexts due to its efficiency and flexibility in handling high-dimensional embeddings. It powers a range of applications, such as semantic retrieval of documents, images, and FAQs, where results are based on meaning rather than exact keyword matches, and recommendation systems that suggest products, content, or services by leveraging user embeddings and item metadata. Beyond commercial applications, Qdrant is also used in cultural and academic domains to index semantically rich resources with multilingual or annotated metadata, enabling more intuitive and context-aware search and retrieval across large, diverse datasets (BORISOV; GARKUSHA, 2023; TEAM, 2023). Some features that are interesting for this project included in Qdrant project:

- **Persistent Storage:** Built-in disk persistence for vectors and payloads.
- **HNSW Indexing:** Accurate and fast vector search using graph-based indexing (MALKOV; YASHUNIN, 2018).
- **Scalability:** Supports replication, distributed deployment, and multi-tenancy.
- **Cloud/On-Prem Deployment:** Flexible hosting, including Kubernetes and Docker environments.

2.4.3 Comparison

Unlike FAISS, which is primarily an in-memory indexing library without database features, Qdrant is a full vector DBMS. Here's a quick comparison:

Feature	Qdrant	FAISS
Vector Storage	Yes (persistent)	No (external needed)
Metadata Filtering	Yes	No
Indexing Algorithms	HNSW, Flat	Flat, IVF, HNSW, PQ
API Access	REST, gRPC	Python/C++ only
Production Ready	Yes	Requires integration effort

Table 1 – Comparison between Qdrant and FAISS

In summary, Qdrant offers a complete ecosystem for deploying semantic search at scale, combining performance, flexibility, and ease of integration. Its filtering and structured payload support make it particularly well-suited for applications that go beyond pure similarity search and require deeper context awareness.

3 Methodology

To develop the ArtEvoke platform prototype, the methodology is structured around three main components: **Art Feature Extraction**, **User Interaction**, and the **Clinical Evaluation Framework**.

The first component focuses on extracting semantic features from the dataset’s artwork. This process includes people analysis, scene understanding and emotion detection to generate comprehensive image descriptions. These features allow the platform to gather meaningful context about each piece, enabling the system to effectively retrieve and align visual content with the patient’s personal content.

The second component addresses the design of the user experience and system logic. It defines how users interact with the software through two distinct modes: *Memory Reconstruction* and *Art Exploration*. Each mode is subdivided into specific workflows designed to stimulate cognitive engagement.

The third component, a core focus of this phase of the project, establishes the Session and Evaluation Framework. This module allows patients to interact with the platform within a structured clinical session. Following the interaction, the system facilitates a feedback mechanism comprising specific questions. This approach not only provides quantitative data regarding the patient’s engagement and cognitive response but also incentivizes attention and interaction during the session.

Finally, this chapter details the criteria and benchmarks used to guide the selection and evaluation of the LMMs, LLMs and embedding models integrated into the platform. It also describes the vector database indexing architecture and the text segmentation strategies used to optimize the matching of stories to images.

3.1 Art Features Extraction

In this component, we aim to extract features from the art pieces in our dataset to gain a better understanding of the context or scene depicted in the paintings. Figure 8 shows the initial architecture designed to extract features from the art pieces and to generate a description of the art piece.

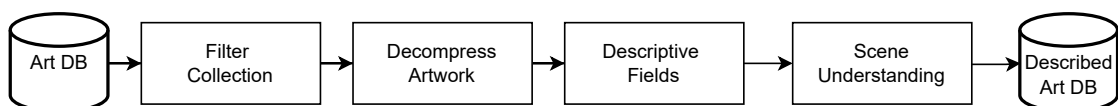


Figure 8 – Art Features Extraction Process

The process follows these main steps:

1. **Filter collection:** We start with a dataset of art pieces from different artistic movements. For this application, we focus only on styles with realistic elements, since the ultimate goal is to match these artworks with real-life stories. The datasets used are discussed in the Section 5.1.
2. **Decompress Artwork:** To reduce processing time and lower computational cost, the selected images are decompressed to a default, lower resolution.
3. **Descriptive Fields:** In some of the datasets used in the project, fields containing descriptive information are available. When present, these fields are incorporated into the scene understanding process, as they can enhance the quality of the generated descriptions.
4. **Scene Understanding:** The images are analyzed using a scene understanding module. One or more Large Multimodal Models (LMMs), as discussed in Section 3.4.2, are tested to identify which model provides the most informative and coherent descriptions while maintaining reasonable performance.

An additional strategy initially considered was increasing the image resolution when the description quality was below the threshold. However, after testing this approach with different LMMs, we observed that raising the resolution did not lead to a significant improvement in the output descriptions, as it will be discussed in the Section 5.2.2.4. As such, this method was discarded in favor of more efficient alternatives.

It is important to note that this step only needs to be run once per artwork. Therefore, even though running LMMs might be computationally expensive, the overall computational cost is not very high.

For the development of the **retrieval system**, we adopted the workflow illustrated in Figure 9.

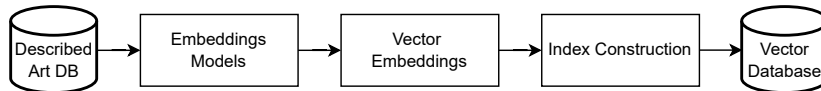


Figure 9 – Vectorization and Indexing Workflow

As shown in the diagram, once the art descriptions are generated and stored, they are passed through embedding models to create dense vector representations. These vectors are then indexed and stored in a dedicated vector database.

To select the most appropriate embedding model, the different models presented in Section 3.4.3 will be evaluated. Each model will be assessed in terms of computational

efficiency, storage footprint, and retrieval effectiveness to determine the most suitable option for the platform. In parallel, we will also test different indexing configurations to optimize search speed, memory usage, but also overall usage, testing FAISS and Qdrant options. These tests help us balance recall performance and latency, ensuring that the retrieval system remains scalable and responsive as the database grows.

With these descriptions, vectors, and the database in place, the platform can move from feature extraction to active use. Descriptions of each art piece can now serve as inputs for the interactive modes, allowing the system to retrieve relevant artworks based on user-provided texts, as will be detailed in the next section.

3.2 User Interaction

As mentioned earlier, the interaction between the user and the platform can be done in two separate modes: **Memory Reconstruction** and **Art Exploration**.

3.2.1 Memory Reconstruction

The first mode of interaction is called **Memory Reconstruction** mode. The idea is to select a sequence of art pieces that closely correlates with the story provided by the user. Normally, the user, the caregiver or the patient themselves, is asked to write a story about a past experience or a familiar event. With the help of our platform, a sequence of artworks will be selected to help trigger the memory and stimulate different parts of the brain (ZAIDEL, 2010).

As shown in Figure 10, the workflow follows a structured process:

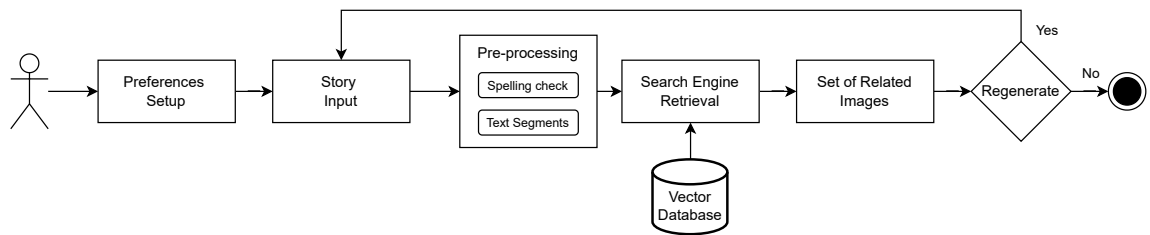


Figure 10 – Memory reconstruction mode steps

The details of each step are described below:

1. **Preferences Setup:** Before entering the story, the user selects the preferred language and the specific dataset to work with. This ensures a personalized and context-relevant experience.
2. **Story Input:** The user provides a story about a past experience or familiar event. This input can be entered manually or dictated via a speech-to-text interface, which

automatically transcribes the spoken narration. This story serves as the foundation for retrieving meaningful artwork that may help stimulate memory.

3. **Pre-processing:** Once submitted, the text undergoes automatic pre-processing. This includes correcting grammar mistakes, which could negatively impact model performance (MERCHANT; LI; HOCKENMAIER, 2020). The story is then segmented into semantically significant parts, studied in the next Chapter, to allow more targeted retrieval of visual content.
4. **Search Engine Retrieval:** Each text segment is converted into an embedding and compared with the stored descriptions in the vector database. This search engine retrieves the most relevant artwork for each segment based on similarity.
5. **Result Evaluation:** The resulting sequence of images is displayed to the user. These images are selected to best match the textual input and are intended to spark memory recall. For each text segment, the top-6 retrieved images are chosen.
6. **Optional Regeneration:** If the user recalls additional details or is not fully satisfied with the results, they can either provide more input or regenerate the output. If the story remains unchanged, the platform will select other art works. However, if any changes are made to the story, a new search is performed. This iterative process allows users to gradually reconstruct richer and more complete memories.

This iterative interaction is grounded in the idea that users may only retain fragments of a past memory or experience. By presenting them with relevant artworks throughout the process, the platform helps reconstruct these fragmented memories and encourages deeper recall.

3.2.2 Art Exploration

In this interaction mode, called **Art Exploration**, we allow an AI model to generate a story based on artworks selected by the user. The goal is to help the patient form connections between different art pieces, which may evoke forgotten memories. Figure 11 presents the workflow that illustrates this mode.

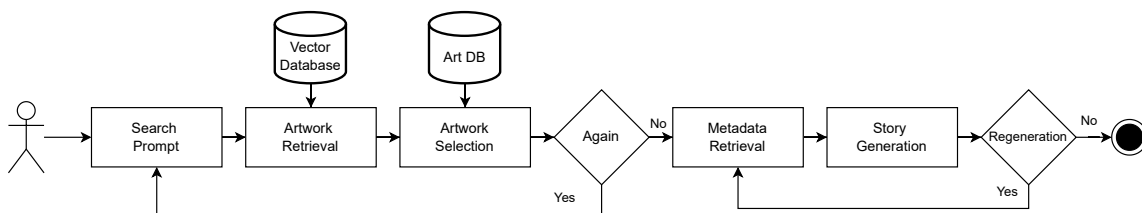


Figure 11 – Art exploration mode steps

The workflow is structured as follows:

1. **Search Prompt:** The user provides a brief input to the system, this can be a word, phrase, or short description. This differs from the longer narrative used in the Memory Reconstruction mode.
2. **Artwork Retrieval:** A search engine queries the vector database based on the user's input and returns a selection of paintings from the art database that best match the prompt. A simplified, faster search engine may be used here to enhance interactivity.
3. **Artwork Selection:** The user selects one or more artworks from the retrieved set. If they are not satisfied, they may refine the search by submitting new search terms, and adding more images to their artworks selection.
4. **Metadata Retrieval:** Once images are selected, their descriptions are retrieved to provide context for the next step.
5. **Story Generation:** An LLM generates a coherent story based on the selected artworks and any integrated metadata.
6. **Result Regeneration:** The user reads the generated story. If the result is not satisfactory, they may choose to regenerate the story or end the process.

This mode encourages creative exploration and helps build connections between artworks and personal meaning, potentially stimulating memory through narrative construction.

3.3 Clinical Evaluation Framework

A core contribution of this thesis is the development of a structured framework enabling healthcare professionals to monitor patient progress and evaluate cognitive responses. This framework serves a dual purpose: it provides a continuous longitudinal monitoring tool for patient care and establishes a standardized method for future clinical trials to validate the platform's efficacy.

The evaluation process is designed to occur after a patient completes an interaction mode (Memory Reconstruction or Art Exploration). To ensure clinical relevance, the framework captures both objective metrics (such as time and accuracy) and subjective feedback regarding the patient's perception of the artworks.

3.3.1 Memory Reconstruction

Following the completion of the Memory Reconstruction mode, where the patient has associated personal narrative segments with specific artworks, the system transitions to the evaluation phase. Figure 12 illustrates this workflow.

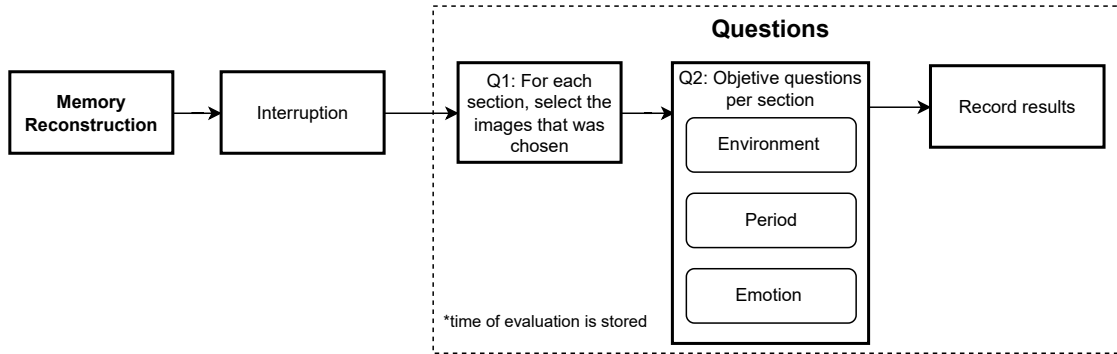


Figure 12 – Evaluation workflow for the Memory Reconstruction mode

The process consists of the following stages:

1. **Interruption:** A deliberate pause is introduced between the interaction and the evaluation. This step allows the patient to disengage from the creative task and prepare for the assessment, like standard cognitive testing protocols that separate stimulus from recall.
2. **Image Recognition (Q1):** The system presents the user with a set of images for **each section of their story**. The patient is asked to identify which image they originally selected. The images contains the 6 images originally showed and two other distractions. This step assesses immediate visual memory and recognition capabilities.
3. **Objective Assessment (Q2):** For the selected artworks, specific objective questions are posed to understand the patient's connection to the piece. These questions categorize the association based on:
 - **Environment:** Spatial recognition (e.g., indoor vs. outdoor).
 - **Period:** Temporal recognition or association with a specific time in the patient's life.
 - **Emotion:** The affective state triggered by the artwork.
4. **Result Recording:** Finally, all responses are saved alongside the total time taken for the evaluation. These results are stored in the relational database, linked to the specific session ID for future review by the physician.

3.3.2 Art Exploration

The evaluation for the Art Exploration mode differs slightly, as this mode focuses on generating a new narrative based on visual stimuli rather than reconstructing an existing memory. Figure 13 details this process.

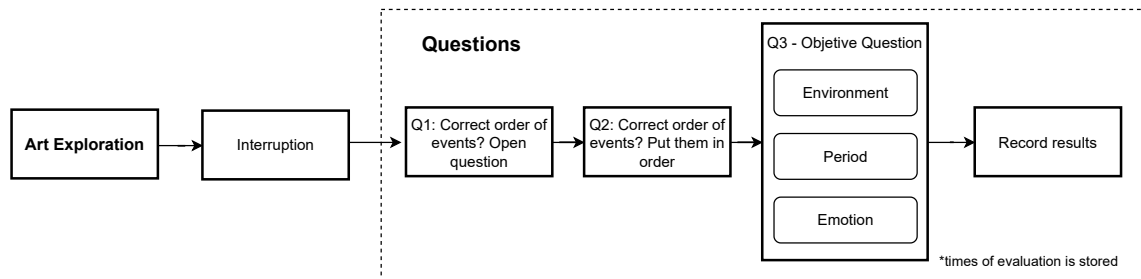


Figure 13 – Evaluation workflow for the Art Exploration mode

The workflow includes the following steps:

1. **Interruption:** Similar to the previous mode, a pause is enforced to clearly demarcate the end of the playful exploration and the beginning of the cognitive assessment.
2. **Narrative Sequencing (Q1 & Q2):** To assess episodic memory and logical sequencing, the patient is tested on the story generated by the AI.
 - **Q1 (Open Question):** The patient is asked an open-ended question regarding the sequence of events in the story.
 - **Q2 (Ordering):** The patient is tasked with arranging the events or selected artworks in the correct chronological order as they appeared in the generated narrative.
3. **Objective Contextualization (Q3):** The patient answers objective questions regarding the attributes of the selected art (Environment, Period, and Emotion). This helps determine if the patient can retain and articulate the semantic details of the visual stimuli they just explored.
4. **Result Recording:** The accuracy of the sequencing, the answers to objective questions, and the evaluation timestamps are persisted in the database to track cognitive engagement levels over time.

3.4 Models Selection

3.4.1 Benchmarks

In this project, as mentioned, our model is expected to retrieve descriptions of art pieces, providing outputs that combine both objective and subjective elements in text format. After investigating widely used benchmarks in academia and the market, the following were chosen, in order of importance. Their use, despite the limitations previously discussed, provides a useful basis for initial comparison. For evaluating LMMs, the following benchmarks were used:

1. **Massive Multi-discipline Multimodal Understanding (MMMU)**: A multi-modal benchmark designed for college-level understanding and reasoning. It features quizzes and exams from various domains and includes various image formats, ranging from paintings to diagrams (YUE et al., 2024). This benchmark is commonly used for evaluation in recent and older models.
2. **Visual Question Answering (VQAv2)**: this benchmark assesses a model’s ability to interpret visual information and answer to natural language questions, showing its understanding of the connection between visual and textual modalities (GOYAL et al., 2017). Although it has more similarity with our project, the availability of this benchmark is not broad.
3. **MathVista**: This benchmark focuses on seven types of mathematical reasoning, such as algebraic and arithmetic reasoning. It includes tasks like figure question answering and geometry problem solving, all grounded in visual contexts (LU et al., 2023).

For evaluating the quality of the embedding models used in the retrieval phase and for the vector database, the benchmarks considered were:

1. **Massive Text Embedding Benchmark (MTEB)**: This benchmark designed to evaluate the quality of text embeddings across 8 task categories, including classification, clustering, retrieval, ranking, and semantic textual similarity (MUENNIGHOFF et al., 2023). The average MTEB score provides a holistic view of embedding utility across diverse downstream tasks.
2. **ImageNet-1K (Zero-shot)**: A benchmark used to assess a model’s ability to classify images into 1000 predefined categories without fine-tuning. The ImageNet zero-shot accuracy measures how well the model aligns visual and textual representations learned during training (DENG et al., 2009). It provides a signal of the model’s potential for visual understanding quality and text-image alignment.

For selecting the LLM used in the **Art Exploration** mode, only one benchmark was considered, as this component plays a secondary role in the overall platform and does not directly impact the retrieval or image understanding processes. The chosen benchmark was:

1. **Massive Multitask Language Understanding-Pro (MMLU-Pro)**: This benchmark is a variant of the original MMLU, covering a wide range of subjects with multitask generalization (WANG et al., 2024b). It was selected primarily because it is widely adopted and reported across the models evaluated, enabling a direct comparison of their general reasoning capabilities.

These benchmarks typically report accuracy scores ranging from 0 to 1, with 1 representing the best possible performance, usually represented by percentage.

3.4.2 Models - LMMs

After evaluating the requirements, such as access to run the model, number of parameters, and sufficient context token length, Table 2 presents the models selected for testing in this project for creating the description of the images. Annex 23 includes all the models that were considered and researched, covering commercial, restricted and open-source options. All considered models are still supported, available via API or Hugging Face, and accept images as input.

Team	Name	Parameters (B)	Release Date	Context tokens (K)	Max output tokens	Weights Access
LLaVA	1.5	7	09/2023	4.096	Not specified	Open-source
	1.5	13	09/2023	4.096		
	NeXT Vicuna	7	01/2024	4.096		
	NeXT Vicuna	13	12/2023	4.096		
	NeXT Mistral	7	12/2023	4.096		
Llama	3.2 Vision	11	09/2024	128	2.048	Restricted
Qwen	2.5-VL	3	02/2025	32	8.192	Open-source
		7	02/2025			

Table 2 – Overview of evaluated multimodal SoTA models with support for image inputs.

Source: (LU et al., 2024; LIU et al., 2023; LIU et al., 2024; BAI et al., 2025; Meta AI, 2024; Meta AI , 2024; LIU et al., 2025).

As mentioned, we use three benchmarks to guide the model selection process. Table 3 shows the performance of the shortlisted models on MMMU (zero-shot), VQAv2, and MathVista. The table with all of the model benchmarks can be found in Annex 24.

Team	Name	Parameters (B)	MMMU (0-shot)	VQAv2	MathVista
LLaVA	1.5	7	-	78,50%	-
	1.5	13	36,40%	80%	-
	NeXT Vicuna	7	35,80%	81,80%	34,60%
	NeXT Vicuna	13	36,20%	82,80%	35,60%
	NeXT Mistral	7	35,30%	82,20%	37,70%
Llama	3.2 Vision	11	50,70%	75,20%	51,50%
Qwen	2.5-VL	3	53,10%	-	62,30%
		7	58,60%	-	68,20%

Table 3 – Performance of multimodal models on benchmark datasets. Results are reported for MMMU (zero-shot), VQAv2, and MathVista when available. Source: (LU et al., 2024; LIU et al., 2023; LIU et al., 2024; BAI et al., 2025; Meta AI, 2024; Meta AI, 2024; LIU et al., 2025).

These models will be tested and further discussed in the following chapters to determine which one is best suited for integration into the platform.

3.4.3 Models - Embeddings

After evaluating the models listed in Annex 25, considering factors such as embedding dimension, number of parameters, date of release and other factors, Table 4 presents the embedding models selected for testing in this project. These embeddings will be used in two main parts: evaluating the LMM models and evaluating the embeddings for use in the application.

Team	Name	Image size	Embedding Dimension	Parameters (M)	Release Date	Modality	Weights Available
OpenCLIP	ViT-G/14	256px	1024	601	Jul 2023	Image-Text	Open-source
	ConvNeXt-XXLarge	224px	1024	848	Jan 2023		
	ConvNeXt-Large@320px	320px	1024	690	Jan 2023		
Google	ViT-H-14-378 quickgelu (DFN)	378px	1024	632	Aug 2023	Image-Text	Open-source
BAAI	bge-large-en-v1.5	-	1024	335	Sep 2023	Text	Restrictive
Sentence Transformers	all-MiniLM-L6-v2	-	384	22.7	Apr 2019	Text	Restrictive
DAMO Academy	GTE-large	-	1024	335	Aug 2023	Text	Open-source
intfloat	E5-large-v2	-	1024	335	Dec 2022	Text	Open-source
Qwen3	Embedding 0.6B	-	1024	600	Jun 2025	Text	Open-source
Qwen3	Embedding 4B	-	2560	4000	Jun 2025	Text	Open-source

Table 4 – Overview of evaluated embeddings models. Source: (ILHARCO et al., 2021; RADFORD et al., 2021; SCHUHMANN et al., 2022; CHEN et al., 2024b; LI et al., 2023b; WANG et al., 2024a; ZHANG et al., 2025)

Table 5 shows the performance of the embeddings models selected on MTEB (Average) and ImageNet1K (zero-shot). The table with all of the models benchmark can be found in Annex 26.

Team	Name	MTEB (Average)	ImageNet (Zero-shot)
OpenCLIP	ViT-G/14	–	80.1%
	ConvNeXt-XXLarge	–	76.9%
	ConvNeXt-Large@320px	–	79.5%
Google	ViT-H-14-378-quickgelu (DFN)	–	84.4%
BAAI	bge-large-en-v1.5	45.08%	N/A
Sentence Transformers	all-MiniLM-L6-v2	56.26%	N/A
DAMO Academy	GTE-large	63.13%	N/A
intfloat	E5-large-v2	62.25%	N/A
Qwen3	Embedding 0.6B	70.70%	N/A
Qwen3	Embedding 4B	74.60%	N/A

Table 5 – Evaluated embedding models and their benchmark scores. Source: (ILHARCO et al., 2021; Hugging Face,)

3.4.4 Models - LLM

As required in the Art Exploration mode, we integrate an LLM capable of efficiently generating narratives from the pre-generated descriptions of each artwork. Since these descriptions are created in advance using a multimodal model and stored in the database, this step does not require image input. Instead, the focus is on transforming text into coherent narratives while maintaining a runtime that ensures a smooth user experience (UX).

Given these requirements, the evaluated models are presented in Table 6. Annex A.5 provides a complete overview of all models analyzed for this task. As shown, two API-accessible models are under consideration, as their cost and capabilities align with the project’s constraints.

Team	Name	Parameters (B)	Release Date	Context tokens (K)	Max output tokens	Weights Access
Qwen	3	1.7 4	04/2024	32	8.192	Open-source
Microsoft	Phi-4-mini	3.8	02/2025	200	Not specified	Open-source
Google	Gemma 3	4	03/2025	128	8192	Open-source
Meta	Llama 3.2	3	09/2024	128	Not specified	Open-source
Maritaca AI	Sabiá-3	-	09/2024	32	Not specified	Closed
	Sabiazinho-3	-	02/2025			

Table 6 – Lightweight LLMs (<4B parameters) for story generation from text-only input. Source: (YANG et al., 2025; MICROSOFT et al., 2025; TEAM et al., 2025; ABONIZIO et al., 2025)

Table 7 summarizes the benchmark results for the selected models, with a detailed overview included in Annex A.6. Notably, Maritaca AI, a Brazilian team with a strong focus on both English and Portuguese, emphasizes benchmarks in these languages, important

for the project. However, standardized metrics such as MMLU-PRO were not reported for their models.

Team	Name	Parameters (B)	MMLU-Pro
Qwen	3	1.7	36.76
Qwen	3	4.0	50.58
Microsoft	Phi-4-mini	3.8	52.8
Google	Gemma 3	4.0	43.6
Meta	Llama 3.2	3.0	34.3
Maritaca AI	Sabiá-3	-	-
Maritaca AI	Sabiazinho-3	-	-

Table 7 – Lightweight LLMs benchmarks. Source: (YANG et al., 2025; MICROSOFT et al., 2025; TEAM et al., 2025; LIU et al., 2025; ABONIZIO et al., 2025)

These models will be evaluated to determine whether they can effectively perform the relatively simple task of composing short stories based on image descriptions.

3.4.5 Vector Database Selection

To ensure the platform enables efficient and scalable retrieval of artworks, this project conducts a comparative evaluation of two distinct vector indexing solutions: the library-based **FAISS** and the database management system **Qdrant**.

As detailed in Section 2.4, FAISS is primarily utilized in this methodology to benchmark the raw performance of different indexing algorithms. By isolating the indexing mechanism from network and database overhead, we can assess the specific trade-offs between retrieval latency, memory usage, and recall accuracy across different hardware setups. The specific FAISS index strategies selected for algorithmic evaluation are:

- **IndexFlatIP (CPU/GPU)** – Exact brute-force search using inner product. This serves as a baseline for accuracy to determine if approximate methods are necessary for the project’s scale.
- **IndexIVF (CPU/GPU)** – Inverted File index that uses coarse clustering to speed up search (Approximate Nearest Neighbor), testing the trade-off between speed and recall.
- **IndexIVFPQ (CPU/GPU)** – Inverted file index combined with Product Quantization, optimizing for memory efficiency through vector compression.

In parallel, **Qdrant** is evaluated as the candidate for the production environment. Unlike FAISS, which functions primarily as a vector indexing library, Qdrant is tested for

its capabilities as a vector database management system (DBMS). The evaluation focuses on the following qualitative operational features required by the *ArtEvoke* platform:

- **Data Persistence and Management:** Assessing the system’s ability to manage vector storage on disk.
- **Architectural Decoupling:** Evaluating the advantage of using a service-based architecture (via REST API) versus a library-based approach.

This comparative approach allows us to determine whether the raw speed and flexibility of a library (FAISS) outweigh the features and ease of use of a dedicated database (Qdrant) for the final system architecture.

3.4.6 Text Segmentation

To investigate the impact of segmentation on retrieval performance, we designed tests that evaluate how different levels of text decomposition influence the quality of retrieved artworks.

The segmentation process considers multiple levels, ranging from entire paragraphs to individual sentences. Two main segmentation strategies are explored: non-overlapping and overlapping segments. The overlapping strategy uses a sliding window approach, in which a fixed number of consecutive sentences are grouped into a segment, and the window advances (step) by one or more sentences at a time. This technique helps preserve contextual continuity across boundaries and reduces information loss.

For each segmentation level, the text is transformed into embeddings and used to query a vector database of artworks. Each of the levels mentioned is tested both with and without overlap. We evaluate the following segmentation levels:

- **Paragraph-level:** One embedding per paragraph;
- **Fixed-length groups:** 3 sentences, 2 sentences, and 1 sentence.

The goal is to assess how segmentation granularity and context continuity affect semantic alignment and visual narrative quality, allowing to decide which one will be used in the Memory Reconstructions. Two types of evaluation are considered:

- **Quantitative metrics:** cosine similarity between input and retrieved image descriptions, and diversity of retrieved results.
- **Qualitative assessment:** manual inspection of whether the sequence of retrieved artworks tells a coherent and relevant story.

4 Requirements

This chapter details the functional and non-functional requirements established for the ArtEvoke platform. These requirements were derived from the methodological needs of therapeutic interventions for elderly patients with AD and the technical constraints of deploying modern AI models.

4.1 Functional Requirements

The functional requirements describe the specific behaviors and functions the system must support to facilitate the interaction between patients, doctors, and the underlying AI components.

User and Session Management

- **[RF01] User Authentication:** The system must provide role-based authentication for doctors and patients using secure credentials and JWT-based session management;
- **[RF02] Patient Management:** The doctor profile must allow for the registration, viewing, and listing of patients associated with their care;
- **[RF03] Session Management:** Doctors must be able to create structured clinical sessions, selecting the interaction mode (Memory Reconstruction or Art Exploration) and configuring parameters such as interruption intervals.

Memory Reconstruction Mode

- **[RF04] Narrative Input:** The system must allow users to input a personal story or memory via text or voice transcription (speech-to-text);
- **[RF05] Automatic Text Segmentation:** The system must automatically segment the user's narrative into meaningful sections (using sliding windows or paragraphs) for individual processing;
- **[RF06] Semantic Artwork Retrieval:** For each text segment, the system must retrieve and display a set of semantically relevant artworks by querying the vector database;
- **[RF07] Artwork Selection:** The user must be able to select a specific image from the suggested set that best resonates with that segment of their story;

- **[RF08] Result Regeneration:** The system must provide an option to regenerate image suggestions if the initial retrieval results are not satisfactory.

Art Exploration Mode

- **[RF09] Keyword-based Search:** The system must support the search for artworks based on keywords, themes, or emotions input by the user;
- **[RF10] Multiple Artwork Selection:** The user must be able to select multiple artworks from the search results to form a creative basis;
- **[RF11] Story Generation:** The system must utilize a Large Language Model (LLM) to generate a short, coherent narrative that connects the selected artworks.

Clinical Evaluation Framework

- **[RF12] Controlled Interruption:** The system must enforce a pause (interruption screen) between the interaction phase and the evaluation phase to separate the stimulus from the recall task;
- **[RF13] Chronological Ordering Assessment:** In Art Exploration mode, the system must present a task where the patient orders events or images according to the generated story;
- **[RF14] Visual Recognition Task:** In Memory Reconstruction mode, the system must ask the patient to identify, among distractors, which image was previously selected for a specific part of the story;
- **[RF15] Objective and Subjective Evaluation:** The system must present questions regarding the context of the artwork (environment, period, emotion) and allow for the recording of qualitative feedback;
- **[RF16] Progress History:** The system must store and present detailed session history to the doctor, including patient responses, reaction times, and selected artworks.

Internationalization and Accessibility

- **[RF17] Internationalization:** The system must support dynamic language switching between Portuguese (PT-BR) and English (EN), adapting both the interface and the prompts sent to the AI models;
- **[RF18] Text-to-Speech:** The system must offer text-to-speech functionality to read generated stories aloud, supporting patients with reading difficulties.

4.2 Non-functional Requirements

The non-functional requirements define the quality attributes, design constraints, and performance standards necessary to ensure the platform is viable for use with the target demographic.

Usability and Accessibility

- **[RNF01] Cognitive Accessibility:** The user interface must adhere to design guidelines for elderly users, utilizing large typography, high contrast, generous spacing, and linear workflows to minimize cognitive load;
- **[RNF02] Error Prevention:** The system must restrict complex navigation during active sessions to prevent patients from accidentally exiting the therapeutic flow;
- **[RNF03] Clear Visual Feedback:** The system must provide immediate and understandable visual feedback for all actions (e.g., loading indicators during AI generation, selection confirmation).

Performance and Efficiency

- **[RNF04] Response Latency:** Vector search operations and UI transitions must occur with minimal latency to maintain user engagement;
- **[RNF05] Asynchronous Processing:** Embedding generation and heavy image processing must be performed asynchronously (during ingestion or background tasks) to avoid blocking real-time user interaction.

Reliability and Security

- **[RNF06] Data Persistence:** The system must ensure the secure persistence of clinical and session data using an ACID-compliant relational database;
- **[RNF07] Fault Tolerance:** The frontend must implement Error Boundaries to gracefully handle component failures without crashing the entire application;
- **[RNF08] Sensitive Data Protection:** Access to patient data and session records must be strictly controlled via Role-Based Access Control (RBAC) in the backend;
- **[RNF09] Secure Communication:** All client-server communication must be encrypted using HTTPS/SSL.

Architecture

- **[RNF10] Modularity:** The system must follow a service-oriented, containerized architecture (Docker), clearly separating Frontend, Backend, and Database layers;
- **[RNF11] AI Independence:** The integration layer with AI models must be abstracted, allowing for the future replacement of models with minimal code refactoring;
- **[RNF12] Portability:** The application must be deployable across different environments (local development, on-premise servers, cloud) via orchestration tools like Docker Compose.

5 Exploration and Evaluation of AI Models

In this chapter, we will discuss how we developed our solution, and how we implemented it. We will include datasets, techniques and technologies that we have tried even if they are not present in the final design of our system.

All experiments and system components were developed and executed in partnership with the Center for Artificial Intelligence ([C4AI](#)) at the University of São Paulo (USP), using a university-provided Linux server running *Ubuntu 24.04.3 LTS*. The server is equipped with an *AMD Ryzen Threadripper PRO 5975WX* processor (32 cores, 64 threads), 128 GB of RAM, and two *NVIDIA RTX 5000 Ada Generation* GPUs, each with 32 GB of VRAM. The system’s firmware corresponds to *Gigabyte MC62-G40-00 (R06)*. All implementations were developed in *Python 3.12.3*, within a virtual environment managed via *venv*. GPU-accelerated computations, such as model inference and FAISS-based vector indexing, were distributed across both GPUs to reduce execution time and enhance parallelism, unless stated otherwise.

5.1 Datasets

In this section, we will verify the datasets used in the project and how they were treated for each of the tests proposed for the parts of the platform.

5.1.1 SemArt

To evaluate each of the models listed in Table 2, a dataset containing pre-described pieces of art is required. After reviewing options, the SemArt dataset, introduced in (GARCIA; VOGIATZIS, 2018), was selected. This dataset was collected from the Web Gallery of Art¹ to support the study of semantic understanding of fine-art paintings through images and artistic commentary. It includes essential information for each artwork, as it can be seen in Figure 14.

The selected dataset contains over 21,000 images of artworks, with a wide range of artistic movements and themes. Given that the full dataset is excessive for the evaluation phase, a subset of 500 images was sampled, called from now on *SemArt500*. To ensure that LMMs can effectively describe a variety of images, the original distribution across types was not preserved during sampling. Specifically, 50 images were randomly selected for each of the ten different Types², using a *random_state* of 42. Figure 15 illustrates the

¹ Available at [link](#)

² Genre, Historical, Interior, Landscape, Mythological, Portrait, Religious, Still-life, and others

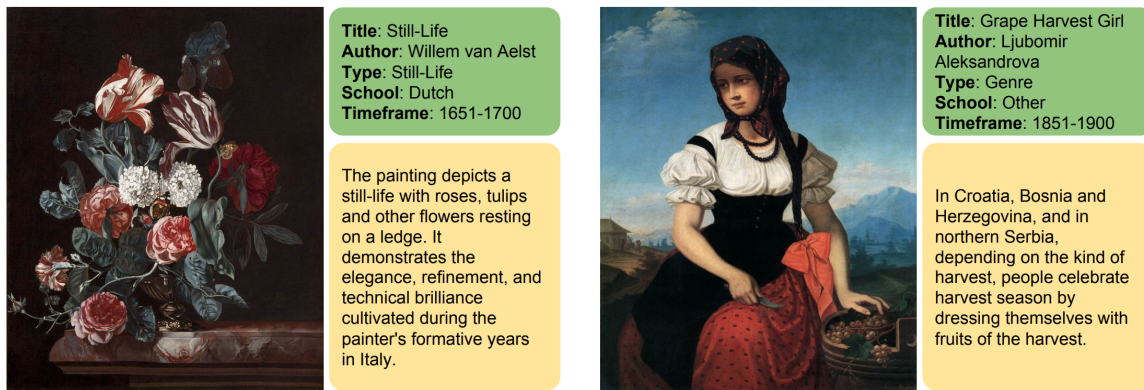


Figure 14 – Information provided in the dataset. Source: (GARCIA; VOGIATZIS, 2018)

distribution in the sampled subset, with Timeframes³ and Schools⁴ grouped for readability.

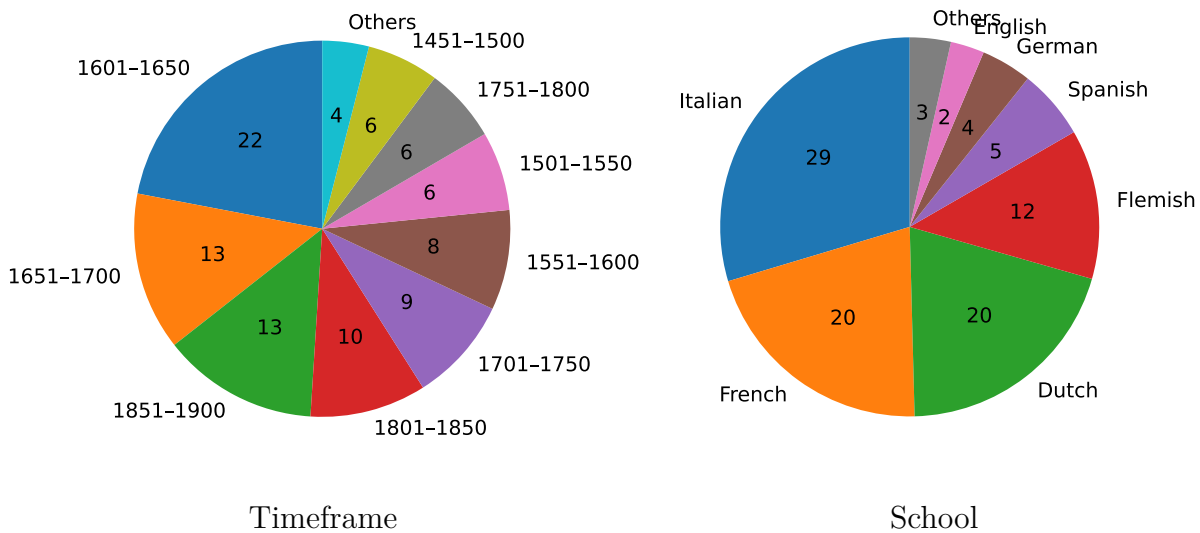


Figure 15 – Distribution of 500 sampled artworks by timeframe (left) and by artistic school/origin (right).

For a broader evaluation of the embeddings discussed in Section 5.3, and following the selection of the final LMM model, a larger subset of images with descriptions was assembled. This expanded set includes the original 500 images, supplemented by 1,500 additional samples randomly selected from the full SemArt dataset, using the same fixed *random_state* of 42. The distribution of the *SemArt2000* is detailed in Annex B.1.

A third dataset, also sampled from SemArt, was specifically prepared to support testing of various FAISS and Qdrant configurations, as described in Section 5.5. To ensure the platform remains scalable, it was important to assess its performance on larger datasets. This dataset will also be used in the final version of the platform, which will be discussed in the next chapter. For this version, we excluded artworks from genres considered less

³ Others: 1401-1450 (9), 1301-1350 (7), and 1351-1400 (4).

⁴ Others: Hungarian (6), Danish (5), Netherlandish (4), Russian (3), Scottish (3), American (2), Austrian (2), Belgian (2), Swiss (1), Norwegian (1), Bohemian (1), and Other (1).

relevant or detrimental to output quality due to their abstract or static nature, in the context of the platform’s two main user modes. The excluded genres were *portrait*, *study*, *still-life*, *interior*, and *other*. After this filtering, the dataset consisted of **15,570** artworks, as can be seen in Annex B.2. In this setup, the original description was included in the model’s prompt to help improve the quality of the generated output by providing the model with contextual information. This dataset will be referred to as *SemArt15000*.

5.1.2 WikiArt

WikiArt is a non-profit initiative aimed at making the world’s art accessible. Hosting over 250,000 artworks by 3,000 artists in more than eight languages, the platform documents pieces from museums, universities, and civic institutions in over 100 countries. The project aspires to cover the entire history of art and is sustained by volunteer contributions, while advertising revenue supports technical and hosting costs (WikiArt, 2024).

Due to restricted access to the official API, we instead used a public sample of the WikiArt dataset available on Kaggle, originally used in the ArtGAN paper (TAN et al., 2019). While the original study employed the dataset for image synthesis tasks, we repurposed it to support both testing and the final deployment of our platform. This sample, referred to here as *WikiArtSample*, contains over 80,000 images organized into artistic genres.

To tailor the dataset to the platform’s goals, we excluded genres that were too abstract or non-representational. Since the platform’s matching mechanism relies on visual storytelling grounded in real-world experiences, abstract artworks tend to lower semantic alignment and overall retrieval quality. After this filtering step, we retained around **51,928** artworks across 14 genres. This sample will be called *WikiArtSampled*. The distribution of images per genre is illustrated in Figure 16, be aware that an artwork can be part of more than one genre.

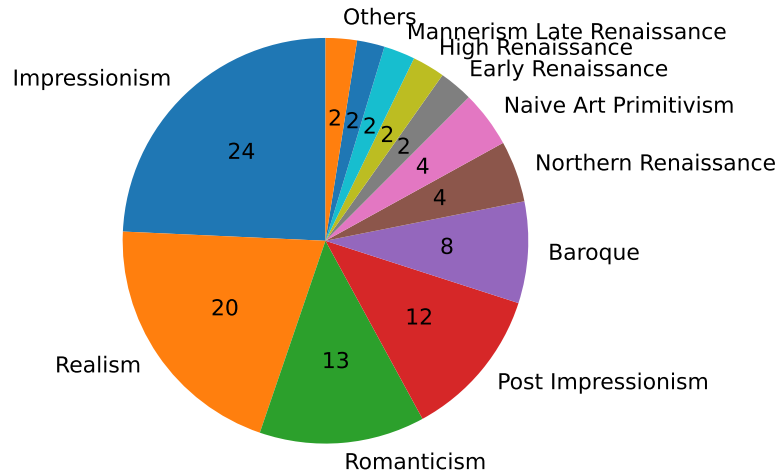


Figure 16 – Distribution of retained artworks in the *WikiArtSample* dataset by genre.

5.1.3 Museu do Ipiranga

The Museu Paulista da Universidade de São Paulo, popularly known as Museu do Ipiranga, is one of Brazil’s most significant historical museums and the oldest public museum in the city of São Paulo. Its collections document material culture, visual memory, and everyday life across different periods of Brazilian history. As part of its institutional mission, the museum aims to disseminate knowledge and broaden public access to its holdings through digital initiatives aligned with contemporary museological paradigms.

To support open access, the museum employs Tainacan, a free and open-source digital collection management system developed by the Laboratório de Inteligência de Redes at the Universidade de Brasília. Tainacan has been adopted by several major cultural institutions in Brazil and abroad, including IBRAM, Funarte, and the Mexican governmental digital culture network.

For this project, we accessed the museum’s digital collection through the official Tainacan public API, which is maintained by the Brazilian Government. This API provides structured metadata for each item in the collection and serves as the sole source of the dataset used in our platform. Importantly, we do not store any images locally. Instead, we only retain the reference URLs provided by Tainacan alongside a selection of metadata fields required for visual storytelling and retrieval tasks. This approach reduces storage overhead and respects institutional policies for digital assets.

Each artwork record in the database stores a minimal set of metadata, including: identifiers (`id`, `external_id`, `inventory_code`), the image reference (`image_file`), basic descriptive metadata (`title`, `description`, `type`, `artist_name`, `location`, `date`, `period`, `technique`), physical attributes (`height`, `width`, `color`), and contextual information (`history`, `collection_alt_name`).

These fields provide a rich semantic foundation for retrieval, contextualization, and narrative alignment within the platform. Since the Museu do Ipiranga collection is primarily historical and documentary in nature, providing representations of daily life, people, spaces, and cultural artifacts from Brazil’s past.

5.2 Evaluation of LMMs

5.2.1 Methodology

As mentioned in Section 3.4.2, several multimodal models were selected for evaluation. The main objective is to find an effective way to describe artworks in text format. This is crucial, as the generated descriptions will be used to match artworks with sections of the story content provided by the user. The evaluation relies on quantifying the similarity between the written description and the image itself. Therefore, selecting the model that

best fits our application is of utmost importance.

5.2.1.1 Evaluation Protocol

To standardize evaluation, the prompt shown in Box 5.2.1.1 was used for all models. This prompt includes an example of a "good description," which serves to guide the model's output toward the desired descriptive style found in the selected dataset. Therefore, we evaluate the models based on their one-shot description generation, using the provided example as context.

Prompt Given

Task: Describe the visual content of the following artwork in detail. Your description should focus only on what is visible in the image, such as people, objects, colors, emotions, actions, setting, and atmosphere. Do not include the title, artist, art movement, or historical context.

Example of a good description: "The lunette on the back wall of the loggia depicts a colorful fruit and vegetable market. The scene is split in two: women run their stalls beneath a tall arcade supported by a striking red pillar. Besides produce, other goods hang from a molding along the back wall."

Now, describe the current image following the same style. Be accurate, descriptive, and grounded in what can be seen.

5.2.1.2 Quantitative Evaluation

To evaluate the quality of the descriptions generated by each model, we used embeddings and compared them using cosine similarity. This metric measures the cosine of the angle between two vectors in an inner product space, producing values between -1 and 1 , where higher values indicate greater similarity. It is computed according to Equation 5.1:

$$\text{cosine_similarity} = \frac{A \cdot B}{|A| \cdot |B|} \quad (5.1)$$

Where:

- A, B : n -dimensional vectors (e.g., model embeddings);
- $A \cdot B$: dot product between vectors A and B ;
- $|A|, |B|$: Euclidean norms of vectors A and B .

In practice, when embeddings are normalized to unit length (i.e., $|A| = |B| = 1$), cosine similarity simplifies to a dot product between the two vectors, making the computation more efficient. In our setup, all embeddings were normalized after being generated by the embedding models.

To assess how well the generated descriptions matched the originals, we used cosine similarity between their respective embeddings. These embeddings were produced using four text-focused models: bge-large-en-v1.5, all-MiniLM-L6-v2, gte-large, and e5-large-v2. We chose these models because they perform well on tasks that measure how well a model captures the meaning of text, as shown by their scores on the MTEB benchmark (see Table 5). Among them, gte-large and e5-large-v2 stood out with the highest scores, at 63.13% and 62.25%. Although all-MiniLM-L6-v2 has a lower score (56.26%), it's known for being fast and lightweight, which makes it a popular choice. We also included bge-large-en-v1.5 for its growing use in multilingual and specialized retrieval tasks. Overall, this mix of models gave us a robust and diverse basis for evaluating how the generated descriptions matched the originals.

This experimental step not only aims to ensure that the chosen resolution does not constrain the model's performance but also supports the proposed Art Feature Extraction workflow. In this workflow, an increase in resolution will be tested to assess whether it improves the quality of the generated descriptions. The complete results of these resolution-based evaluations are presented and analyzed in Chapter 5.2.2.

5.2.1.3 Resolution Analysis Setup

Following the selection of the model most suitable for the application, additional experiments were conducted to assess the potential impact of input image resolution on the quality of the generated descriptions. The underlying hypothesis was that higher-resolution images, by preserving more visual detail, could enhance the model's ability to produce accurate and contextually rich descriptions. To evaluate this, various image resolutions were tested during inference. For each configuration, the generated descriptions were compared to their corresponding reference descriptions using cosine similarity, in accordance with the methodology employed in the primary evaluation.

5.2.2 Results and Analysis

This section presents the evaluation of LMMs, which are responsible for generating artwork descriptions. These descriptions are used in all system functionalities, making this step critical. We evaluate the models based on semantic similarity, runtime, and sensitivity to resolution.

5.2.2.1 Runtime and Resource Usage

After generating all the descriptions, it was possible to extract relevant technical information for each model. Table 8 presents this information, which is considered in the selection of the final model. Memory reserved and memory allocated are key limiting factors that enable (or disable) the parallelization of multiple instances of the LMM, thereby affecting the overall processing speed. The maximum pixel size is also important: higher values enable the analysis of higher-resolution images, which may improve results

but also significantly increase VRAM consumption. Finally, the average processing time per image provides an estimate of the total time required when scaling to larger datasets.

Name	Parameters (B)	Avg Memory Allocated (MB)	Avg Memory Reserved (MB)	Max Pixel	Avg time
LLaVA 1.5	7	14,131.23	15,713.53	336x336	4.22
	13	26,260.52	28,155.96	336x336	6.52
LLaVA NeXT Vicuna	7	13,554.91	20,137.35	672x672 ¹	7.69
	13	25,685.30	35,737.85	672x672 ¹	13.46
LLaVA NeXT Mistral	7	14,963.57	19,195.52	672x672 ¹	7.75
Llama 3.2 Vision	11	20,382.46	28,011.93	560x560	21.04
Qwen 2.5-VL	3	7,183.34	7,903.92	512x512 ²	10.84
	7	22,999.74	24,068.62	512x512 ²	10.68

Table 8 – Summary of runtime specifications for multimodal SoTA models used.

5.2.2.2 Semantic Similarity Results

Table 9 presents the mean, standard deviation, minimum, and maximum cosine similarity scores obtained for the top 3 highest means models across the different embeddings, the full table can be seen in Annex 29. These results provide a general overview of how semantically close the generated descriptions are to the ground truth.

Embedding	Model	Mean	Std	Min	Max
MiniLM-L6-v2	Llama-3.2-11B-Vision-Instruct	0.3737	0.1417	-0.0435	0.7474
	Qwen2.5-VL-3B-Instruct	0.3755	0.1456	-0.0533	0.8427
	Qwen2.5-VL-7B-Instruct	0.3994	0.1435	-0.0672	0.8191
bge-large-en-v1.5	llava-v1.6-vicuna-13b	0.6053	0.0788	0.3713	0.8718
	Llama-3.2-11B-Vision-Instruct	0.6084	0.0797	0.3699	0.8447
	Qwen2.5-VL-7B-Instruct	0.6219	0.0789	0.3839	0.8800
e5-large-v2	Llama-3.2-11B-Vision-Instruct	0.7708	0.0390	0.6497	0.8997
	Qwen2.5-VL-3B-Instruct	0.7700	0.0405	0.6595	0.9161
	Qwen2.5-VL-7B-Instruct	0.7725	0.0407	0.6603	0.9134
gte-large	llava-v1.6-vicuna-13b	0.8375	0.0349	0.7284	0.9477
	Llama-3.2-11B-Vision-Instruct	0.8372	0.0363	0.7192	0.9390
	Qwen2.5-VL-7B-Instruct	0.8434	0.0359	0.7415	0.9637

Table 9 – Summary of the similarity statistics found for the different text embeddings and vision-language models.

¹ The model supports flexible aspect ratios, adapting images into grids based on predefined resolutions like 336×672, 672×672 or 1080x336. Larger images are split into patches of 224×224 during processing.

² The model supports higher resolution and it will be discussed in the next subsection.

5.2.2.3 Statistical Significance

To assess whether the performance differences between the top two models for each embedding method were statistically significant, three tests were conducted: the paired t-test and the Wilcoxon signed-rank test. While the t-test assumes the distribution of differences to be normal, this assumption was not met in any case (as verified by the Shapiro–Wilk test). The Wilcoxon test, valid under symmetric but non-normal distributions, was applicable in all cases. The results can be seen in the Table 10.

Embedding	Compared Models	t-stat (p)	Wilcoxon W (p)	Mean Difference	Better Model
bge-large-en-v1.5	Qwen2.5-7B vs Llama-3.2-11B	$4.3 \cdot 10^{-14}$	$4.2 \cdot 10^{-13}$	0.0135	Qwen2.5-7B
all-MiniLM-L6-v2	Qwen2.5-7B vs Qwen2.5-3B	$5.5 \cdot 10^{-15}$	$1.4 \cdot 10^{-13}$	0.0240	Qwen2.5-7B
gte-large	Qwen2.5-7B vs LLaVA-1.6-Vicuna-13B	$1.0 \cdot 10^{-14}$	$8.3 \cdot 10^{-12}$	0.0059	Qwen2.5-7B
e5-large-v2	Qwen2.5-7B vs Llama-3.2-11B	0.031	0.12	0.0017	-

Table 10 – Statistical comparison between the top two performing vision-language models for each embedding.

5.2.2.4 Resolution Impact Analysis

To investigate the impact of input image resolution, we selected the model that achieved the best overall performance, Qwen 2.5 VL 7B, and evaluated it using various image sizes to analyze how resolution affects similarity scores. The statistical results of these evaluations, based on cosine similarity values, are summarized in Annex 30.

Although the mean values between the top two performing image sizes for each embedding model are very close, statistical tests were conducted to determine whether these differences are significant. As shown in Table 11, with the exception of all-MiniLM, all p-values from both the t-tests and Wilcoxon tests are above the 5% threshold, indicating no statistically significant differences.

Embedding	Compared Sizes	t-stat (p-val)	Wilcoxon W (p-val)	Mean Difference	Better Size
bge-large-en-v1.5	512×640 vs 512×896	$7.4 \cdot 10^{-1}$	$5.4 \cdot 10^{-1}$	0.0004	-
all-MiniLM-L6-v2	512×512 vs 512×896	$4.0 \cdot 10^{-1}$	$5.8 \cdot 10^{-1}$	0.0017	-

Embedding	Compared Sizes	t-stat (p-val)	Wilcoxon W (p-val)	Mean Difference	Better Size
gte-large	512×896 vs 512×640	$6.8 \cdot 10^{-1}$	$9.6 \cdot 10^{-1}$	0.0002	-
e5-large-v2	512×1024 vs 512×640	$6.2 \cdot 10^{-1}$	$8.4 \cdot 10^{-1}$	0.0003	-

Table 11 – Statistical comparison between the top two performing image sizes for Qwen2.5-VL-7B-Instruct

5.2.3 Final Considerations

As shown by the statistical tests in Table 10, **Qwen2.5 VL 7B** was considered statistically better than the other models in three out of the four embedding models evaluated. In the case of *e5-large-v2*, the Wilcoxon signed-rank test yielded a p-value of 0.12, above the 5% significance threshold. Nonetheless, the remaining tests presented notably small p-values, confirming Qwen 2.5 VL 7B as the best-performing model overall.

In addition to generating high-quality descriptions, Qwen 2.5 VL 7B also offers favorable processing times, as shown in Table 8. Depending on image size and server configuration, running multiple instances in parallel may be feasible, further accelerating the pre-processing phase.

Moreover, the tests comparing different input image sizes for the same model, shown in Table 11, reveal no statistically significant differences. Almost all p-values in these tests are greater than 5%, suggesting that Qwen 2.5 VL 7B maintains consistent performance regardless of image resolution within the tested range, with 512×512 adopted as the default input size for this implementation.

5.3 Evaluation of Embedding Models

5.3.1 Methodology

This section presents the procedures used to evaluate the selected embedding models. The goal is to assess how effectively each model encodes textual and visual content into a shared vector space suitable for similarity search. The evaluation considers quantitative, multilingual, and qualitative aspects to determine which model offers the best balance between accuracy, efficiency, and cross-lingual robustness.

5.3.1.1 Evaluation Setup and Protocol

To evaluate the embeddings, we need to assess how effectively and correctly the selected embedding models can transform the content into a vector representation. This enables us to later determine how well the model can match it to a story provided by the user. To simulate the behavior of the platform, we created a test scenario using a

FAISS vector database built with *SemArt2000*, mentioned in Section 5.1.1. Each image is represented by both its original description and the description generated by the model chosen.

To better reflect the actual platform behavior, we focused solely on the generated to original description strategy. In this setting, the FAISS database is built from generated descriptions (as they will be in the final version of the vector database), and queries come from original human-written descriptions. This setup relates with the platform’s need of retrieving relevant artworks based on user input.

To initiate the evaluation, we recorded the time each embedding model required to encode the 2,000 generated descriptions and, when applicable, the 2,000 corresponding images. The total duration shown reflects the time to embed all descriptions at once, using a single GPU. For models that support image input, the images were processed in batches of 100 to manage VRAM constraints. This comparison helps us assess the computational efficiency of each model for both text and image embedding tasks.

5.3.1.2 Quantitative Metrics (Recall@K)

For the main evaluation, we used the **Recall@K** metric with $K = 1, 3, 6$, which measures the percentage of times the correct image is retrieved within the top-K results. This allows us to assess how well each embedding model ranks the correct image based on an original description query. To perform this retrieval efficiently, we used the FAISS library to build and query the index for each embedding model. Specifically, we used an IndexFlatIP index, chosen for its speed, exact search and simplicity. Since all embeddings were normalized, using the inner product (dot product) in this setup is equivalent to cosine similarity. While these results are based solely on text embeddings, some models also support generating image embeddings in the same vector space. To test whether using image embeddings in the index could improve retrieval, we conducted an additional experiment combining both modalities.

5.3.1.3 Multilingual Evaluation

In addition to the main quantitative evaluation, we conducted a multilingual test to examine how well the embedding models preserve semantic alignment across languages. For this purpose, we used the *SemArt500* subset, containing 500 artworks whose English descriptions were translated into Portuguese and French. The evaluation followed the same procedure described earlier, computing the **Recall@K** metric ($K = 1, 3, 6$) to measure the retrieval accuracy when queries were written in a different language from the indexed descriptions. Specifically, the FAISS index was built using English descriptions, while the queries corresponded to their Portuguese and French translations. This setup simulates multilingual user interactions on the platform and allows us to identify models

that maintain cross-lingual consistency, an essential property for real-world deployment in a multilingual context.

5.3.1.4 Qualitative Evaluation

While Recall@K is useful to assess whether the exact target image is retrieved, it doesn't capture the semantic relevance of the results returned. The primary goal of the platform is not to find the original image, but to retrieve artworks that are similar in theme, content, or visual style of a certain part of a text. Therefore, even if the original artwork is not present in the top-k, the result can still be considered successful if the retrieved images are contextually appropriate.

To take this into consideration, we conducted a qualitative evaluation in which subjective judgment was used to assess the top-k results retrieved by each model. For three randomly selected query images, presented in Figure 17, we manually reviewed the top 10 retrieved images for each model and assigned a binary relevance score: 1 if the image was considered semantically relevant to the query description, and 0 otherwise.



(a) Village scene by Pieter Brueghel.



(b) Horse gallop painting by Géricault.



(c) Market scene by Louise Moillon.

Figure 17 – Side-by-side images used for evaluating subjective similarity.

The purpose of this qualitative evaluation was to narrow down the number of candidate models for final deployment. By assessing the semantic relevance of the top retrieved results, we were able to identify the most promising models, which were then selected for further evaluation through a user survey. This step ensured that only the top-performing models, based on both quantitative and qualitative criteria, were considered in the subsequent phase of subjective analysis.

5.3.2 Results and Discussion

In this part, we evaluate the embedding models used to transform both user text and artwork descriptions into vectors for similarity search. Since embeddings drive the retrieval accuracy, we analyze each model in terms of performance, efficiency, and resource usage.

5.3.2.1 Performance and Efficiency

As mentioned in Section 5.3.1.1, the embedding times were recorded and are presented in Table 12, as they may influence the decision on which embedding model will be adopted in this project. These times were measured using the *SemArt2000*, a set of **2,000 image-description pairs**.

Team	Model	Text-GPU (s)	Text-CPU (s)	Image (s)
OpenCLIP	ViT-G/14	5.47	125.99	102.06
	ConvNeXt-XXLarge	5.46	149.34	93.08
	ConvNeXt-Large@320px	2.76	61.29	54.85
Google	ViT-H-14-378-quickgelu (DFN)	6.03	155.75	146.17
BAAI	bge-large-en-v1.5	8.20	167.96	N/A
SentenceTransformers	all-MiniLM-L6-v2	0.87	7.08	N/A
DAMO Academy	GTE-large	8.00	152.82	N/A
intfloat	E5-large-v2	8.10	161.48	N/A
Qwen3	Embedding 0.6B	6.88	1361.84	N/A
	Embedding 4B	27.51	>1500	N/A

Table 12 – Embedding generation time per model.

It is worth noting that the CPU times refer to the embedding of 2,000 text descriptions. These times are expected to scale linearly with the number of embeddings. In contrast, the GPU times benefit from parallel computation and are less sensitive to increases in input size, making them more suitable for larger-scale processing.

5.3.2.2 Text-Based Retrieval

Table 13 presents the recall results for all tested models, using the *SemArt2000* generated descriptions from the Qwen2.5 VL model as the database and the original descriptions (GARCIA; VOGIATZIS, 2018) as queries.

Team	Model	Recall@1	Recall@3	Recall@6
OpenCLIP	ViT-G/14	0.084	0.146	0.201
	ConvNeXt-Large@320px	0.054	0.099	0.130
	ConvNeXt-XXLarge	0.070	0.116	0.169
Google	ViT-H-14-378-quickgelu (DFN)	0.064	0.099	0.136
BAAI	bge-large-en-v1.5	0.101	0.161	0.216
SentenceTransformers	all-MiniLM-L6-v2	0.054	0.102	0.141
DAMO Academy	GTE-large	0.121	0.186	0.251
intfloat	E5-large-v2	0.098	0.172	0.225
Qwen3	Embedding 0.6B	0.069	0.127	0.173
	Embedding 4B	0.098	0.175	0.237

Table 13 – Recall@K retrieval performance using text-only FAISS index

5.3.2.3 Dual-Index Retrieval Strategy

Initially, we attempted to add image embeddings into the same FAISS index alongside the text embeddings. However, since image and text embeddings have different distributions and origins, the metric barely changes as it can be shown in Table 14. The retrieval tended to prioritize text embeddings heavily, ignoring the contribution of the image embeddings.

Team	Model	Recall@1	Recall@3	Recall@6
OpenCLIP	ViT-G/14	0.089	0.148	0.206
	ConvNeXt-Large@320px	0.057	0.102	0.131
	ConvNeXt-XXLarge	0.078	0.127	0.184
Google	ViT-H-14-378-quickgelu (DFN)	0.069	0.105	0.148

Table 14 – Results of the Recall@K results by models supporting images embeddings.

To preserve the richness of the text-based and image-based information, we developed a method that considers both image and text embeddings. Instead of combining them in a single index, we created two separate FAISS indexes: one for image embeddings and another for text embeddings. During retrieval, we perform a search on both indexes and then interleave the results while ensuring there are no duplicates. Which modality will start is chosen randomly. The results can be seen in Table 15 and show a significant improvement in this metric between this approach and the other two mentioned.

Team	Model	Recall@1	Recall@3	Recall@6
OpenCLIP	ViT-G/14	0.188	0.357	0.462
	ConvNeXt-Large@320px	0.158	0.321	0.425
	ConvNeXt-XXLarge	0.189	0.352	0.461
Google	ViT-H-14-378-quickgelu (DFN)	0.206	0.382	0.488

Table 15 – Recall@K results using interleaved text and image embeddings from separate FAISS indexes.

5.3.2.4 Multilingual Performance

Table 16 presents the recall results for tested models, considering Portuguese and French translation and using the *SemArt2000* generated descriptions from the Qwen2.5 VL model as the database and the original descriptions as queries.

Team	Model	Recall@1			Recall@3			Recall@6		
		EN	PT	FR	EN	PT	FR	EN	PT	FR
OpenCLIP	ViT-G/14	0.084	0.044	0.045	0.146	0.063	0.088	0.201	0.098	0.132
	ConvNeXt-Large@320px	0.054	0.013	0.026	0.099	0.025	0.047	0.130	0.047	0.071
	ConvNeXt-XXLarge	0.070	0.022	0.036	0.116	0.041	0.070	0.169	0.067	0.098

Team	Model	Recall@1			Recall@3			Recall@6		
		EN	PT	FR	EN	PT	FR	EN	PT	FR
Google	ViT-H-14-378 quickgelu	0.064	0.044	0.045	0.099	0.025	0.047	0.136	0.067	0.098
BAAI	bge-large en-v1.5	0.101	0.011	0.024	0.161	0.025	0.046	0.216	0.044	0.074
Sentence	all-MiniLM L6-v2	0.054	0.009	0.015	0.102	0.014	0.025	0.141	0.024	0.036
DAMO Academy	GTE-large	0.121	0.019	0.033	0.186	0.041	0.067	0.251	0.062	0.106
intfloat	E5-large-v2	0.098	0.013	0.031	0.172	0.022	0.058	0.225	0.045	0.080
Qwen3	Emb 0.6B	0.069	0.047	0.044	0.127	0.084	0.087	0.173	0.120	0.126
	Emb 4B	0.098	0.086	0.810	0.175	0.144	0.148	0.237	0.208	0.206

Table 16 – Multilingual Recall@K retrieval performance using text-only FAISS index (EN = English, PT = Portuguese, FR = French).

The results highlight clear performance contrasts across models and languages. The GTE-large model achieves the highest recall in English, however, its performance drops sharply for Portuguese and French, indicating limited multilingual robustness.

In contrast, both Qwen3 embedding models show markedly better cross-lingual adaptability. The smaller Qwen3-0.6B already outperforms most baselines in Portuguese and French, while the Qwen3-4B achieves the best multilingual balance overall, improving recall in both translation settings. This gain likely comes from the model’s larger parameter count (≈ 4 billion), which allows richer multilingual representations, but also implies higher memory and computational costs during inference and FAISS indexing.

5.3.2.5 Subjective Evaluation

Table 17 shows the average number of semantically relevant images among the top-3, top-5, and top-10 results for each model, adding performance beyond exact matches.

Team	Model	Avg Top-3	Avg Top-5	Avg Top-10
OpenCLIP	ViT_G/14	1.67	2.33	4.67
	ConvNeXt-Large@320px	2.33	3.67	6.00
	ConvNeXt-XXLarge	1.33	2.33	5.00
Google	ViT-H-14-378-quickgelu (DFN)	1.33	2.33	4.67
BAAI	bge-large-en-v1.5	2.67	4.33	8.00
SentenceTransformers	all-MiniLM-L6-v2	2.00	3.00	4.67
DAMO Academy	GTE-large	3.00	4.67	8.33
intfloat	E5-large-v2	2.67	4.00	7.33
Qwen3	Embedding 0.6B	2.33	3.67	7.67
	Embedding 4B	2.67	4.33	8.33

Table 17 – Subjective evaluation results showing average number of relevant images retrieved in Top-K.

As observed, although the Recall@K metric suggested that models supporting image embeddings achieved significantly higher scores, a closer qualitative evaluation says differently. When analyzing three specific images, the models that relied solely on text embeddings consistently retrieved results that aligned more accurately with the intended meaning or theme of the original descriptions. This suggests that, despite lower Recall@K values, these models might capture more effectively the semantic and contextual alignment with the original description, a dimension not fully reflected in the retrieval metrics used.

In our view, detailed text descriptions capture subtle details that image-based embeddings often miss, like we can see in GTE-large and both Qwen3 models. A good text-only embedding model usually finds results that better match what we actually mean. In short, a stronger text embedding makes the search engine more useful overall.

5.3.3 Final consideration

Based on the combination of quantitative metrics and subjective evaluation results shown in the last Sections, we selected the **Qwen3 Embeddings 4B** model as the primary embedding model for our system. Despite not achieving the highest Recall@K scores when using multimodal retrieval strategies, Qwen3 Embeddings 4B demonstrated consistent and robust performance in capturing the semantic intent of the descriptions, both in automatic and human evaluations. Most importantly, it also exhibited strong and stable results across multiple languages, particularly Portuguese and French, while maintaining compatibility and reliable performance in several other supported languages.

Its exclusive reliance on text embeddings makes it a simpler and more scalable choice, significantly reducing the memory and computation footprint of the FAISS index, by approximately half compared to models that require both image and text embeddings. This not only streamlines the system architecture but also enhances efficiency.

Furthermore, we anticipate that future improvements in visual language models (such as Qwen 3 or Llama 4 with enhanced VL capabilities) will result in even higher-quality generated descriptions. Since our pipeline leverages text embeddings for retrieval, any gain in caption generation quality should translate directly into improved retrieval performance, without needing to modify the embedding backbone or the indexing structure.

5.4 Evaluation of LLMs

5.4.1 Methodology

This evaluation focuses on identifying the most suitable LLM for the **Art Exploration** mode, responsible for generating short narratives inspired by artwork descriptions. The objective is to assess how efficiently each model produces coherent and engaging stories that connect multiple image descriptions. The evaluation procedure includes both quantitative and qualitative criteria, ensuring that the selected model balances performance, creativity, and practical deployability.

5.4.1.1 Evaluation Protocol

To evaluate the different LLMs presented in Section 3.4.4, we conducted a focused experiment using the lightweight models selected. Since the story generation component is significantly lighter than the multimodal description step, the selected models are expected to run in a period that doesn't disrupt the user experience. Three representative images were selected for testing, as shown in Figure 18.

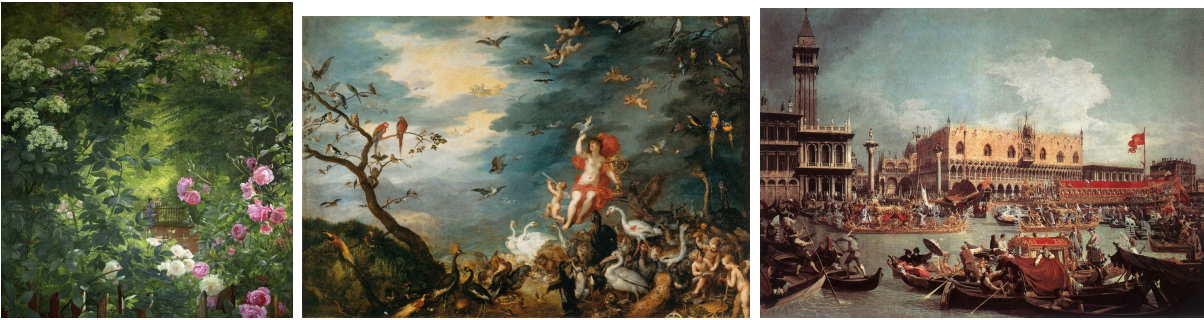


Figure 18 – Images whose description was used for testing LLMs.

Each model was prompted with the same text input, illustrated in Box 5.4.1.1, which includes the generated descriptions for the image. The task of the model is to generate a short story that logically integrates and builds upon the provided descriptions.

Test Prompt

Descriptions:

Art descriptions will be inserted here

Write a story that takes inspiration from these scenes. Use 2–3 short paragraphs (approximately). Tell it like a simple, flowing story with a start, middle and an end. The paragraphs have to be connected and follow a sequence of events.

Each model is evaluated using the same prompt to account for variation in generation time and potential randomness in output. All tests were conducted under the same hardware conditions.

5.4.1.2 Evaluation Metrics

The following metrics were used to assess model performance:

- **Average Generation Time (in seconds):** The average time taken to generate a story, computed over 10 runs per image, both using the GPU and the CPU.
- **Story Success Rate:** The proportion of successful outputs that result in a coherent story. Outputs were considered successful if they included at least two narrative sentences logically connected to the descriptions.

- **Average Quality of Story:** A subjective evaluation performed by the authors, assessing how well the generated story flows, its coherence, and narrative quality. This score ranges from 1 to 5, if the story is successful.

These metrics help guide the selection of the most suitable LLM for integration into the **Art Exploration mode**.

5.4.2 Results and Discussion

This section focuses on evaluating the LLMs used to generate short narratives from the selected image descriptions in the **Art Exploration mode**. We assess them based on generation time, success rate, and storytelling quality.

The models LLaMA 3.4 3B and Gemma 3B were excluded from testing due to consistently lower benchmark scores and reduced relevance for our use case. The selected models evaluated were **Qwen3-1.7B**, **Qwen3-4B**, **Phi-4-mini-instruct**, **Sabiá-3** and **Sabiazinho-3**.

Table 18 summarizes the results of the manual evaluation process described in Section 5.4.1.1. The *Quality Score* was assigned subjectively by the authors, based on narrative coherence and how well the story incorporated all three scenes, from 1 to 5.

Model	Avg Time GPU (s)	Avg Time CPU (s)	Avg Time API (s)	Success Rate (%)	Avg Quality Score
Qwen3-1.7B	12.35	63.49	-	100.0	3.00
Qwen3-4B	17.62	141.14	-	90.0	3.33
Phi-4-mini-instruct	15.66	117.3	-	80.0	2.75
Sabiá-3	-	-	12.68	100.0	4.8
Sabiazinho-3	-	-	4.29	100.0	4.6

Table 18 – Manual evaluation results for selected LLMs.

Some observations emerged from the evaluation with the locally run models:

- **Qwen3-1.7B** was the most consistent in following the prompt structure. While its outputs were not very creative, they reliably formed coherent stories covering all descriptions.
- **Qwen3-4B** demonstrated greater narrative variation, producing some higher-quality stories. However, this came with longer generation times and a slightly lower success rate, as some outputs deviated.
- **Phi-4-mini-instruct** showed the most creativity, often producing imaginative outputs. However, this led to inconsistent, with some stories focusing on only one or ignoring the descriptions entirely.

On the other hand, with the models run via API, it was observed:

- **Sabiá-3** proved the most consistent and creative overall, almost always producing high-quality outputs. The drawback, however, is its higher cost (R\$5 per million tokens) and long API response times.

- **Sabiazinho-3** also delivered consistent and creative stories. Unlike its larger counterpart, it offered much shorter response times and a lower cost (R\$1 per million tokens).

5.4.3 Final Considerations

For the first version of the project, we initially selected the Qwen3-1.7B model, as it offered low generation time, high consistency, reduced VRAM usage, and acceptable quality scores, making it a practical option for integration into the prototype.

Nonetheless, a recurring pattern was observed: the model frequently produced stories with female protagonists regardless of the input content. This suggests a possible bias in the training data, as discussed in (CHEN et al., 2025), which highlights how storytelling models can inadvertently reinforce gender stereotypes. Although this issue did not compromise the overall technical performance, it underscores the importance of monitoring and mitigating such biases in future iterations of the platform.

For the current platform release, we ultimately opted for **Sabiazinho-3**, a Brazilian model capable of understanding and generating content in both Portuguese and English. Unlike most open-source alternatives, it natively handles Portuguese, which is crucial for the intended user base. Also, it demonstrated consistent and creative storytelling with shorter response times and lower computational and financial costs compared to larger models.

Thus, while Qwen3-1.7B remains a suitable option for local experiments, **Sabiazinho-3** aligns better with the goals of accessibility, multilingualism, and practical deployment in this stage of the platform.

5.5 Evaluation of Vector Database

5.5.1 FAISS

5.5.1.1 Methodology

The purpose of this evaluation is to analyze the performance of different FAISS indexing strategies for efficient similarity search within the system. FAISS enables scalable nearest-neighbor retrieval by organizing embeddings into compressed or clustered structures. Here, it is aimed to identify the configuration that provides the best balance between retrieval accuracy and efficiency under the scale and constraints of our current platform. For the whole test the embedded model used was the *GTE-Large*.

To better understand how different FAISS indexing strategies and hardware setups perform, we tested four representative index types using the *SemArt15000* dataset, a set of more than 15,000 artworks with its generated descriptions, as introduced in Section 5.1.1.

All descriptions were converted into embeddings using the model chosen in the earlier step in the project.

Even though 15,000 images are relatively few for FAISS, which is optimized for datasets with millions or billions of vectors, our goal wasn't only to test scalability. Instead, we wanted to explore the trade-offs in latency, memory usage, and retrieval quality that are most relevant to our current application. This also gives us a clearer idea of how the system might behave if the platform grows or faces resource limitations in the future.

We evaluate each method based on:

- **Search latency** (average time per query),
- **Retrieval quality** (**Recall@K** metric with $K = 1, 3, 6$),
- **Memory requirements.**

Some FAISS index configurations mentioned in the Section 3.4.5 will be tested with these metrics. This selection covers a mix of exact and approximate methods, and contrasts CPU and GPU-based setups to analyze the impact of hardware on performance.

5.5.1.2 Results and Discussion

Efficient similarity search is essential for image retrieval in our system. FAISS handles this by indexing vector embeddings and returning the closest matches. Here, we evaluate different index types and configurations to find the best trade-off between retrieval accuracy, speed, and memory usage.

To evaluate the performance of different FAISS indexing strategies, we conducted a series of retrieval experiments using increasing dataset sizes, ranging from 1,000 to 15,000 samples. Three index types were tested on the CPU: IndexFlatIP, IndexIVF, and IndexIVFPQ. The number of clusters (*nlist*) and quantization parameters (*m_pq*, *nbits_pq*) were scaled appropriately with the dataset size. These parameters are required only for the second and third index types and are detailed in Annex 32.

5.5.1.3 CPU Performance

Figure 19 presents the evolution of Recall@1 and latency as the number of samples increases. The latency plot uses a logarithmic scale on the y-axis to better visualize differences between index types, which would otherwise be difficult to distinguish on a linear scale. The number of samples is expressed as $\cdot 10^3$ to simplify the axis and improve readability.

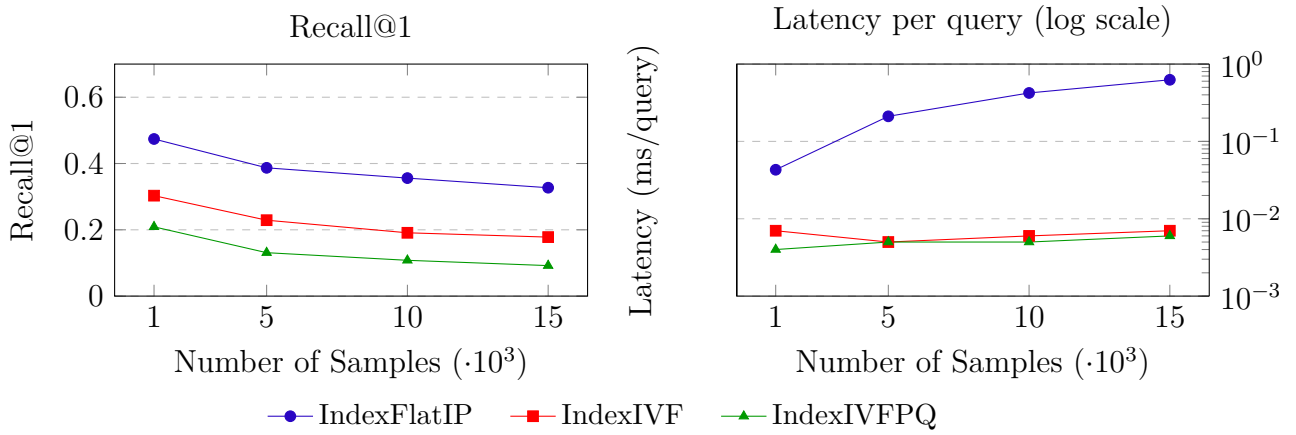


Figure 19 – Performance of FAISS indices on CPU: (left) Recall@1 across dataset sizes; (right) Latency per query in log scale.

IndexFlatIP consistently achieves the highest recall, as it performs exact brute-force search. However, its latency increases approximately linearly with the dataset size, which appears as a curved trajectory in the log scale. In contrast, IndexIVF and IndexIVFPQ maintain significantly lower latency, mainly in larger scales, thanks to their use of clustering and compression techniques.

We observe that recall tends to decrease as the dataset size increases. This is an expected behavior: with more candidate images, the chance of confusing the correct match with a visually or semantically similar one increases. As a result, especially in top-1 evaluation, the index may return a near match rather than the exact corresponding image.

To illustrate this behavior, Figure 20 shows the top-6 retrieval results for a single textual query using a FAISS index with 1,000 images. The correct image is retrieved as the top-1 result, but the remaining retrieved images were considered only partially related to the query. This shows the limited diversity of a smaller dataset, which can restrict the model’s ability to return multiple relevant candidates beyond the exact match.



Figure 20 – Top-6 retrieval results with 1,000 images. Displayed in reading order

Figure 21 presents the same query, now using a FAISS index with 15,000 images. Although the top-1 image remains correct, the other top-6 results are more semantically aligned with the description. This demonstrates that increasing the dataset size enhances the richness of the retrieved set, enabling not only accurate retrieval but also more meaningful alternatives. Such behavior is particularly valuable for associative or exploratory use cases, where users benefit from seeing a broader range of conceptually relevant images.



Figure 21 – Top-6 retrieval results using FAISS with 15,000 images.

Figure 22 presents the memory consumption of each index across dataset sizes. The memory usage of IndexFlatIP increases sharply in an almost quadratic proportion, reflecting the cost of storing all full-precision vectors in memory. In contrast, IndexIVF achieves lower memory usage due to coarse clustering, and IndexIVFPQ provides the most compact representation by combining clustering with compressed vector encoding.

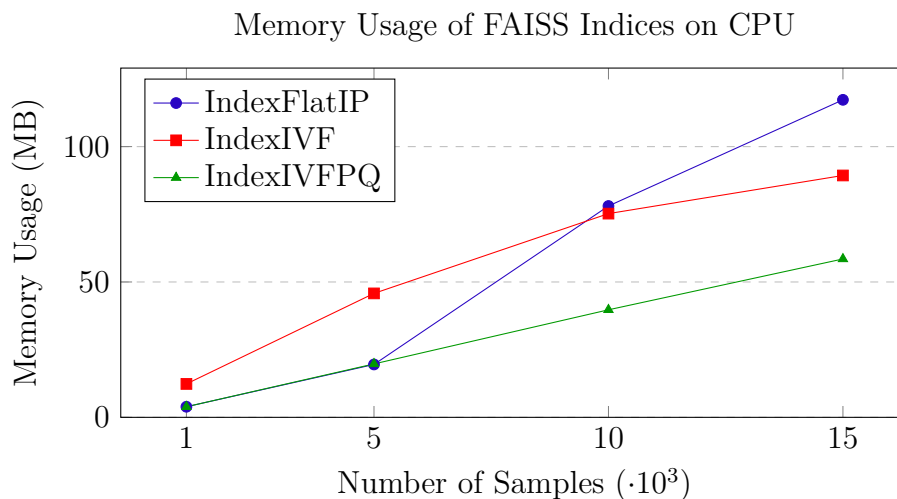


Figure 22 – Memory usage (in MB) of different FAISS indices on CPU across dataset sizes.

5.5.1.4 GPU Performance

Figure 23 illustrates the evolution of Recall@1 and latency across increasing dataset sizes using the GPU implementation of FAISS indices. As in the CPU case, the latency plot is shown on a logarithmic scale to highlight small differences that would otherwise be imperceptible. Due to the significantly faster computation enabled by GPU acceleration, all indices exhibit extremely low query latency, even for large datasets.

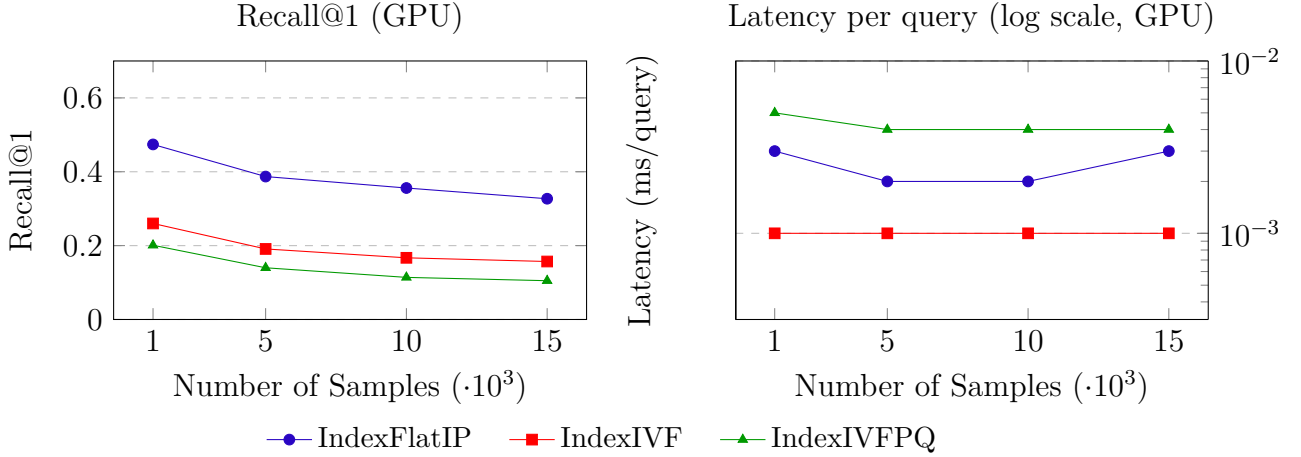


Figure 23 – Performance of FAISS indices on GPU: (left) Recall@1 across dataset sizes; (right) Latency per query in log scale.

IndexFlatIP also achieves the highest recall since it performs exact nearest neighbor search. While IndexIVF and IndexIVFPQ present slightly lower recall values due to their approximate nature. Notably, IndexIVF’s latency was so small that it fell below the resolution, showing the advantage of GPU parallelism.

Figure 24 shows the GPU memory consumption as the dataset size increases. As expected, IndexFlatIP demonstrates linear growth in VRAM usage due to its full storage of all vectors. IndexIVF consumes even more memory in some cases, which can be attributed to the overhead of storing the inverted lists and centroids in VRAM. In contrast, IndexIVFPQ maintains the lowest memory footprint across all dataset sizes, thanks to its use of product quantization to compress the stored vectors.

Memory Usage of FAISS Indices on GPU

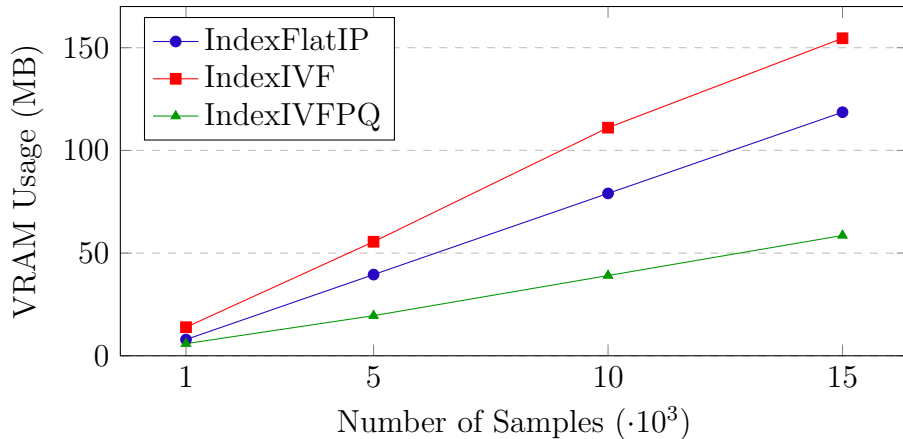


Figure 24 – Memory usage of different FAISS indices on GPU across dataset sizes.

5.5.1.5 FAISS Considerations

Given the characteristics of our dataset and the observed results, **IndexFlatIP** remains the most suitable choice for the platform if utilizing FAISS. It achieves the highest recall across all tested configurations and maintains sub-millisecond latency even with 15,000 embeddings. The latency growth observed in our tests appears to be nearly linear, suggesting that even with larger datasets, the increase in response time will not significantly degrade the user experience. Also, it is clear that no GPU is required for the size of our vector database and embeddings.

5.5.2 Qdrant

While FAISS offers a collection of indexing algorithms that operate as an in-memory library, Qdrant functions as a fully featured vector search engine. Instead of requiring manual index management, Qdrant provides persistent storage, metadata filtering, and a production-ready API layer. To evaluate its suitability for the platform, we analyzed its functional characteristics alongside a quantitative benchmark of its default indexing mechanism, the Hierarchical Navigable Small World (HNSW) graph.

5.5.2.1 Evaluation Protocol

Using the same *SemArt15000* dataset, Qdrant was deployed locally in a Docker container using the standard production configuration. The evaluation focuses on two aspects:

- **System Latency:** In contrast to FAISS, which is invoked as an in-memory library function, Qdrant queries have network overhead due to REST communication and JSON serialization. We therefore measure the *end-to-end round-trip time* for each query.
- **Retrieval Quality (Recall@K):** Qdrant's storage engine uses an HNSW index by default. We evaluated two configurations, varying the m and $ef_construct$ parameters to represent high-recall and balanced search profiles:
 - **High-Recall:** $m = 32$, $ef_construct = 200$, $ef_search = 128$.
 - **Balanced:** $m = 16$, $ef_construct = 100$, $ef_search = 64$.

5.5.2.2 Recall and Latency

Figure 25 presents the Recall@1 and latency results across dataset sizes ranging from 1,000 to 15,000 samples. The latency plot is displayed on a logarithmic scale to highlight subtle differences.

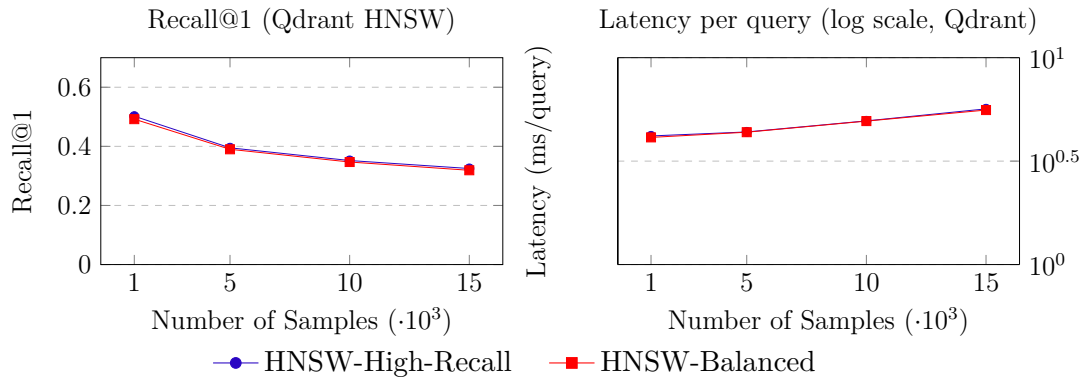


Figure 25 – Performance of Qdrant HNSW configurations: (left) Recall@1 across dataset sizes; (right) Latency per query in log scale.

Both HNSW configurations produced **very similar Recall@1 values** for all dataset sizes. This is expected: the dataset (15k samples) is too small to expose the typical recall differences between HNSW graph densities. Recall declines gradually as the dataset grows, following the same trend observed in the FAISS benchmarks. With more candidates, the probability of semantically similar distractors increases, which is visible for Recall@1.

Latency increases mildly with the number of samples, from approximately **4.1 ms/query** at 1,000 vectors to about **5.6 ms/query** at 15,000 vectors. This growth remains modest because: Qdrant operates almost entirely in memory and HNSW search scales logarithmically with the dataset size.

Most of the latency is due to the REST API round-trip rather than the vector search itself, meaning Qdrant is inherently more expensive than FAISS but still well within requirements.

5.5.2.3 Memory Consumption

Figure 26 presents the memory usage for the two index configuration chosen. As it can be seen, memory usage grows linearly with the dataset size, as expected for HNSW. While the high-recall configuration uses slightly more memory due to its denser graph (higher m), the differences remain small relative to overall consumption.

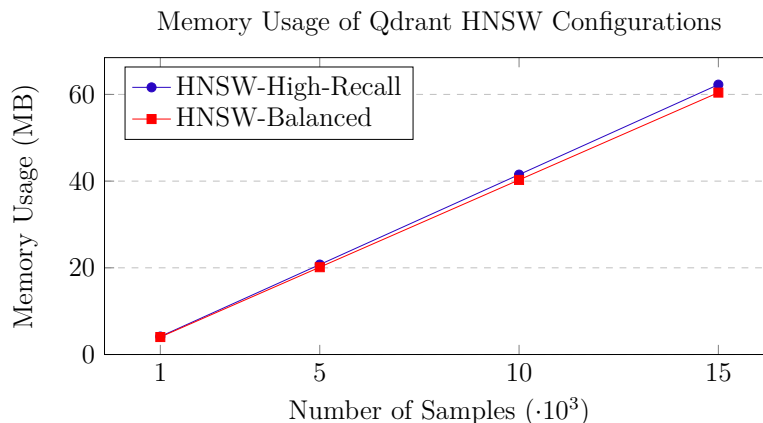


Figure 26 – Memory usage of different HNSW configurations across dataset sizes.

These values are significantly higher than FAISS’s IndexFlatIP on CPU but are typical for graph-based ANN structures, which require additional storage for neighbor links.

5.5.3 Final System Selection

The experiments show a clear distinction between FAISS and Qdrant in terms of role and suitability for the platform. **FAISS (IndexFlatIP)** achieves the best raw performance: it provides the highest recall, sub-millisecond latency, and predictable scaling for the evaluated dataset sizes. This makes it very well suited for offline experiments and algorithmic prototyping, as it was used during the research.

However, FAISS is a library, not a database. Using it directly in production would require additional components for persistence, metadata handling, API exposure, and operational management. **Qdrant**, on the other hand, offers slightly higher end-to-end latency (around 4–6 ms per query in our setup), but provides a complete vector search engine with built-in persistence, metadata-aware filtering, and a production-ready API.

Given that the observed Qdrant latency remains well within the requirements of an interactive web application, and that its HNSW recall closely follows the FAISS results for our scale, we adopt the following design choice: FAISS is used as a research/prototyping tool, while Qdrant is selected as the production vector database for the platform.

5.6 User Input Pre-Processing

5.6.1 Methodology

This section details the preprocessing steps applied to user text prior to retrieval. We first correct grammatical and orthographic errors, then segment the text into coherent units to balance semantic fidelity with visual diversity during retrieval. The procedures below describe the tools, segmentation strategies, and evaluation criteria adopted.

5.6.1.1 Spell-Checking

As discussed in Section 2.3.4, grammatical errors can negatively affect the quality of embeddings. Therefore, it is essential to verify and correct user input before evaluating its semantic content. The first step in this preprocessing pipeline is spell-checking. After reviewing several available tools, we selected the Python library provided by LanguageTool (LanguageTool Contributors, 2024). This open-source library is widely used across different sectors and offers robust multilingual support, covering more than 20 languages. Its effectiveness in detecting grammatical errors has been demonstrated in previous research (OMELIANCHUK et al., 2021), and it is even integrated into commercial applications such as Microsoft Office (LanguageTool Contributors, 2024).

5.6.1.2 Text Segmentation Setup

We selected a simple narrative divided into three paragraphs, shown in Box 5.6.1.2. This text was segmented according to the strategies outlined in Section 3.4.6. The *SemArt15000* dataset was used for retrieval due to its larger scale, which enhances the diversity and representativeness of the retrieved results.

A Walk Through the Village

I remember arriving in a small village by train while it was raining. The station was quiet, just a couple of people waiting, and a bike leaning against the wall. I had a small bag with me and a letter in my pocket with an address. A dog was sleeping under the bench, and the only sound was the soft hum of the engine as the train pulled away.

Instead of going straight to the house, I took a walk through the village. The streets were narrow and made of stones. I passed a small shop with flowers in front, and a boy kicking a ball by himself. There was a little church with a bell that rang as I walked by. Everything felt calm, like time moved slower there.

When I got to the house, no one was home yet. I sat on the front step and opened the envelope I had brought. Inside was a postcard from my girlfriend at the time. She was smiling in the photo, standing in a wide field with hills in the background and flowers all around. On the back, she had written, “Wish you were here. Enjoy your vacations in your uncle’s house.”

Two distinct configurations were implemented for sentence-level segmentation: **non-overlapping** and **overlapping (sliding window)**. In the non-overlapping configuration, fixed-size sentence groups are created such that each sentence appears in exactly one segment. On the other, the sliding window strategy produces overlapping segments by advancing the window or step, one or two at a time. This last setup helps to preserve context and mitigate semantic fragmentation.

For each segmentation strategy, the resulting segments were embedded using the model identified as the most effective in earlier evaluations. All queries were performed using a FAISS index with the IndexFlatIP configuration on GPU. The top-1 most similar image was retrieved for each segment based on cosine similarity, generating a total of seven distinct image sequences, discussed more in the Section 5.3.

The following evaluations were carried out to assess the performance of each strategy:

- **Cosine similarity:** average similarity between each segment and its retrieved image description.
- **Diversity score:** total number of unique images retrieved across the full sequence.
- **Qualitative review:** manual assessment of narrative coherence and semantic alignment between the story and retrieved images.

5.6.2 Results and Discussion

This part evaluates how different text segmentation strategies affect the retrieval process. We compare overlapping and non-overlapping sliding windows to measure their impact on semantic consistency and visual diversity in the results.

5.6.2.1 Quantitative Results

As shown in Table 19, the average cosine similarity scores remain consistently high across **all segmentation strategies**, indicating that each approach is generally capable of retrieving semantically aligned images. Nevertheless, the use of overlapping segments slightly reduces the diversity of the retrieved images. This is expected, as overlapping strategies reuse partial content across multiple segments, increasing the likelihood of retrieving similar or even identical images.

Segmentation Strategy	Type	Step	Average Cosine Similarity	Total Retrieved	Diversity (Unique Images)
Paragraph	-	-	0.8492	3	3
1 Sentence	Non-overlapping	1	0.8383	15	14
2 Sentences	Non-overlapping	2	0.8432	7	7
3 Sentences	Non-overlapping	3	0.8435	5	5
2 Sentences	Overlapping	1	0.8430	14	11
3 Sentences	Overlapping	1	0.8415	13	12
3 Sentences	Overlapping	2	0.8444	7	7

Table 19 – Retrieval performance by segmentation strategy, type, and step size.

5.6.2.2 Diversity Criteria and Strategy Selection

For a story composed of three paragraphs (as seen in Box 5.6.1.2), an ideal range for unique image outputs would be between 6 and 12. This range ensures that each part of the narrative is visually represented without excessive redundancy. In this context, we rely on the *Diversity* metric, which counts only unique images.

Based on this criterion, three strategies fall outside the acceptable range: *Paragraph-level* (3 images), *3 Sentences – Non-overlapping* (5 images), and *1 Sentence – Non-overlapping* (14 images). These are therefore excluded from further consideration.

The strategies that best balance semantic relevance and diversity are:

- *2 Sentences – Non-overlapping*
- *2 Sentences – Overlapping (step=1)*
- *3 Sentences – Overlapping (step=2)*

5.6.3 Final Consideration

Each of these configurations produced a satisfactory visual narrative when reviewed qualitatively. After filtering out duplicate images, the resulting sequences preserved coherence and reflected the storyline with varying degrees of breadth. The strategies with lower

diversity offered more focused and consistent image sets, while those with higher diversity enabled broader visual interpretations.

To accommodate different user needs in the *Memory Reconstruction* mode of the platform, we propose supporting two distinct segmentation profiles:

- **Conservative mode** (*3 Sentences – Overlapping (step=2)*): prioritizes focus and visual continuity with fewer but more contextually rich segments.
- **Broader mode** (*2 Sentences – Overlapping (step=1)*): provides greater narrative detail and more visual variety, while still maintaining strong semantic alignment.

To document the results, we selected the final paragraph of the story (re-shown in Box 5.6.3) and applied both strategies.

Final Paragraph Used in Segmentation Test

When I got to the house, no one was home yet. I sat on the front step and opened the envelope I had brought. Inside was a postcard from my girlfriend at the time. She was smiling in the photo, standing in a wide field with hills in the background and flowers all around. On the back, she had written, “Wish you were here. Enjoy your vacations in your uncle’s house.”

Figure 27 below shows the output of the conservative strategy for the last paragraph.



Figure 27 – Conservative mode: 3-sentence overlapping (step=2).

In contrast, Figure 28 below illustrates the broader strategy, with more images and a more varied interpretation:



Figure 28 – Broader mode: 2-sentence overlapping (step = 1).

5.7 Discussion

This chapter presented the evaluation of the models and strategies used in our platform. Among the LMMs tested, **Qwen2.5 VL 7B** emerged as the top performer, achieving higher semantic similarity scores in three out of four benchmarks. It also delivered consistent results across different image resolutions, which allowed us to adopt 512×512 as the default size without lossing quality. Overall, this model is a reliable choice for generating the image descriptions used throughout the system.

We selected **Qwen3 Embedding 4B** for its high average performance on the MTEB benchmark and robust multilingual capabilities, which ensure support to the platform’s supported languages. While it did not achieve the highest automatic retrieval scores, it outperformed others in subjective evaluations, effectively capturing the meaning of descriptions. Additionally, its text-only architecture simplifies the indexing pipeline and reduces computational requirements compared to multimodal alternatives.

For the Art Exploration mode, we evaluated several LLMs and API-based models to generate stories from the images selected by the user. The assessment focused on generation time, the ability to interpret visual descriptions, and the quality of the narratives produced. Based on these criteria, the model chosen was **Sabiazinho-3**, which consistently delivered satisfactory stories with low latency and reduced financial cost.

Regarding the vector database, **Qdrant** was selected as the final production system. While FAISS showed raw speed during prototyping, Qdrant offers a complete vector search engine with built-in persistence, metadata-aware filtering, and a ready API. Its latency remains well within the requirements for an interactive web application, and its HNSW indexing provides a robust balance between recall and performance, eliminating the need for the manual index management required by FAISS.

The **conservative mode**, based on overlapping 3-sentence segments with a step of 2, provides coherent and focused visual outputs. The **broader mode**, using overlapping 2-sentence segments with a step of 1, generates varied results. These modes are designed to give users flexibility in how much details they want when reconstructing a narrative.

Together, the choices made, as shown in Table 20, represent a balanced solution, combining performance, efficiency and user adaptability, to support the platform.

Component	Selected Model / Strategy	Reason for Selection	Notes
Description Generation (LMM)	Qwen2.5 VL 7B	Best semantic similarity overall Consistent across resolutions	Default resolution set to 512×512
Text Embeddings	Qwen3 Embedding 4B	Best average MTEB score Strong user survey performance	Text-only model simplifies pipeline
Story Generation (LLM)	Sabiazinho-3	Consistent, coherent narrative generation with low cost	Used in Art Exploration mode
Similarity Search	Qdrant	Built-in persistence and metadata filtering	Production-ready vector engine
Text Segmentation	<i>Overlapping 3-sentence (step 2)</i> <i>Overlapping 2-sentence (step 1)</i>	Balanced between coherence and diversity	-

Table 20 – Summary of Evaluation Results and Final Choices

6 System Architecture and Web Implementation

6.1 Overview

This chapter details the technical implementation of ArtEvoke, translating the methodological findings and AI model selections discussed in previous chapters into a functional, production-ready software system. The objective was to engineer a robust, scalable, and accessible web platform capable of supporting the distinct interaction modes, Memory Reconstruction and Art Exploration, while ensuring secure and structured data management for clinical sessions. The development process prioritized a modular architecture, decoupling the user interface from the logic and data layers to facilitate future iterations and the integration of new AI capabilities.

6.1.1 System Architecture

The platform adopts a service-oriented architecture, fully containerized to ensure consistency across environments. The system is divided into three primary layers: Front-end, Back-end, and Data. The Front-end Layer consists of a Single Page Application (SPA) that acts as the interface for patients, caregivers, and doctors, communicating with the back-end server via RESTful APIs. The Back-end Layer serves as the orchestrator, managing authentication, session control, and the business logic required for the therapeutic modes. Finally, the Data Layer handles persistent storage and high-performance retrieval, integrating both relational structures for clinical records and vector-based engines for semantic search.

All components are encapsulated within Docker containers and orchestrated via Docker Compose, which manages networking, volume persistence, and startup dependencies. Traffic management and security enforcement are handled by an Nginx reverse proxy, which acts as the single entry point for client requests, routing them to the appropriate backend services or static asset volumes. This separation of concerns ensures that the system remains maintainable and that individual components can be scaled or updated independently without disrupting the platform's overall functionality.

6.1.2 Technology Stack

The selection of the technology stack was driven by the requirement for asynchronous processing, type safety, and the ability to seamlessly integrate with Python-based AI ecosystems. For the backend, we employed Python 3.12 with the FastAPI framework, chosen for its speed and native support for asynchronous operations, which is detailed further in

Section 6.3. The frontend was built using React 18, utilizing functional components and hooks to manage the application state efficiently, as described in Section 6.2.

Regarding data persistence, a hybrid strategy was adopted. MySQL was selected as the relational database management system (RDBMS) to ensure ACID compliance for user profiles and clinical evaluation data. For the semantic retrieval of artworks, we implemented Qdrant as the vector database engine. As evaluated in Section 6.4, Qdrant was chosen over library-based solutions like FAISS for the production environment due to its built-in persistence, API capabilities, and metadata filtering features.

The platform creates a bridge to external intelligence through specific AI services. Text generation and Portuguese language processing are handled by Maritaca AI (Sabiazinho-3), while visual understanding and embedding generation are performed by Qwen models (Qwen2.5-VL and Qwen3-Embedding), as defined in the methodology and integrated via the backend clients described in Section 6.3.

6.1.3 Design Principles and Rationale

The engineering and design of ArtEvoke were guided by the specific constraints and needs of its target demographic: elderly individuals, mainly those living with dementia, and their caregivers. Unlike general-purpose software, the primary design principle for this platform is cognitive accessibility. The user interface prioritizes linear workflows while deliberately restricting complex navigation patterns during active sessions. This approach minimizes cognitive load, ensuring that the technology acts as a facilitator rather than a barrier to the therapeutic process.

From a data perspective, the architecture enforces a strict separation between clinical integrity and exploratory flexibility. Patient sessions and evaluations are stored in a normalized relational schema to ensure that medical data remains consistent and valid for longitudinal analysis. Conversely, the artwork retrieval system prioritizes flexibility and semantic approximation, allowing for a more fluid exploration of visual content.

Furthermore, system latency was a critical consideration. Given that AI inference on images can be computationally expensive and time-consuming, the architecture relies on pre-computed embeddings and asynchronous processing. Artwork descriptions and vector representations are generated during a data ingestion phase rather than at runtime. This decision ensures that user interactions, such as searching for a memory or generating a story, remain within a reasonable time, not interfering with the user experience (UX).

6.2 Front-end Architecture

The front-end serves as the primary interface between the complex AI-driven system and its end users, comprising healthcare professionals, caregivers, and elderly patients undergoing cognitive therapy. The target demographic imposes unique constraints on user

experience design. Elderly patients, particularly those with Alzheimer’s disease or related dementia, require interfaces that minimize cognitive load through linear workflows and avoidance of complex navigation patterns. Healthcare professionals need efficient tools to configure therapy sessions and monitor patient progress, while caregivers require intuitive interfaces operable with minimal training. All the decisions and implementations described in the next subsections follow these principles.

6.2.1 General Architecture

The front-end is architected as a Single Page Application (SPA) using React 18, leveraging functional components and React Hooks for state management encapsulation. The application structure is organized around feature-based modules with distinct directories for Art Exploration, Memory Reconstruction, and Clinical Evaluation. This organization ensures that changes to one therapeutic module do not affect others, supporting independent development and testing cycles. Each feature module encapsulates its own components, hooks, and local state logic, exposing only necessary interfaces to the rest of the application. The project structure reflects this organization, with the frontend directory containing all React application code, including components, pages, features, and configuration files. The general structure can be seen in the Figure 29 and will be detailed in the next paragraphs.

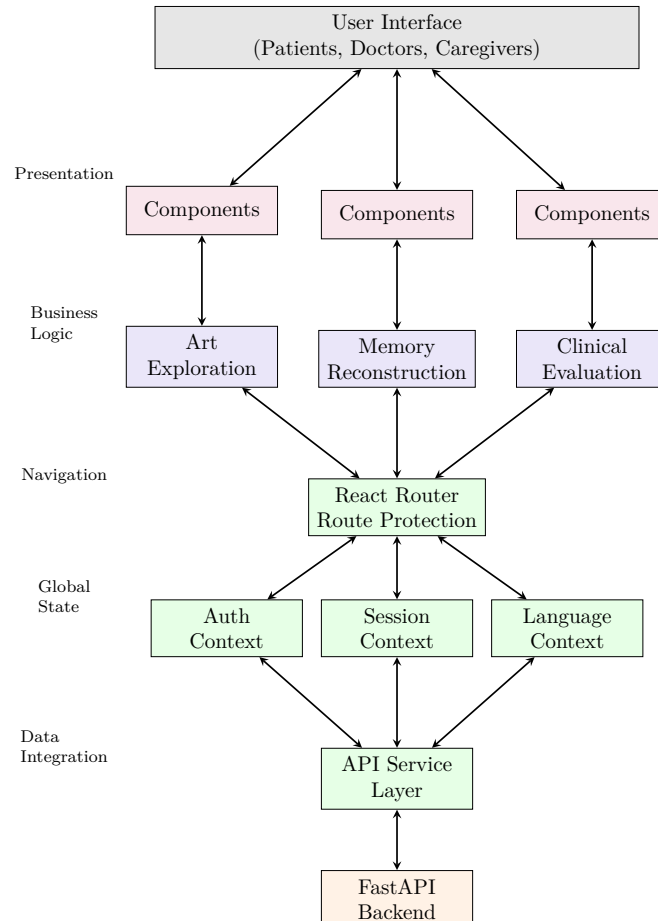


Figure 29 – Front-end architectural overview with component organization and data flow

React Router provides client-side navigation with route protection mechanisms ensuring appropriate access control based on user roles. As illustrated in Figure 29, it acts as the central entry point into the three feature modules, determining which workflow (Art Exploration, Memory Reconstruction, or Clinical Evaluation), is accessible based on user authentication, session status, and language context. The transitions between API layer, global contexts, router, and feature modules follow the vertical data-flow expressed in the diagram, where each layer progressively refines the information needed by the next.

The component architecture follows a compositional pattern in which UI elements enforce accessibility standards at the lowest level, composed into functional units and assembled into complete interface sections. This mirrors the upper portion of the figure, where each feature module exposes its own component set that ultimately feeds into the unified user interface. This organization ensures consistent styling and behavior while still allowing the independence required for each feature.

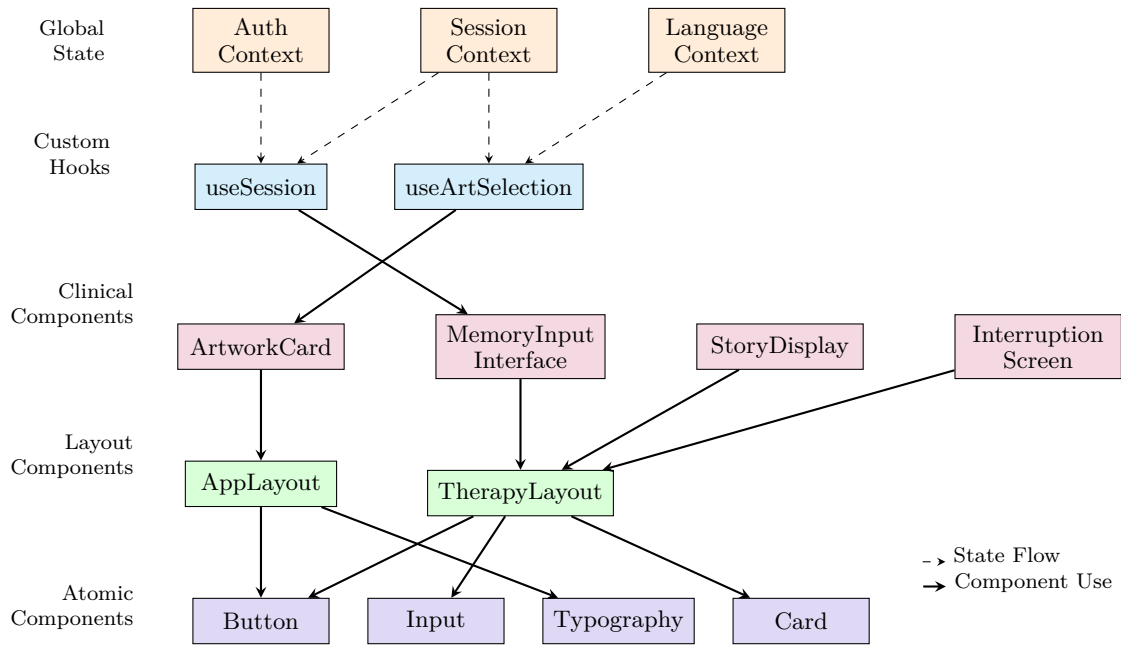
To support this modularity, the application uses a centralized configuration approach for API endpoints and authentication settings. The *config* directory provides the shared infrastructure referenced in the lower layers of the architecture diagram, ensuring that communication between the front-end and FastAPI backend remains consistent across deployments.

Performance optimizations further reinforce the layered design. Route-level code splitting and lazy loading ensure that each module loads only when necessary, aligning with the feature isolation shown in the diagram. Likewise, the hybrid state React Context for global concerns and local component state for feature-specific logic—maps directly onto the separation of “Global State” and “Business Logic” layers. This approach avoids the overhead of external libraries while keeping data flow predictable and maintainable.

Finally, the architecture emphasizes robustness and user experience quality. Error boundaries prevent component failures from affecting the rest of the interface, and network resilience mechanisms ensure that users, particularly elderly patients and caregivers, remain informed during long operations. Custom hooks encapsulated in the *hooks* directory implement this behavior and represent the internal logic connecting the business and presentation layers shown in the diagram.

6.2.2 Component Architecture and State Management

The component architecture and state management strategy prioritize clarity, robustness, and maintainability, requirements essential for reliable clinical use. The design follows a layered, compositional structure, represented in Figure 30, where global state flows through custom hooks into clinical and layout components while UI logic remains isolated and predictable.



State management adopts a pragmatic hybrid approach: React Context API handles application-wide concerns, while individual components manage their own transient UI state. This avoids the overhead of external libraries such as Redux (Redux Team, 2025) while remaining idiomatic to React.

Global state is organized into three primary contexts:

- **Authentication Context:** Maintains user credentials, role information, and session tokens, ensuring consistent authentication state and enabling automatic redirects based on permissions.
- **Session Context:** Tracks the active therapy session, including the current patient, selected therapeutic module, and progress through multi-step workflows.
- **Language Context:** Manages internationalization preferences, storing the selected language and providing translation functions throughout the component tree.

These contexts compose at the application root, making their state available without prop drilling while maintaining clear boundaries between state domains.

6.2.2.1 Local State Management

Local UI behavior is managed using React’s built-in *useState* and *useReducer* hooks, which keep short-lived or component-specific interactions isolated from the global state. This includes elements such as form inputs in the *MemoryInputInterface*—where text values and validation must update frequently without impacting other components—along with loading indicators tied to asynchronous operations and small UI toggles that control temporary interface states. By keeping these interactions local, the application avoids unnecessary

global state updates, reduces re-renders of unrelated components, and maintains smooth performance during complex therapy workflows.

6.2.2.2 Custom Hooks

Reusable business logic is encapsulated in custom hooks, which provide a clear separation between functional behavior and UI presentation. The *useSession* hook centralizes the management of the therapy session lifecycle, handling API interactions and guiding transitions between workflow steps. Similarly, the *useArtSelection* hook manages artwork browsing logic, including filtering, pagination, and selection state. By placing these abstractions between the global contexts and the clinical components, the application ensures consistency across features while allowing individual components to remain focused solely on rendering responsibilities.

6.2.2.3 Component Hierarchy

The architecture follows a layered structure inspired by atomic design principles:

- **Atomic Components:** Core UI primitives such as *Button*, *Input*, and *Card*, enforcing visual consistency.
- **Layout Components:** Define structural templates, like *TherapyLayout* for linear therapy flows.
- **Clinical Components:** Implement key therapeutic interactions, including:
 - *ArtworkCard*: Artwork display with lazy loading.
 - *MemoryInputInterface*: Text and voice input for reconstruction tasks with validation and feedback.
 - *StoryDisplay*: LLM-generated narratives with adjustable text size and text-to-speech options.
 - *InterruptionScreen*: Safety interface for session pauses or patient distress events.

Each layer builds on the one below it, enabling predictable composition and simplifying maintenance.

6.2.3 Core Interface Flows

These flows are implemented using the layered component and state management patterns described in the previous subsections, ensuring that clinical logic remains decoupled from presentation and navigation concerns. The ArtEvoke interface implements three primary therapeutic flows, each designed to guide users through structured cognitive exercises while minimizing confusion and cognitive load. These flows represent the core clinical value of the platform, translating therapeutic protocols into interactive digital

experiences that maintain consistency with established clinical practices while leveraging AI capabilities. The flows are shown in Figure 31

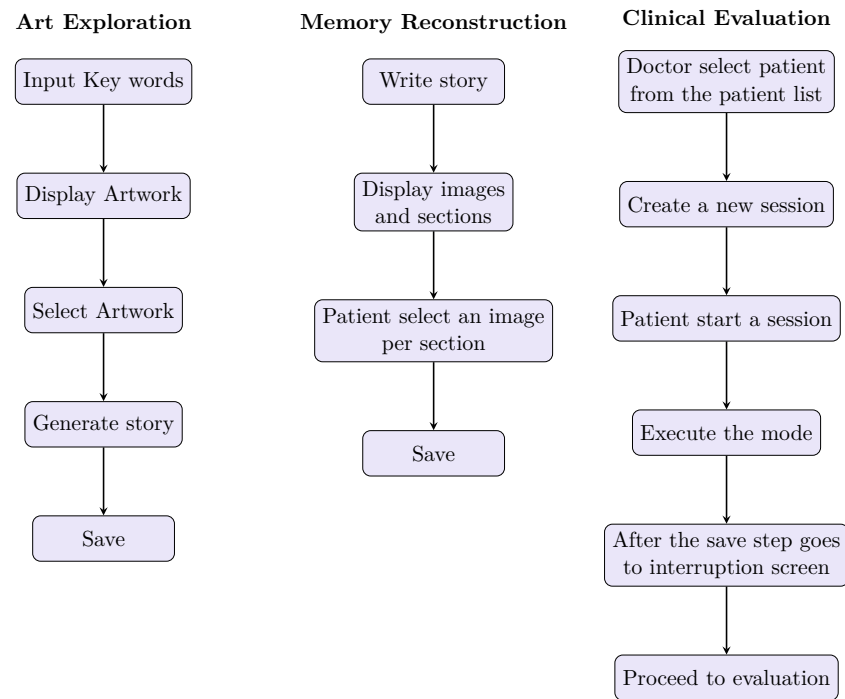


Figure 31 – Core therapeutic interface flows showing sequential steps for each module

6.2.3.1 Memory Reconstruction Flow

The Memory Reconstruction flow, as shown in Figure 31, follows a narrative and visually guided structure. Patients begin by reading the initial instructions displayed on the screen (Figure 32).

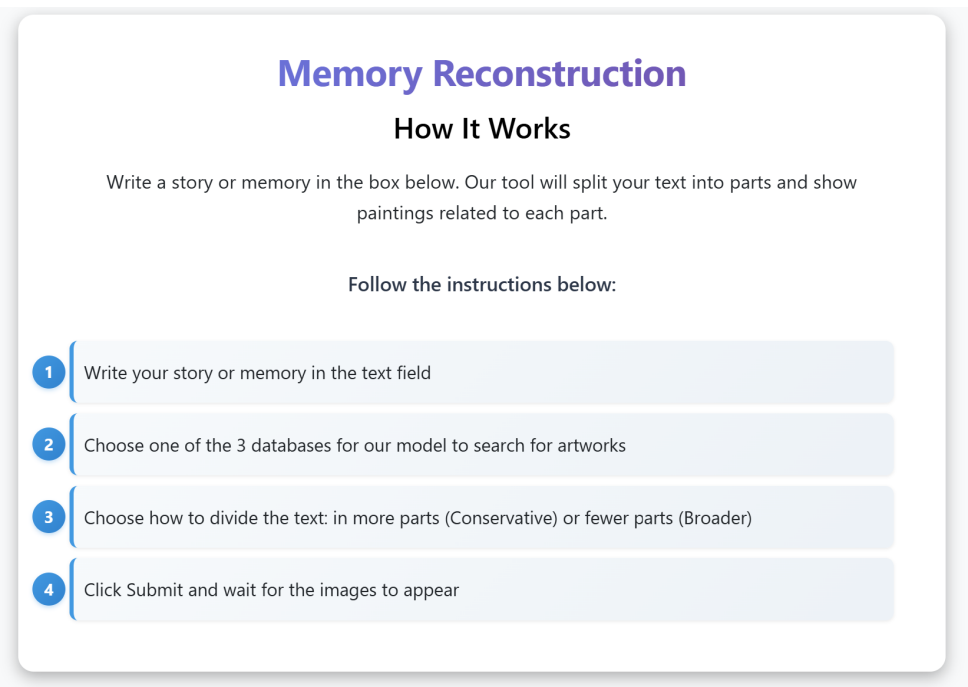


Figure 32 – Instructions displayed before story input

After reviewing the instructions, the patient writes a short story in a text grid and submits it through the interface, as shown in Figure 33.



Write your story below!

The city towers glimmered above me like steel giants. I wandered toward the quiet outskirts where the noise softened. Sunlight spilled across open fields, turning everything warm and golden. Wildflowers brushed my legs as colors replaced concrete. And at last, a still blue lake opened before me.

Select Dataset: Wikiart Image Selection: Conservativ

Submit
Speak

Figure 33 – Patient writing and submitting the story

Once submitted, the backend processes the narrative, dividing it into sections and retrieving the six most semantically similar images for each segment. An example of the first generated section is shown in Figure 34.

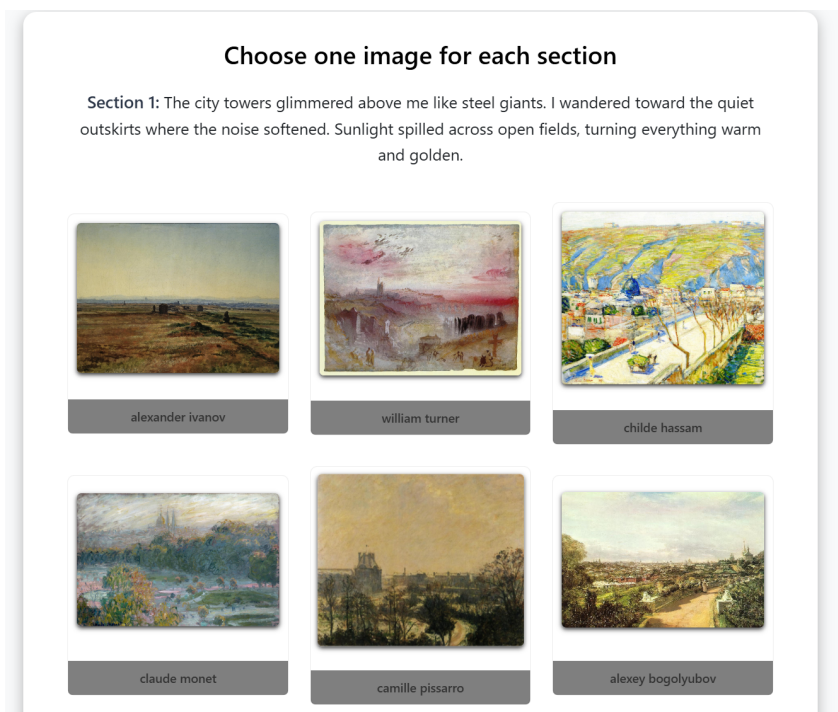


Figure 34 – Example of the first generated section with suggested images

All sections and their corresponding image sets are presented in vertically ordered grids with large image previews to enhance visibility. Figure 35 illustrates the final section, where the patient selects one image per segment and reaches the end of the flow.

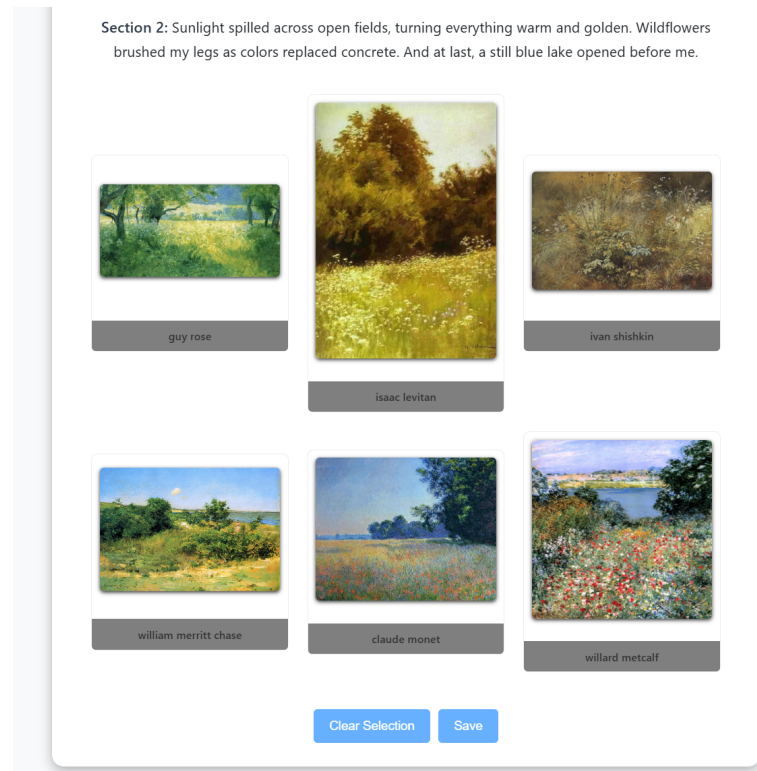


Figure 35 – Final section where the patient proceeds to save or evaluation

At this point, the patient may either continue to the evaluation phase (if inside a session) or simply save their results. In both cases, the system finalizes the interaction by issuing a save request to the backend, storing the execution data.

6.2.3.2 Art Exploration Flow

The Art Exploration flow guides patients through a structured creative task composed of four sequential steps. The flow begins with an instruction screen that introduces the activity and clarifies how patients should interact with the interface (Figure 36).

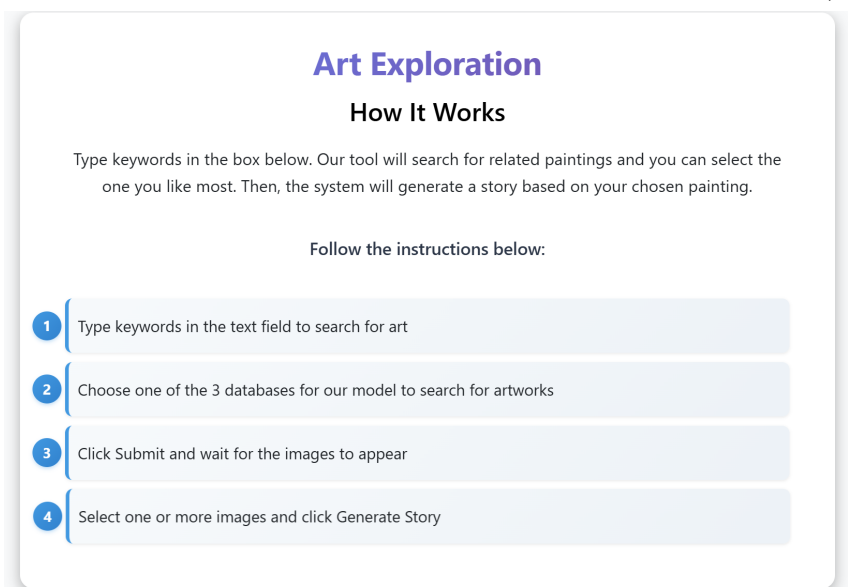


Figure 36 – Instruction screen

After reading the instructions, patients enter one or more keywords related to themes, emotions, or concepts they wish to explore, as shown in Figure 37. These keywords trigger a backend retrieval process that identifies relevant artworks.

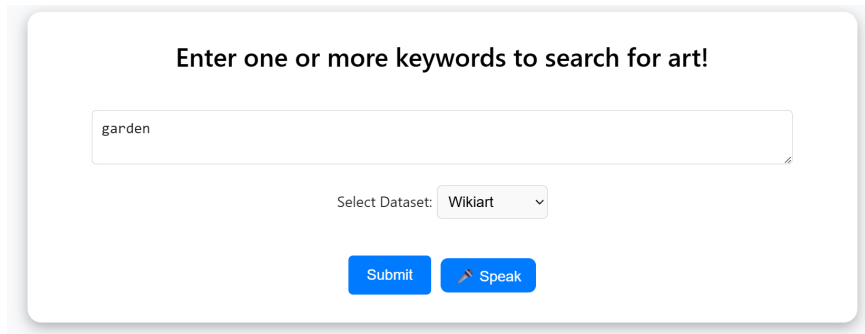
A white rectangular search interface with a light gray border. At the top, it says "Enter one or more keywords to search for art!". Below this is a text input field containing the word "garden". Under the input field is a "Select Dataset:" label followed by a dropdown menu showing "Wikiart". At the bottom are two blue buttons: "Submit" and "Speak" with a microphone icon.

Figure 37 – Keyword search interface

The system then displays a curated grid of artworks matching the selected terms. Patients may select one or multiple images, and the interface allows them to refine their search or run additional queries as needed. The Figure 38 shows the selection and how it is done. This flexible selection step supports exploratory behavior while maintaining low cognitive load through a minimal, image-focused layout.

Images Selection

Please select one or more of them to generate a story. Selected images will remain chosen across multiple searches.

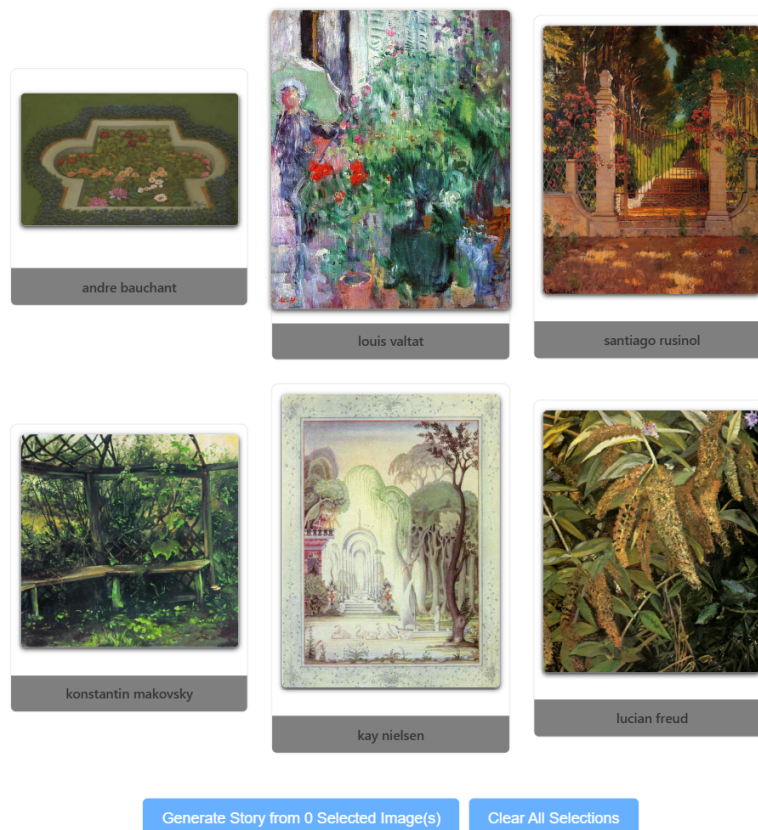


Figure 38 – Artwork selection screen

Once the patient finalizes their selections, they activate the “Generate Story” button. The backend synthesizes the chosen images into a coherent narrative, which is presented in a text grid for the patient to read and review (Figure 39). After reviewing the story, patients may either save the result or continue to the evaluation phase if the task occurs within an active session. As with the other flows, the interaction concludes with a save request to the backend, ensuring that all data is properly recorded.

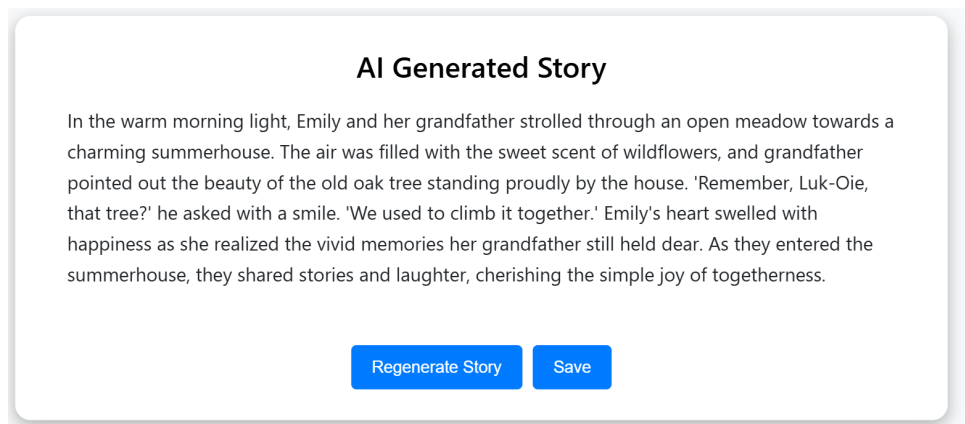


Figure 39 – Story generation screen

6.2.3.3 Clinical Evaluation Flow

The Clinical Evaluation flow structures how doctors review patient progress and how patients complete post-session assessments. Doctors begin by accessing a patient list filtered by their assigned caseload. When initiating a new evaluation, the doctor first creates or selects an existing session, as shown in Figure 40, which illustrates the creation of a Memory Reconstruction session.

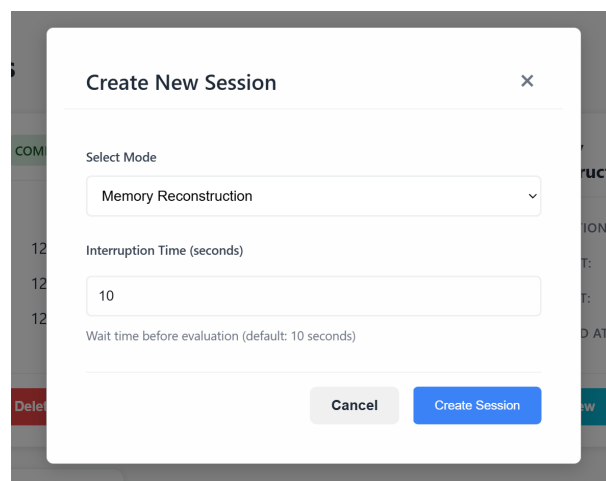


Figure 40 – Memory Reconstruction session creation

Selecting a patient reveals all recorded sessions in chronological order, including summary information such as session duration, interruption time, and completion status (Figure 41).

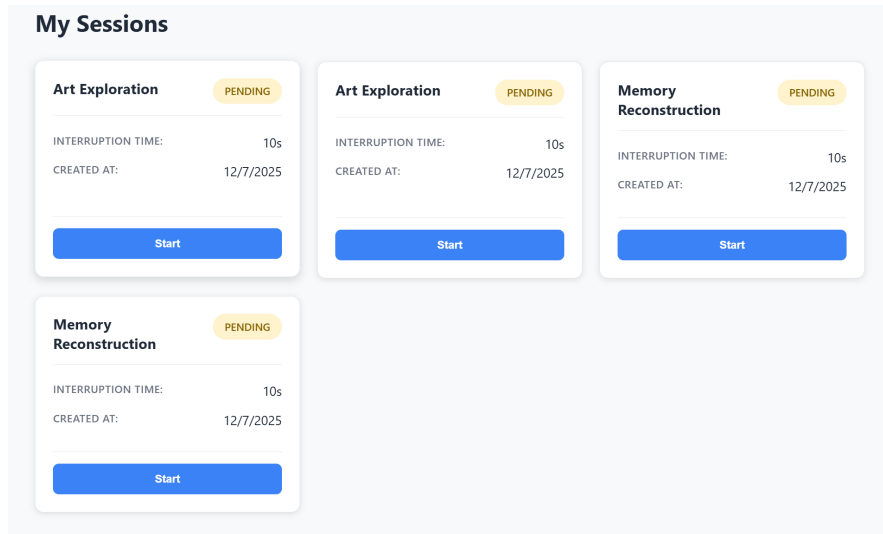
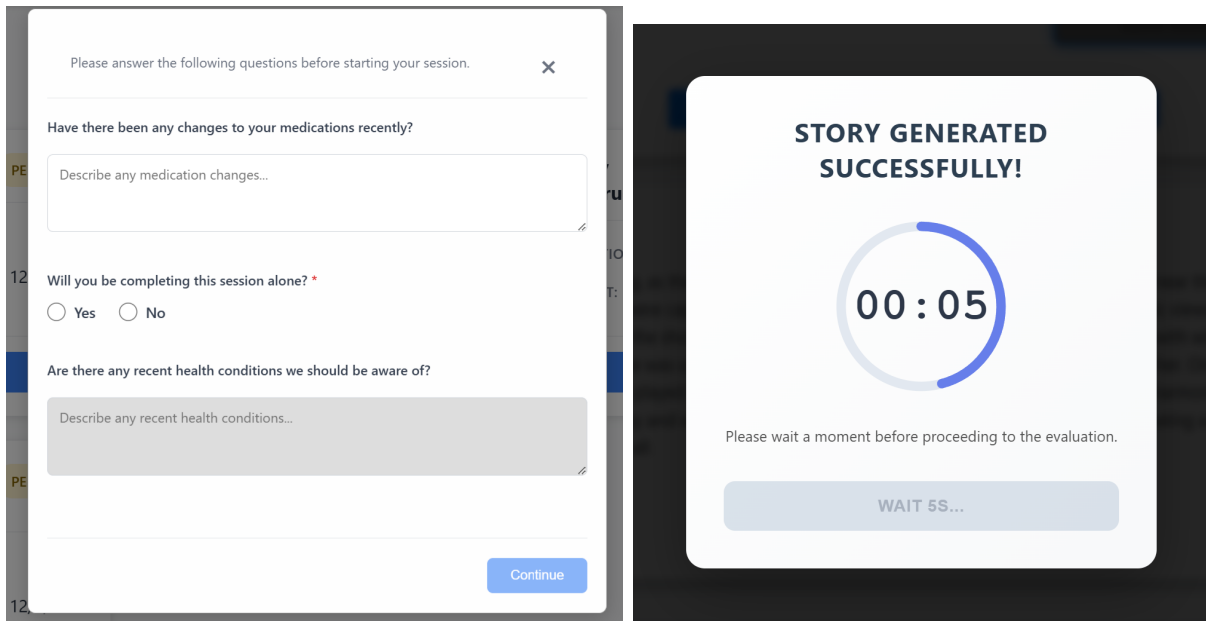


Figure 41 – Patient session list with chronological ordering and summary information

Figure 42 illustrates the two screens shown before and after the therapeutic task. The first provides instructions for the pre-evaluation, while the second introduces a brief interruption period to allow patients to reset cognitively before beginning the evaluation.



(a) Pre-evaluation instructional form

(b) Interruption screen before evaluation

Figure 42 – Screens presented before and after the therapeutic task

Once the interruption ends, the patient proceeds to the evaluation itself. For Art Exploration, the first set of questions consists of open-ended to describe the story generated displayed at Figure 43a. These questions assess narrative understanding, memory coherence, and emotional interpretation. Also, an ordering question is displayed, asking the patient to arrange story events in chronological order (Figure 43b). The backend computes event boundaries, while the frontend provides an interface for reordering.

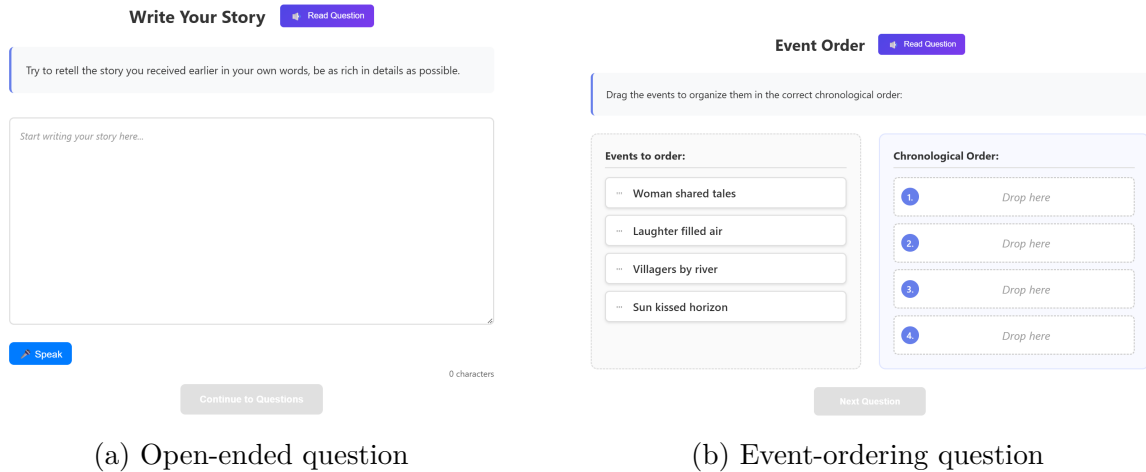


Figure 43 – Art Exploration question types: open-ended and event ordering

In Memory Reconstruction, the patient must choose again the image they originally selected for a given section, now presented with distractors to assess memory accuracy, as shown in Figure 44.

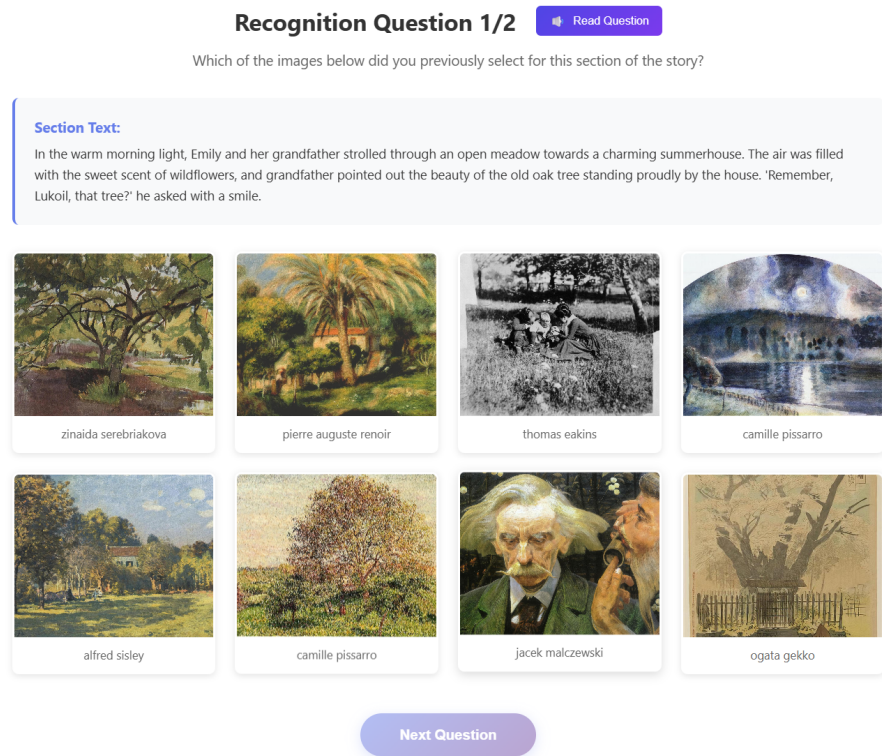


Figure 44 – Image re-selection task with two distractors

The second class of questions consists of predefined objective questions used across both modes (Art Exploration and Memory Reconstruction). Figure 45 shows these questions with multiple-choice inputs and a progress bar showing the patient's advancement through the evaluation.

Recap - Art Exploration

Progress: 4 / 5 80%

Question 2/3 [Read Question](#)

What time of day was it?

☐ Morning

☐ Afternoon

☐ Night

[Next Question](#)

Figure 45 – Example of an objective evaluation question with progress indicator

Upon completing all evaluation steps, the system generates a results screen summarizing user responses, computed metrics, and AI-assisted scoring. This layout is seen in Figure 46 and it is shared between both modes, with minor adaptations.

[← Back](#) **Session Results** [Patient Info](#)

MODE: Art Exploration COMPLETED AT: 12/7/2025, 12:00:00 AM

Session Results Dataset: wikiart Language: en

Generated Story

One morning, as the sun gently kissed the horizon, a group of villagers gathered near the Vistula River. They were captivated by the vibrant poster from 1914, advertising the scenic views that lay just beyond the shore. Among them was an old peasant woman, her face etched with wisdom and kindness. She was unlike any other, with two distinct personalities swirling within her. One day, as the villagers played by the river, the woman shared tales of yore, her dual voices harmonizing in a melody of joy and wonder. The atmosphere was light and filled with laughter, creating a cherished memory for all.

Chronological Order Correct: 1/4 25.0%	Objective Questions Correct: 0/3 0.0%	Overall Accuracy 0.0%
---	---	---

Figure 46 – Evaluation result overview including automatic scoring and response summary

For the order event question, the result screen marks which events were correctly placed and highlights any misplacements (Figure 47).

Chronological Order

Chronological Order Question 00:00:33

Order the artworks from oldest to newest

Your Order	Correct Order
1 Woman shared tales ✗	1 Sun kissed horizon
2 Villagers by river ✓	2 Villagers by river
3 Laughter filled air ✗	3 Woman shared tales
4 Sun kissed horizon ✗	4 Laughter filled air

✗ Some artworks were not in the correct chronological position.

Figure 47 – Event-order result screen showing correct and incorrect placements

For Memory Reconstruction, a detailed image comparison is shown, highlighting the patient’s chosen image versus the expected or reference image (Figure 48). This provides immediate visual insight into memory accuracy.

Image Recognition Questions

Image Question 1 of 2 00:01:32

In the warm morning light, Emily and her grandfather strolled through an open meadow towards a charming summerhouse. The air was filled with the sweet scent of wildflowers, and grandfather pointed out the beauty of the old oak tree standing proudly by the house. 'Remember, Lukoil, that tree?' he asked with a smile.

pierre auguste renoir	✓ Correct Answer alfred sisley	✗ Your Answer thomas eakins	camille pissarro	gogata gekko
jacek malczewski	Distractor zinaida serebriakova	Distractor camille pissarro		

✗ Incorrect answer

Figure 48 – Memory Reconstruction: comparison between selected and correct images

Objective-question results are also displayed, indicating whether the patient’s answers were correct and summarizing their performance (Figure 49).

Figure 49 – Objective question results with correctness feedback

Finally, a general feedback panel is presented to the patient, shown in Figure 50, which can collect qualitative insights or allow the patient to reflect on their experience. This component supports ongoing clinical improvement and patient engagement.

Figure 50 – Feedback interface used for patient reflections and system improvements

Navigation between flows is strictly role-based. Patients do not have access to the evaluation interface outside an active session, and doctors cannot perform patient-facing tasks such as Art Exploration or Memory Reconstruction. These constraints are enforced both at the routing level and through backend authorization checks, ensuring clinical reliability and proper role separation.

6.2.4 Accessibility

6.2.4.1 Overview

As the project is aimed at aiding people suffering from Alzheimer's, it is crucial for the web application mentioned above to meet accessibility standards. In order to do so,

we will follow some conventions mentioned in the Web Content Accessibility Guidelines (WCAG) (W3C, 2018). The latter was created by the World Wide Web Consortium (W3C), which is the main entity defining international standards involving the World Wide Web.

In order to make the accessibility features easy to access for any user, we created an accessibility panel that consolidates most of these features. As shown in Figure 51, this panel can be accessed via a button located in the navigation menu of the web application, making it available on every page.

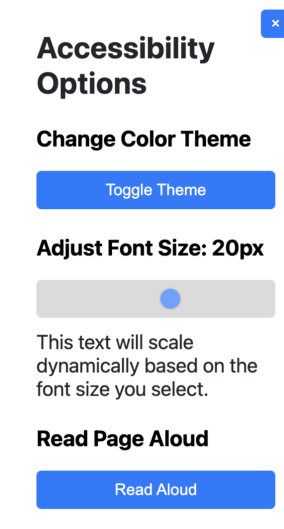


Figure 51 – Interface – Accessibility Panel

6.2.4.2 Perceivability

One of the most important aspects is font size, as elderly users may have reduced visual acuity. It is essential that they are able to resize the website content. To address this, a font size slider is included on the homepage, allowing users to scale the entire application up or down. This feature aligns with principle 1.4.4 of the WCAG. An example of this slider is shown in Figure 52.



Figure 52 – Interface – Font Size Slider

6.2.4.3 Operability

There are many standards involving the operability of the web application. For example, principle 2.1 and all of its sub-principles require that the entire interface is accessible with a keyboard. We have ensured that it is indeed the case as the user can

navigate through the pages, elements, and access all of the functionality solely with the use of the keyboard, and there are no trap elements that would block the user from exiting them without another input method.

Furthermore, guideline 2.2 is respected, as there are no time-sensitive events within the application, users can take as much time as they need when interacting with the website. Guideline 2.3 is also met, as the application contains no flashing lights or animations that could trigger seizures or cause adverse physical reactions.

The website allows for different input modalities by offering a speech-to-text alternative where needed, as illustrated in Figure 53. This feature does not restrict the use of other input forms and adheres to guideline 2.5 of the WCAG.

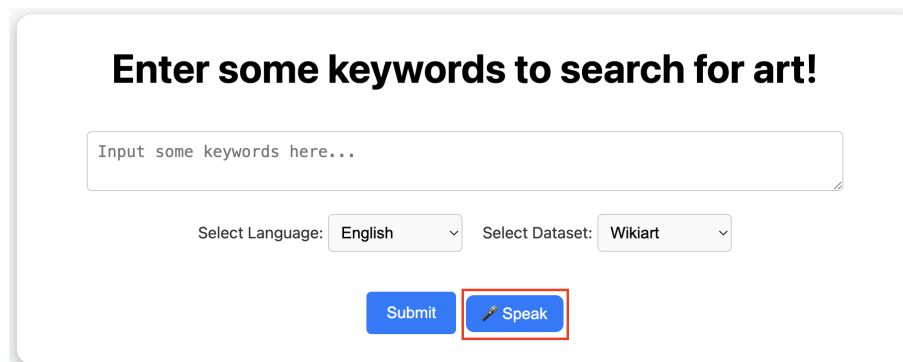


Figure 53 – Interface – Speech-to-Text

6.2.4.4 Readability

Following principle 3.1 of the WCAG, the website ensures readability by avoiding jargon or unusual vocabulary. The language of each page can be programmatically determined, and the content does not require a reading level higher than lower secondary education.

Moreover, clinical research has shown that color vision deficiencies become more common with age (Wolters Kluwer Health: Lippincott Williams & Wilkins, 2014). It is therefore beneficial to include the option for users to toggle color schemes within the interface. Further studies suggest that recommended color schemes for these individuals include warm colors (HENCOVÁ; KOTRADYOVÁ, 2023) and high-contrast options (WANG; CHO, 2022). Consequently, a button was added to the application's navigation bar, allowing users to switch between these schemes. This feature is illustrated in Figure 54.



Figure 54 – Interface – Toggle Theme

6.2.4.5 Understandability

While there is no specific WCAG standard that mandates providing an audio alternative to on-screen text, we determined that it would be a valuable addition to the application. As a result, we included a Text-to-Speech (TTS) button that can read aloud the contents of each webpage, as illustrated in Figure 55. This button is available on every page and is particularly useful for users who may have difficulty reading, enhancing their ability to navigate the application. Furthermore, the TTS functionality is implemented entirely on the frontend, offering several advantages such as native browser support, fast performance, and automatic adaptation to the user’s preferred language settings.



Figure 55 – Interface – Text-to-Speech

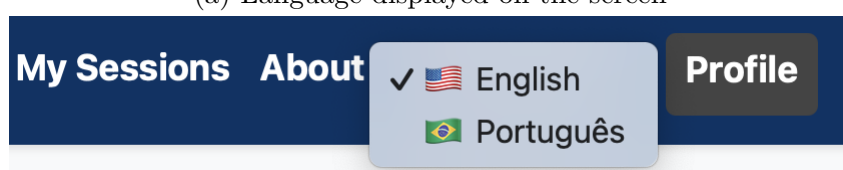
6.2.4.6 Internationalization (i18n)

The platform implements comprehensive internationalization using `react-i18next`, enabling Portuguese (pt-BR) and English (en) support essential for diverse patient populations in cognitive therapy contexts where native language familiarity directly impacts therapeutic efficacy and patient engagement.

Translation files are organized in the `src/i18n` directory, following a namespace-based structure that separates functional domains such as `common`, `artExploration`, and `memoryReconstruction`. This modular approach prevents key collisions and facilitates maintenance as the application scales. Components access translations through the `useTranslation` hook, which provides the translation function and ensures automatic re-rendering on language changes.



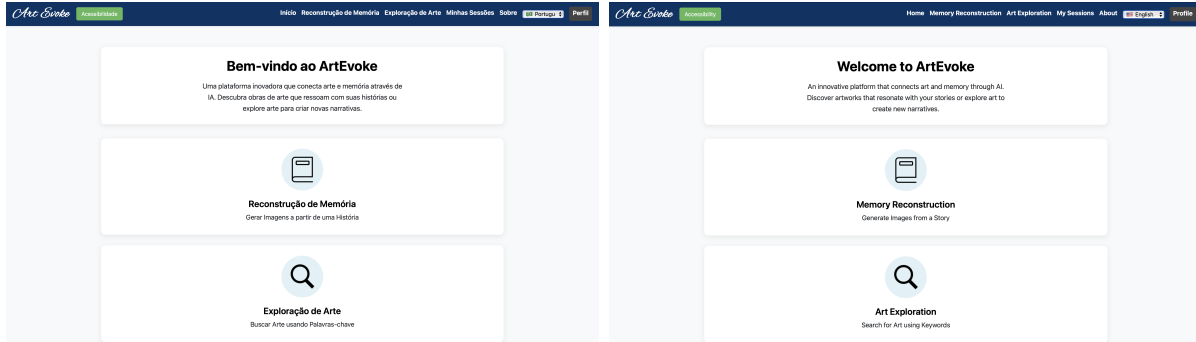
(a) Language displayed on the screen



(b) Language selector

Figure 56 – Language selector component enabling runtime language switching

Translation keys follow hierarchical dot-notation (e.g., `artExploration.selectArt.title`), promoting clear organization. The system supports interpolation for dynamic content and contextual variations, which is crucial for therapeutic modules where instructions adapt to patient selections. For instance, Memory Reconstruction feedback dynamically incorporates patient responses in both languages.



(a) English version.

(b) Portuguese version.

Figure 57 – Side-by-side comparison of localized interface elements

Translation resources are lazy-loaded based on selected language, minimizing bundle size. Language preference persists in browser local storage, maintaining user selection across sessions. The backend API receives language preference in requests, enabling appropriate content generation.

6.3 Backend Architecture

The backend is the core of the ArtEvoke system, responsible for business logic, data persistence, and integration with external AI services. It is designed as a modular, service-oriented architecture to ensure scalability, maintainability, and performance. This section details the key components of this architecture, from the application framework to the external service integrations.

6.3.1 FastAPI Application Structure

The choice of FastAPI as the web framework was a strategic decision driven by its high performance, asynchronous support, and automatic OpenAPI and JSON Schema generation. Its performance is comparable to that of NodeJS and Go, which is critical for a responsive user experience. The native support for ‘`async`’ and ‘`await`’ syntax allows for efficient handling of I/O-bound operations, such as database queries and external API calls, without blocking the main execution thread.

The application is structured in a modular fashion to promote separation of concerns and code reusability. The core logic is organized into distinct layers: routers, services, and repositories.

- **Routers (Endpoints):** Located in the `routes/` directory, these define the API interface. Unlike traditional thin controllers, the routers in this architecture act as the primary orchestrators. They handle request validation (via Pydantic), interact directly with the database using SQLAlchemy sessions for standard CRUD operations, and coordinate calls to external clients. This reduces boilerplate code for data retrieval and storage.
- **ORM Models:** Located in the `orm/` directory, these classes define the database schema using SQLAlchemy. They serve as the single source of truth for data structure and relationships. By using the ORM directly in the routers, the application takes full advantage of SQLAlchemy's capabilities, such as lazy loading and change tracking, without an intermediate repository layer.
- **API Types (DTOs):** Located in the `api_types/` directory, these Pydantic models define the request and response schemas for the API endpoints. They provide automatic validation, serialization, and documentation generation. By separating these from the ORM models, the application maintains a clear boundary between the internal data representation and the external API contract, allowing for independent evolution of both.
- **Clients & Utils:** Complex business logic, external service integrations (like AI models or Vector Databases), and reusable algorithms are encapsulated in the `clients/` and `utils/` directories.
 - **Clients:** Handle the communication with external systems (e.g., `qdrant_client.py`, `maritaca_client.py`), isolating the connection logic and API specifics.
 - **Utils:** Contain pure functions and helpers for tasks such as authentication (`auth.py`), text processing, and embedding generation, ensuring these logic pieces remain testable and reusable across different routes.

FastAPI's dependency injection system is extensively used to manage dependencies, such as database sessions and service instances. This simplifies testing by allowing mock objects to be injected during unit tests and ensures that components are loosely coupled.

6.3.2 Database Layer

The database layer is responsible for the persistent storage of all application data, including user information, artwork metadata, and generated prompts. It is composed of a relational database managed by MySQL and an Object-Relational Mapper (ORM) for seamless interaction between the Python application and the database.

MySQL was chosen as the relational database management system (RDBMS) due to its proven reliability, ACID compliance, and robust performance for structured data.

The schema is designed to be normalized to reduce data redundancy and improve data integrity. More details on the database can be found in Section 6.4. A complete visual representation of the database schema is provided in the annex (see Appendix F).

ORM Implementation

SQLAlchemy was selected as the ORM to bridge the gap between the object-oriented Python code and the relational MySQL database. The primary advantage of using an ORM is the ability to interact with the database using Python objects and methods instead of writing raw SQL queries. This improves developer productivity and reduces the risk of SQL injection vulnerabilities, as SQLAlchemy automatically sanitizes inputs.

The implementation uses SQLAlchemy's Core and ORM components. The schema's tables are mapped to Python classes (models), and the columns are mapped to class attributes. The session management is integrated with FastAPI's dependency injection system. A new database session is created for each incoming request and is automatically closed after the response is sent, ensuring that connections are properly managed and released back to the connection pool. This pattern ensures that each request operates within its own transactional scope.

6.3.3 External Services Integration

The backend architecture is not monolithic; it integrates with several specialized external services to provide its core AI-powered features. This approach allows the system to leverage best-in-class solutions for specific tasks like vector search and AI model inference. The Figure 58 illustrates it.

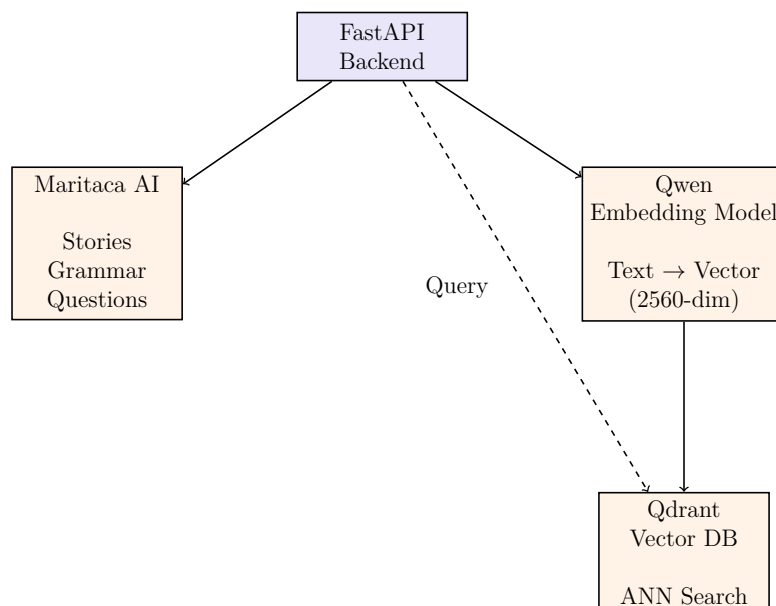


Figure 58 – External services integration architecture showing the interaction between FastAPI backend, AI services (Maritaca AI and Qwen), and Qdrant.

6.3.3.1 Vector Database (Qdrant)

To enable semantic search and find relationships between artworks and text prompts, Qdrant is used as a dedicated vector database. The integration is handled by the `QdrantClient` class in the `clients/` directory.

When a user performs a search, the backend queries the vector database with the query's embedding. The database search to find the most similar vectors from the indexed collection. The IDs of the matching items are returned to the backend, which then retrieves the full metadata from the primary MySQL database to present to the user. This two-step process combines the speed of vector search with the structured data retrieval of a relational database.

The Qdrant integration serves two primary use cases within the application. In Art Exploration Mode, it enables the discovery of semantically similar artworks based on user search queries, allowing patients to navigate through the art catalog in a more intuitive manner. In Memory Reconstruction Mode, the vector database is used to match patient responses to relevant catalog items during story exercises, facilitating the therapeutic process by connecting patient input with appropriate visual stimuli.

6.3.3.2 AI Services

The generation of creative prompts, evaluation questions, and text-correction outputs is powered by external LLM providers. The backend acts as an intermediary between the application and these AI models through dedicated client classes located in the `clients/` directory.

The primary LLM provider is **Maritaca AI**, a Brazilian language model selected for its strong performance in Portuguese-language tasks (see Section 5.4). The `MaritacaClient` encapsulates this integration, handling prompt construction, contextual formatting, and response parsing.

Maritaca AI is used extensively in Memory Reconstruction mode, where it generates narrative contexts and scenarios based on the patient's chosen artworks and therapeutic objectives. It also produces evaluation questions and answer options for both the Art Exploration and Memory Reconstruction modes, ensuring that assessment items remain coherent with the therapeutic context. Additionally, Maritaca supports grammar-correction tasks, improving the accuracy and readability of speech-to-text outputs.

For embedding generation, the system integrates with Qwen, specifically the **Qwen3-Embedding-4B** model, which produces 2560-dimensional vector representations used for semantic search in the vector database. This functionality is encapsulated in the `EmbeddingClient`, implemented using the SentenceTransformers library.

This multi-provider architecture enables the platform to leverage Maritaca AI for

high-quality Portuguese text generation and refinement, while Qwen handles computationally efficient embedding generation. The abstraction offered by the client layer ensures that AI providers can be replaced or expanded with minimal impact on the broader codebase, supporting long-term scalability and maintainability.

6.3.4 API Design and Endpoints

The API serves as the contract between the frontend and the backend, structured around FastAPI's automatic documentation and type-safe request/response handling through Pydantic models.

6.3.4.1 Request and Response Schema Architecture

The API follows a strict schema-driven design pattern where all request bodies and response structures are defined using Pydantic models located in the `api_types/` directory. This directory is organized by domain, with separate modules for each major feature area: `art_exploration.py`, `memory_reconstruction.py`, `evaluation.py`, `session.py`, `patient.py`, and others. Each module defines Data Transfer Objects (DTOs) that specify exactly what data can be sent to and received from the API.

6.3.4.2 Authentication and Authorization

Security is implemented through a JWT-based authentication system that supports both patient and doctor user types. The authentication logic is centralized in the `utils/auth.py` module. When users submit credentials to role-specific login endpoints (`/patient/login` or `/doctor/login`), the backend verifies them using a two-stage hashing approach: SHA-256 followed by `bcrypt`. Upon successful verification, a signed JWT is generated containing either a `patientId` or `doctorId` claim and an expiration timestamp. The `get_current_user` dependency extracts and decodes this token from the `Authorization: Bearer <token>` header, injecting the user's identity into route handlers through FastAPI's dependency injection system.

Authorization is enforced through specialized helper functions that perform role-based and resource-level access control. The `verify_doctor_role` function restricts administrative endpoints to doctor users, while `verify_evaluation_access` and `verify_session_access` perform ownership checks by querying the database to confirm that the authenticated user has permission to access the requested resource. These helpers also validate UUIDs before database queries, preventing malformed identifiers from causing errors. This multi-layered approach ensures that users can only access and modify data they are authorized to interact with.

6.4 Data Management

This section details how the system handles the persistence, retrieval, and processing of diverse data types, ranging from structured relational data to high-dimensional vector embeddings. The strategy prioritizes data integrity for clinical records while optimizing read operations for the extensive art catalog.

6.4.1 Embedding Generation and Storage

To enable semantic search and content-based recommendation, the system transforms data in form of images, text and then vector embeddings. This process is critical for bridging the gap between visual features and natural language descriptions provided by the users. The generation pipeline operates asynchronously from the main user interaction flow to ensure system responsiveness.

We utilized a LMM model to process the images and create their associated textual descriptions. This process phase was explained in the Section 5.2 with more details. During the data ingestion phase, a script iterates through the catalog, generating vectors for each artwork description. These embeddings were pre-compute and persistent rather than generating them on-the-fly, given the computational cost of inference on images, generation would introduce latency during user sessions.

The storage strategy for these embeddings is dual-layered. The raw vectors are stored within the vector database for indexing and retrieval. However, for versioning and backup purposes, the system also maintains serialized snapshots of these vectors in the file system. This backup ensures that the vector index can be rapidly rebuilt or migrated without re-running the expensive inference process on the entire image dataset.

6.4.2 Art Catalog Structure

The art catalog serves as the core content repository for the application. Since the system aggregates artwork from multiple disparate sources, specifically WikiArt, SemArt, and the Ipiranga Museum collection, the data management strategy required a unified schema that could accommodate varying metadata standards while maintaining the source references together. This structure can be seen in Figure 59 and more on this can be read in the section 5.1

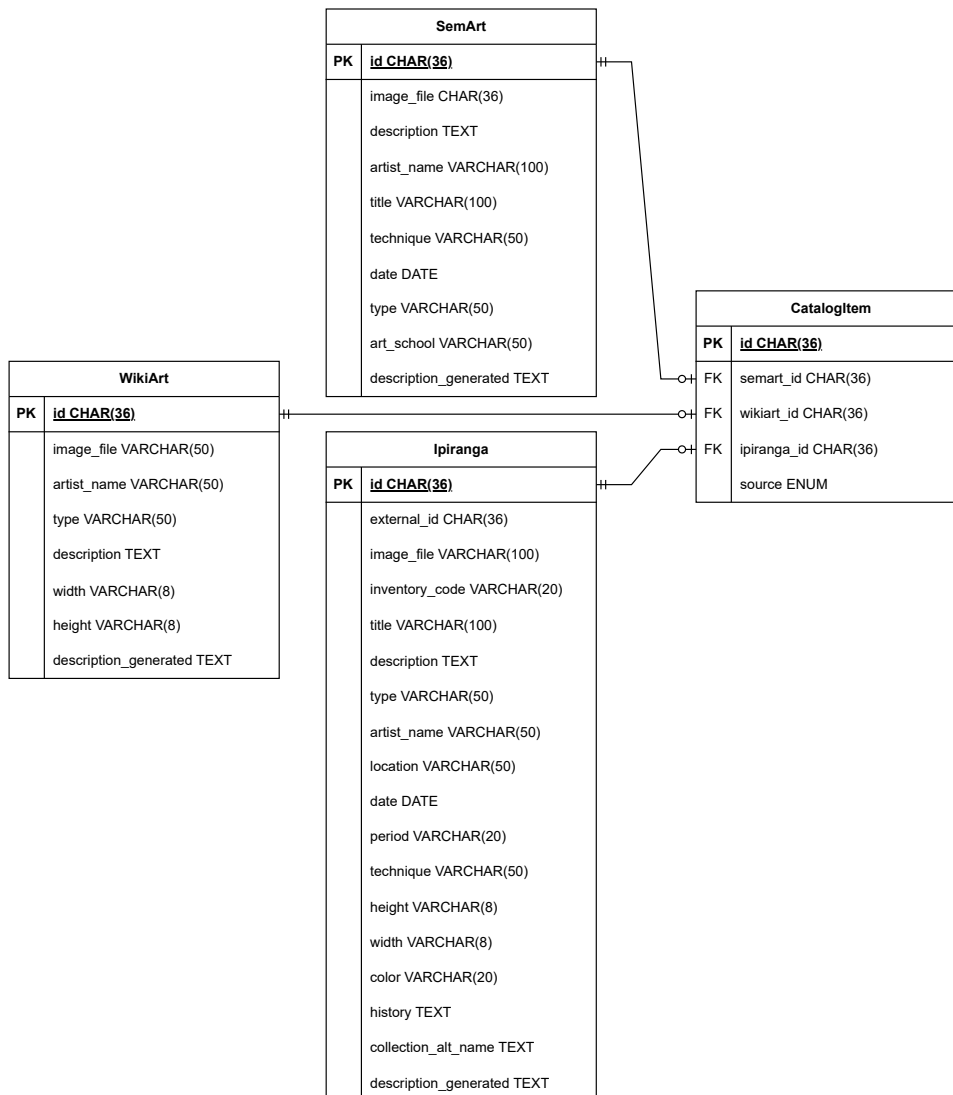


Figure 59 – Entity-Relationship Diagram of the Art Catalog normalization strategy.

The implementation utilizes a relational database approach. We established a central entity, referred to as the Catalog Item, which acts as a normalized reference point. This entity holds universal attributes such as the unique identifier and a reference to the source dataset. Linked to this central entity are specific tables for each dataset (e.g., WikiArt, Ipiranga). This separation allows the system to store unique fields, such as specific historical context for Ipiranga or artistic style tags for WikiArt, without enforcing a sparse schema on the entire catalog.

From a practical standpoint, this structure simplifies the query logic in the backend. When the application requires an artwork's details, it queries the central catalog to locate the resource and then joins the specific metadata table based on the source type. This design decouples the application logic from the specificities of data ingestion, allowing new datasets to be added in the future by simply creating a new extension table and linking it to the central catalog.

6.4.3 Interactive Modes Data Flow

Examining the end-to-end data lifecycle during the execution of the modes, we focus beyond the immediate retrieval of content to the structural persistence of the user's experience, shown in Figure 60. This ensures that every interaction is recorded as a data entity linked to the patient's history.

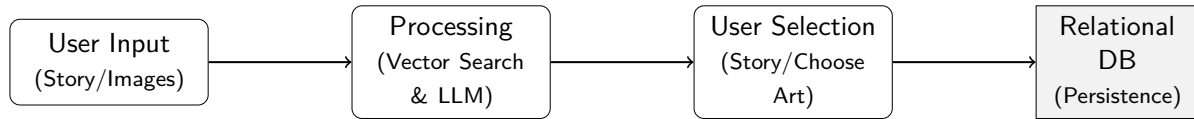


Figure 60 – End-to-end data flow: from user input to relational persistence.

The Memory Reconstruction module uses vector embeddings for semantic retrieval from the vector database, correlating images with the user's narrative. Persistence is achieved via a transaction to the *MemoryReconstruction* table, which stores the user-provided narrative, session metadata, and an ordered list of selected artwork IDs. This relational approach ensures the exact visual narrative is reproducible by referencing catalog items directly.

In the Art Exploration mode, the data flow initiates with a user prompt. The backend retrieves images using a vector database search, allowing the user to select those they wish to incorporate into a story. Once the images are selected, the *description* attribute of each image within Qdrant is retrieved. This information is passed via a prompt to the LLM to generate a story for the user. This action triggers a write operation to the *ArtExploration* table. This entity stores the generated narrative text and a direct foreign key reference to the selected *CatalogItem*, effectively binding the search experience to a permanent record.

These modes can be executed by the patient independently of a clinical session. In such cases, the data is linked only to the *Patient* table, whereas in a session, it is linked to the *Session* table as well, as will be explained in the next Section. This architectural decision allows the relational database to serve as the single source of truth for the patient's activity history, accessible via their profile regardless of clinical supervision.

6.4.4 Session Management and Evaluation Persistence

While the previous subsection described how data is processed in the platform's two interactive modes, this section explains how that data is stored within a clinical context. The system enforces a strict hierarchical data model: a Doctor manages Patients, and Patients participate in Sessions. The *Session* entity acts as the transactional boundary, ensuring that all interactions within a given timeframe are grouped together and become immutable once the session ends.

When an interaction mode is executed during a session, the transient data previously described is persisted in the relational database. To support this, we implemented

association tables that link a *Session* to its respective modes, such as *Memory Reconstruction*, *Art Exploration*, and even the *CatalogItem*, avoiding data duplication. Instead of copying artwork metadata into each session, the system stores only a foreign key reference, ensuring that if the catalog metadata is later updated, historical session records continue to reflect the canonical source.

The evaluation flow forms the final layer of this data model. After completing the mode, the patient proceeds through the interruption step, followed by the pre-evaluation step, which fills the *PreEvaluation* entity with basic session information. This is followed by answering the session questions, as further discussed in Section 3.3, and finally providing feedback. Feedback typically includes both a quantitative score and qualitative textual notes, which are stored in the dedicated *PosEvaluation* entity.

Crucially, the evaluation entity is linked not only to the session but also to the granular evaluation events, the questions and their respective answers. These are stored in specialized tables such as *ChronologicalOrderQuestion*, *ObjectiveQuestion*, *StoryOpenQuestion*, and *SelectImageQuestion*. This normalized structure allows individual evaluation data to be stored independently of the mode. It also enables the system to reconstruct the exact application state during the Review phase, allowing doctors or patients to retrieve session results showing the selected artwork alongside the feelings recorded at that moment.

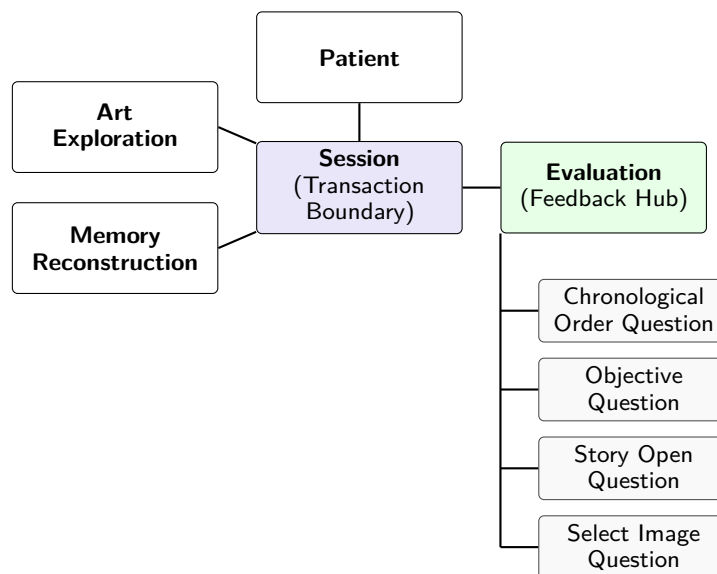


Figure 61 – Schema for session persistence and evaluation linkage, highlighting the separation between interaction modes and granular evaluation data.

This structured approach to storage supports the longitudinal analysis requirement. By querying the Evaluation table, the system can efficiently aggregate patient progression data without parsing unstructured logs, enabling the generation of progress charts and clinical insights directly from the SQL layer.

6.5 Infrastructure and Deployment

The containerization strategy facilitates horizontal scaling for stateless services (frontend and backend) while maintaining single-instance constraints for stateful services (databases). Resource limits are defined in the Docker Compose configuration to prevent resource contention, with the backend and vector database allocated higher memory limits given their computational requirements for embedding generation and similarity search operations.

6.5.1 Containerization Strategy

The ArtEvoke platform employs Docker containerization to ensure consistency across development, testing, and production environments. This approach addresses the inherent complexity of managing multiple interdependent services while maintaining reproducibility and simplifying deployment procedures. Figure 62 shows the architecture that will be explained in the next paragraphs.

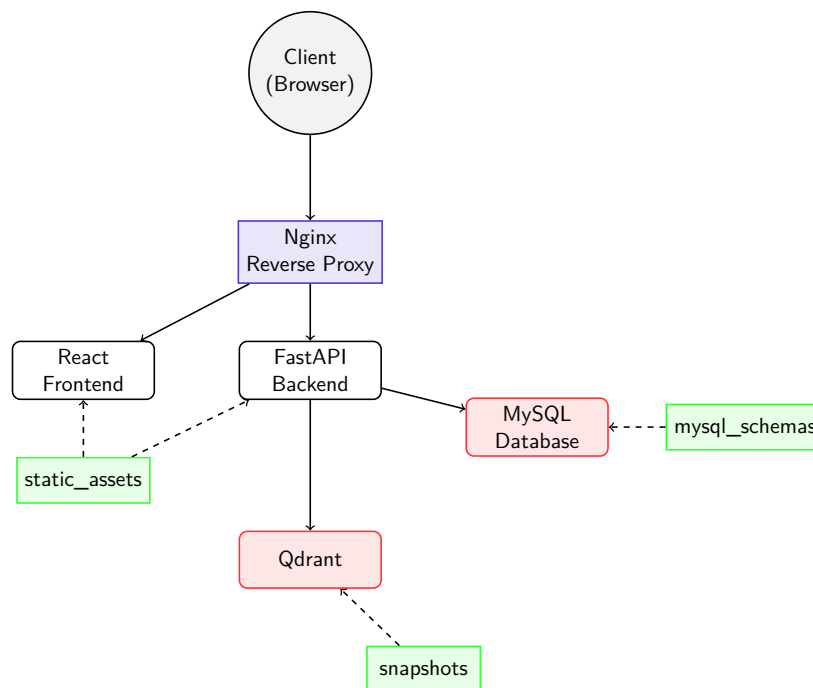


Figure 62 – Container architecture and inter-service communication flow

The system architecture comprises five containerized services: the React frontend application, FastAPI backend server, MySQL database, Qdrant vector database, and Nginx reverse proxy. Each service is encapsulated in its own Docker container with specifically defined resource boundaries and networking configurations, following microservices principles that promote isolation and independent scalability.

The frontend container builds the React application using a multi-stage Dockerfile that first compiles the production bundle using Node.js, then serves the static assets through a lightweight Nginx image. This two-stage approach minimizes the final image

size by excluding development dependencies and build tools from the production artifact. The container exposes port 80 internally, which is mapped through the reverse proxy for external access.

The backend container runs the FastAPI application using Python 3.13, with all dependencies managed through `uv` for deterministic builds, as explained in (React Team, 2025). The Dockerfile implements layer caching optimization by copying dependency files before application code, ensuring that dependency installation only occurs when requirements change. Environment variables for database connections, API keys, and service endpoints are injected at runtime through Docker Compose environment files, maintaining separation between configuration and code.

Data persistence is handled through Docker volumes mounted to each containers. The MySQL database utilizes named volumes for data storage and configuration files, ensuring data survives container restarts. Similarly, Qdrant employs volumes for vector index persistence, while static assets (artwork images) are mounted as read-only volumes accessible to the backend containers. This volume strategy separates ephemeral container state from persistent application data. Table 21 summarizes these discussed topics.

Volume Name	Mount Point	Purpose
mysql_data	/var/lib/mysql	Persistent storage for MySQL database files
qdrant_storage	/qdrant/storage	Vector database indices and collections
qdrant_snapshots	/qdrant/snapshots	Backup snapshots for vector database
static_assets	/app/static	Artwork images (WikiArt, SemArt, Ipiranga)

Table 21 – Docker volumes and their purposes

6.5.2 Service Orchestration

Docker Compose orchestrates the multi-container application stack, defining service dependencies, networking, and startup sequences. The composition file specifies a custom bridge network that enables service discovery through DNS resolution, allowing containers to communicate using service names rather than IP addresses. This abstraction simplifies configuration and enhances portability across different hosting environments.

Service dependencies are declared to enforce proper initialization order. The backend service depends on both MySQL and Qdrant availability, preventing failures due to missing database connections. Docker Compose implements dependency checking through health checks and wait conditions, ensuring that dependent services only start after their prerequisites are operational. The MySQL container exposes a health check endpoint that validates database readiness, while Qdrant’s REST API provides status verification.

Environment configuration is centralized through dedicated environment files located in the `env/` directory. Each service has its own environment file (e.g., `.backend.env`, `.mysql.env`) containing service-specific parameters such as credentials, connection strings, and feature flags. This separation allows environment-specific configurations without modifying the orchestration definition, supporting different configurations for development, staging, and production deployments.

The orchestration includes a startup script (`qdrant-startup.sh`) that initializes the vector database with pre-computed embeddings from the artwork catalogs. This script executes during container initialization, loading snapshots for the three artwork collections (WikiArt, SemArt, and Ipiranga Museum). The automated initialization ensures that the vector search functionality is immediately available after deployment without manual intervention.

Port mapping in the orchestration configuration exposes only essential services to the host network. The Nginx reverse proxy is the sole service with external port exposure (ports 80 and 443), while internal services communicate through the private Docker network. This configuration implements defense-in-depth security by limiting the attack surface and preventing direct access to backend services and databases. Table 22 summarizes the ports structure.

Service	Internal Port	External Port	Access
Nginx (HTTP)	80	80	Public
Nginx (HTTPS)	443	443	Public
Frontend	3000	-	Internal only
Backend API	8000	-	Internal only
MySQL	3306	3307	Dev/Admin access
Qdrant	6333	6333	Dev/Internal

Table 22 – Service port mappings and exposure configuration

The composition supports both local development and production deployment through configuration profiles. Development mode enables hot-reloading for the frontend and backend, mounts source code as volumes for live updates, and exposes additional ports for debugging tools. Production mode optimizes for performance and security, disabling debug features, implementing read-only file systems where appropriate, and enforcing stricter resource constraints.

6.5.3 Reverse Proxy Configuration

Nginx serves as the reverse proxy and web server, Figure 63, providing a unified entry point for client requests and routing traffic to appropriate backend services. The proxy configuration implements several critical functions: request routing, SSL termination, load balancing preparation, static file serving, and security header injection.

The routing configuration distinguishes between API requests and static content based on URL path prefixes. Requests to `/api/*` are proxied to the FastAPI backend

container, while requests to `/static/*` serve artwork images directly from mounted volumes. Frontend application requests are handled by serving the React build artifacts, with HTML5 History API support implemented through fallback routing to `index.html` for client-side navigation.

Two configuration variants exist: `nginx-local.conf` for development environments and `nginx-prod.conf` for production deployments. The local configuration operates on HTTP port 80 with relaxed security policies to facilitate development workflows. The production configuration enforces HTTPS through SSL certificate configuration, implements HTTP to HTTPS redirection, and applies strict security headers including Content Security Policy (CSP), HSTS, and X-Frame-Options.

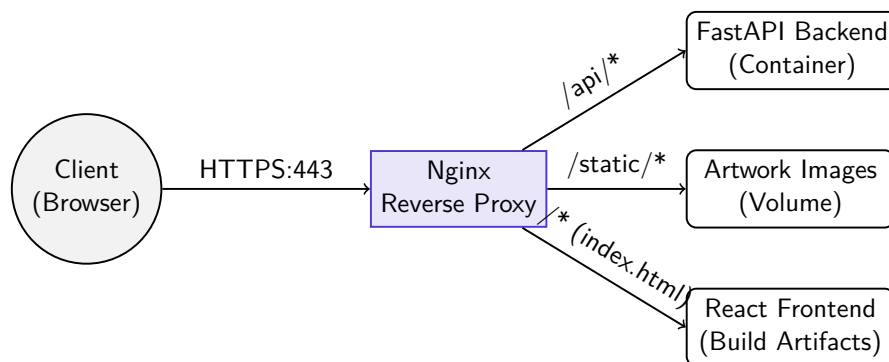


Figure 63 – Request routing logic in the Nginx reverse proxy

Proxy configuration includes optimized buffering and timeout settings tailored to the application’s characteristics. Static file serving enables browser caching through appropriate Cache-Control headers, reducing bandwidth consumption and improving client-side performance.

Security headers are systematically applied to all responses. The CSP header restricts resource loading to approved sources, mitigating XSS attacks. The X-Content-Type-Options header prevents MIME-sniffing vulnerabilities, while X-Frame-Options protects against clickjacking attacks. This forward-looking configuration facilitates future implementation of collaboration features or live session monitoring without requiring infrastructure changes. The modular configuration structure allows easy addition of new upstream services or routing rules as the system evolves.

6.5.4 Database Administration

Database administration encompasses both the operational MySQL database. The MySQL container uses version 8.0, selected for its JSON data type support, improved performance characteristics, and enhanced security features compared to earlier versions. The database initialization process leverages SQL migration scripts organized in the `data/db/` directory structure.

Schema management follows a sequential migration approach, with files prefixed alphabetically to enforce execution order. The `A_TABLES.sql` script defines the core relational schema including user accounts, patient records, session management tables, and therapy module data structures. Subsequent scripts (prefix B) populate catalog tables with artwork metadata from the three collections, establishing the foundational data required for art exploration and memory reconstruction modules. The `C_POPULATE_CATALOGITEM.sql` script creates the unified catalog view, while `Z_POPULATE_INITIAL.sql` seeds initial administrative data.

The database configuration file (`my.cnf`) customizes MySQL server parameters for the application's workload characteristics. Character set configuration enforces UTF-8 encoding (*utf8mb4*) throughout the system, ensuring proper handling of international text and special characters in artwork metadata and user-generated content. Connection pool settings are tuned to accommodate the expected concurrent user load, while query cache parameters optimize read-heavy operations common in art catalog browsing.

6.6 Platform Access

The **ArtEvoke** platform is **publicly accessible** at www.artevoke.com.br, offering users the opportunity to explore its features and interact with the prototype. Additionally, the full source code, test scripts, and documentation are available in the open **GitHub repository**: [ArtEvoke](#). These resources are provided to encourage further exploration and improvement of the platform by researchers and developers interested in AI-assisted cognitive stimulation.

7 Conclusions and Further Work

7.1 Summary and Main Findings

This thesis presented the development and implementation of **ArtEvoke**, a comprehensive AI-powered platform designed to support non-pharmacological interventions for individuals with Alzheimer’s disease. Architected around two core interaction modes, **Memory Reconstruction** and **Art Exploration**, the system integrates state-of-the-art models selected through extensive benchmarking to generate high-fidelity image descriptions, perform robust multilingual retrieval of artworks, and create culturally relevant narratives. Furthermore, the work successfully transitioned from experimental concepts to a functional prototype by implementing a service-oriented architecture that ensures persistent vector management and reliable data storage. Finally, a key contribution is the establishment of a **Clinical Evaluation Framework** that captures patient responses within structured sessions, transforming the platform into a measurable therapeutic instrument for longitudinal cognitive monitoring.

7.2 Demonstration at Ipiranga Museum

To validate the practical applicability of *ArtEvoke* and assess its suitability for the target demographic, a field demonstration was conducted on December 4th, 2025, at the Ipiranga Museum (Museu Paulista). This initiative took place within a multidisciplinary partnership involving the Museum, the University of São Paulo Medical School (FMUSP), and the Polytechnic School (Poli-USP). All interactions were carried out using the museum’s tablets, which accessed the ArtEvoke platform through a standard web browser, as shown in Figure 64. Approximately 30 elderly visitors participated in the session, from which ten volunteered to test the platform in full.



Figure 64 – Demonstration at the Ipiranga Museum: group instructions (left) and assisted interaction (right).

7.2.1 Participant Demographics

Figure 65 summarizes the demographic characteristics of the ten volunteers, including age and education level.

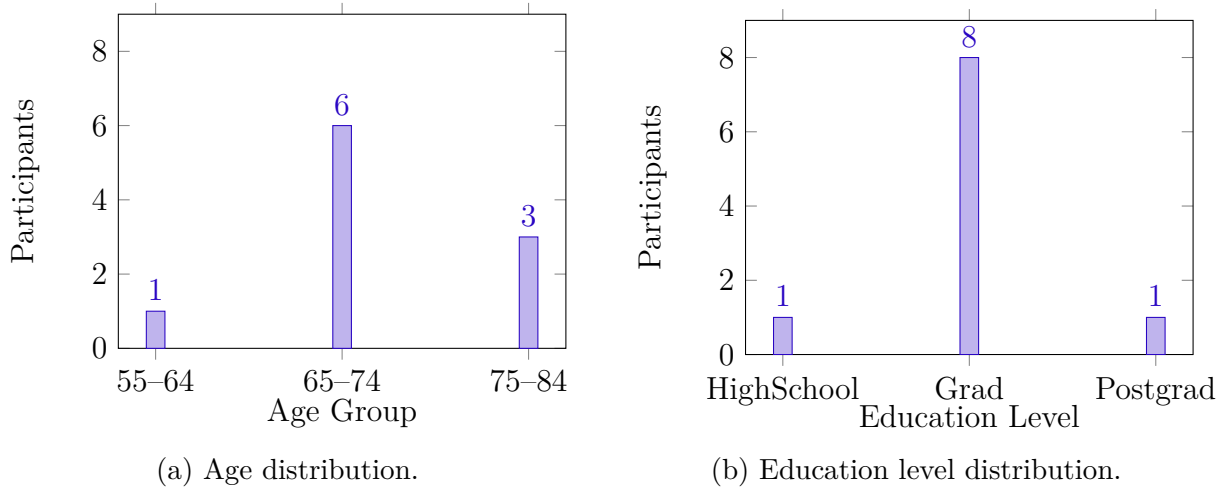


Figure 65 – Aggregated demographic characteristics of platform testers.

7.2.2 Evaluation Scores

Participants completed both modes of the platform, **Art Exploration (AE)** and **Memory Reconstruction (MR)**, followed by an integrated evaluation where they rated the experience and perceived difficulty.

Across all ten participants, the following aggregated metrics were observed:

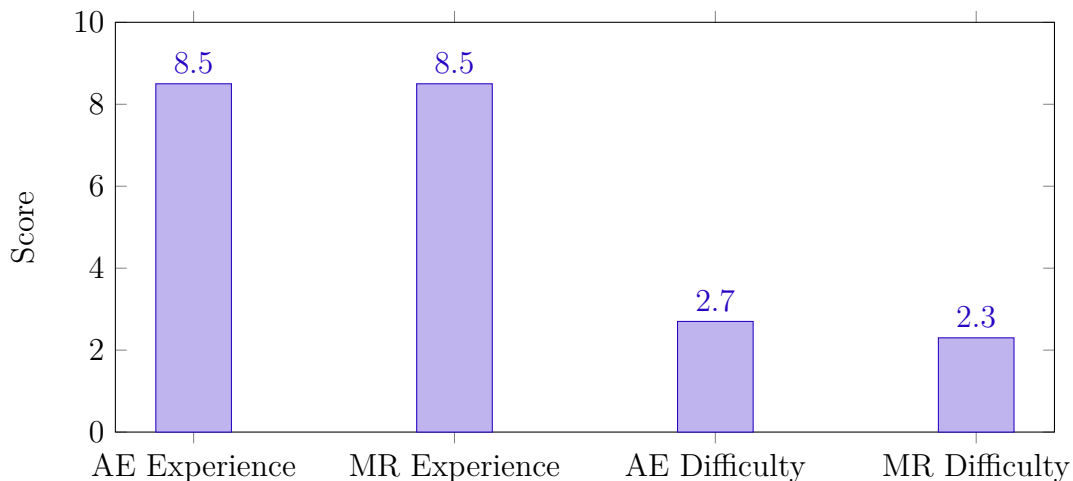


Figure 66 – Average experience and difficulty ratings for both platform modes.

7.2.3 Qualitative Observations

Participants also provided written feedback that offered valuable insights for future refinements. Common themes included:

- **Clarity of Instructions:** some participants requested clearer guidance during the activities.
- **Content Alignment:** a few participants mentioned that certain artworks or generated stories did not perfectly match their expectations.
- **Performance and Engagement:** several participants described the system as “easy,” “simple,” and “very enjoyable.”

These qualitative insights are aligned with known accessibility considerations for older adults and directly informed the improvements discussed in later sections.

7.2.4 General Outcome

Overall, the demonstration confirmed that *ArtEvoke* is an engaging and meaningful tool for elderly users, successfully fostering interaction with both artistic content and personal memories. However, the results also reinforced that user experience remains of utmost importance and should be carefully revisited. In particular, the user interface would benefit from simplification, with clearer visual cues and more intuitive navigation tailored to the cognitive and perceptual needs of older adults. Furthermore, the observations indicate that, even with interface improvements, participants still require some degree of external guidance during the activities.

7.3 Limitations and Improvement Areas

Several limitations regarding the current implementation and scope of this study should be acknowledged:

- **User Interface and Accessibility:** Although the interface is functional, it requires further optimization to fully meet the needs of the target demographic, elderly individuals, potentially with cognitive impairments. The instructional design for the interaction modes needs to be simplified, and the visual layout must be made more intuitive and "friendly" to minimize cognitive load and frustration during use.
- **Contextual Relevance of Evaluation Questions:** The automated generation of evaluation answers occasionally produces response options that lack direct relevance to the specific narrative or selected artworks. Improving the logic behind question generation is crucial to ensure that the assessment phase remains coherent with the story.
- **Hardware and Software Constraints:** some high-parameter models that were initially of interest could not be evaluated due to VRAM insufficiency or driver

incompatibility, which may have precluded the use of higher-performing options for image description and retrieval.

- **Semantic Alignment in Retrieval:** Despite the use of advanced embedding models, there are instances where the retrieved images diverge from the narrative context of the user's story. Future work must focus on optimizing the retrieval of art works, ensuring that the visual semantic matching is more consistent with the story.
- **System Scalability and Robustness:** The current prototype focuses on functionality for single-user scenarios. However, scalability remains a concern; concurrent usage by multiple patients could lead to latency. Furthermore, the codebase requires refactoring to improve error handling and simplify execution flows.

7.4 Future Work

Building upon the findings and the identified limitations of this prototype, several strategic directions are proposed for the continued development of ArtEvoke:

- **Clinical Trials and Longitudinal Studies:** The most critical next step is the execution of formal clinical trials with patients diagnosed with Alzheimer's disease. Utilizing the *Clinical Evaluation Framework* developed in this work, future studies should aim to quantify the platform's impact on memory recall and emotional engagement over time. Supervision by healthcare professionals will be essential to validate the safety and efficacy of the intervention in real-world therapeutic settings.
- **User Experience Refinement for Target Population:** One of the main insights from the demonstration phase was the need to further simplify the interaction model for older adults and individuals with cognitive impairments. Future iterations should focus on reducing interface complexity, minimizing decision points, and incorporating clearer visual cues and guided onboarding. Additionally, usability testing with representative users will be essential to validate whether these refinements effectively reduce cognitive load and improve overall accessibility of the platform.
- **Deep Personalization and Profiling:** The current retrieval system treats queries in isolation. Future development should implement a user profiling mechanism that weighs retrieval results based on the patient's biographical data. This would allow the system to prioritize artworks and narratives that are culturally familiar and emotionally resonant, reducing the likelihood of retrieving generic or contextually irrelevant content.
- **Dynamic Text Segmentation and Retrieval Diversity:** Addressing the redundancy observed in image retrieval, future work should replace the text segmentation

with dynamic, semantic-aware segmentation strategies. Furthermore, implementing diversity-promoting algorithms (such as Maximal Marginal Relevance) during the vector search would ensure that the retrieved set of images offers a broader visual perspective of the narrative.

- **Multisensory Expansion (Music and Audio):** To enhance the immersive quality of reminiscence therapy, future iterations should incorporate auditory stimuli. A "Music & Art" mode could be developed, where the system retrieves not only artworks but also music tracks contemporary to the patient's youth or thematically linked to the selected images. This multimodal approach aligns with evidence suggesting that combining visual and auditory cues can trigger deeper memory consolidation.
- **Personal Archive Integration:** While the current platform relies on public art datasets, integrating a feature that allows caregivers to upload personal family photographs would significantly expand the platform's utility. The system could then use the existing embedding pipeline to analyze these personal photos and weave them into the generated narratives, creating a seamless blend of art history and personal biography.

Addressing these areas will help make the platform more robust, flexible, and better suited for real-world use. While ArtEvoke began as a research prototype, it has potential to become a practical tool to support memory care. To get there, ongoing collaboration with healthcare professionals and the people who will actually use the platform is essential.

Bibliography

- ABONIZIO, H. et al. *Sabiá-3 Technical Report*. 2025. Disponível em: <<https://arxiv.org/abs/2410.12049>>. 51, 52, 142
- AGARAP, A. F. *Deep Learning using Rectified Linear Units (ReLU)*. 2019. Disponível em: <<https://arxiv.org/abs/1803.08375>>. 27
- AIZAWA, A. An information-theoretic perspective of tf-idf measures. *Information Processing Management*, v. 39, n. 1, p. 45–65, 2003. ISSN 0306-4573. Disponível em: <<https://www.sciencedirect.com/science/article/pii/S0306457302000213>>. 33
- ALAM, A.; RABBANI, M. G.; PRYBUTOK, V. R. Effectiveness, adoption determinants, and implementation challenges of ict-based cognitive support for older adults with mci and dementia: A prisma-compliant systematic review and meta-analysis (2015–2025). *Healthcare*, v. 13, n. 12, 2025. ISSN 2227-9032. Disponível em: <<https://www.mdpi.com/2227-9032/13/12/1421>>. 25, 26
- ALAYRAC, J.-B. et al. *Flamingo: a Visual Language Model for Few-Shot Learning*. 2022. Disponível em: <<https://arxiv.org/abs/2204.14198>>. 32
- Alzheimer's Association. 2024 alzheimer's disease facts and figures. *Alzheimer's & Dementia*, v. 20, n. 5, p. 3708–3821, 2024. Accessed: 2025-01-15. Disponível em: <<https://alz-journals.onlinelibrary.wiley.com/doi/abs/10.1002/alz.13809>>. 19
- ANGELUCCI, F. et al. Integrating ai in fighting advancing alzheimer: diagnosis, prevention, treatment, monitoring, mechanisms, and clinical trials. *Current Opinion in Structural Biology*, v. 87, p. 102857, 2024. ISSN 0959-440X. Disponível em: <<https://www.sciencedirect.com/science/article/pii/S0959440X24000848>>. 22, 23
- Anthropic . *Claude 3.7 Sonnet Announcement*. 2024. Accessed: 2025-04-07. Disponível em: <<https://www.anthropic.com/news/claude-3-7-sonnet>>. 139, 140
- Anthropic. *Claude 3.5 Haiku*. 2024. Accessed: 2025-04-07. Disponível em: <<https://www.anthropic.com/claude/haiku>>. 139, 140
- ARCAS, B. A. y. Do large language models understand us? *Daedalus*, v. 151, n. 2, p. 183–197, 05 2022. ISSN 0011-5266. Disponível em: <https://doi.org/10.1162/daed_a_01909>. 27
- AşİRET, G.; KAPUCU, S. The effect of reminiscence therapy on cognition, depression, and activities of daily living for patients with alzheimer disease. *Journal of geriatric psychiatry and neurology*, v. 29, 08 2015. 21
- BAI, S. et al. *Qwen2.5-VL Technical Report*. 2025. Disponível em: <<https://arxiv.org/abs/2502.13923>>. 49, 50, 139, 140
- BAJPAI, S. et al. Development and validation of cognitive training intervention for alzheimer's disease (cti-ad): A picture-based interventional program. *Dementia (London, England)*, SAGE Publications, v. 19, n. 4, p. 1203–1219, May 2020. ISSN 1741-2684. Epub 2018 Sep 4. Disponível em: <<https://doi.org/10.1177/1471301218797043>>. 22

- BARRAGAN, C. J. et al. Cognitive trajectory in alzheimer disease and clinical mimics. *Neurology Open Access*, v. 1, n. 3, p. e000034, 2025. Disponível em: <https://www.neurology.org/doi/abs/10.1212/WN9.0000000000000034>. 23, 24
- BORISOV, A.; GARKUSHA, I. Semantic image search at scale with qdrant. *Qdrant Use Cases*, 2023. Disponível em: <https://qdrant.tech/documentation/use-cases/semantic-image-search>. 39
- CAMPISI, M. et al. Mitigating cellular aging and enhancing cognitive functionality: visual arts-mediated cognitive activation therapy in neurocognitive disorders. *Frontiers in Aging Neuroscience*, v. 16, 2024. ISSN 1663-4365. Disponível em: <https://doi.org/10.3389/fnagi.2024.1354025>. 22
- Centers for Disease Control and Prevention. *About Dementia*. 2024. Accessed: 2025-01-17. Disponível em: <https://www.cdc.gov/alzheimers-dementia/about>. 19, 20
- CHEN, E. et al. *From Structured Prompts to Open Narratives: Measuring Gender Bias in LLMs Through Open-Ended Storytelling*. 2025. Disponível em: <https://arxiv.org/abs/2503.15904>. 76
- CHEN, H. et al. *Multi-Modal Generative AI: Multi-modal LLM, Diffusion and Beyond*. 2024. Disponível em: <https://arxiv.org/abs/2409.14993>. 31, 32
- CHEN, J. et al. *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. 2024. Disponível em: <https://arxiv.org/abs/2402.03216>. 50, 141
- CHENG, S.-T. Dementia caregiver burden: a research update and critical analysis. *Current Psychiatry Reports*, v. 19, n. 9, p. 64, 2017. ISSN 1535-1645. Disponível em: <https://doi.org/10.1007/s11920-017-0818-2>. 22
- CHERTI, M. et al. Reproducible scaling laws for contrastive language-image learning. In: *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2023. p. 2818–2829. Disponível em: <http://dx.doi.org/10.1109/CVPR52729.2023.00276>. 31
- CHUNG, J. et al. *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling*. 2014. Disponível em: <https://arxiv.org/abs/1412.3555>. 27, 28
- DENG, J. et al. Imagenet: A large-scale hierarchical image database. In: *2009 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2009*. United States: IEEE Computer Society, 2009. (2009 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2009), p. 248–255. ISBN 9781424439911. Publisher Copyright: © 2009 IEEE.; 2009 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2009 ; Conference date: 20-06-2009 Through 25-06-2009. 48
- DEVLIN, J. et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. Disponível em: <https://arxiv.org/abs/1810.04805>. 29, 30, 34, 36
- DOSOVITSKIY, A. et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. Disponível em: <https://arxiv.org/abs/2010.11929>. 31
- DOUZE, M. et al. *The Faiss library*. 2025. Disponível em: <https://arxiv.org/abs/2401.08281>. 36, 37, 38

DOVE, A. et al. The impact of diabetes on cognitive impairment and its progression to dementia. *Alzheimer's & Dementia: The Journal of the Alzheimer's Association*, Wiley Periodicals LLC on behalf of the Alzheimer's Association, v. 17, n. 11, p. 1769–1778, Nov 2021. ISSN 1552-5279. Disponível em: <<https://doi.org/10.1002/alz.12482>>. 20

Facebook AI Research. *Faiss Wiki*. 2024. <<https://github.com/facebookresearch/faiss/wiki>>. Accessed: 2025-05-15. 37, 38

FANG, Y. et al. Injecting semantic concepts into end-to-end image captioning. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. [s.n.], 2022. p. 18009–18018. Disponível em: <https://openaccess.thecvf.com/content/CVPR2022/papers/Fang_Injecting_Semantic_Concepts_Into_End-to-End_Image_Captioning_CVPR_2022_paper.pdf>. 36

FOLSTEIN, M. F.; FOLSTEIN, S. E.; MCHUGH, P. R. “mini-mental state”: A practical method for grading the cognitive state of patients for the clinician. *Journal of Psychiatric Research*, v. 12, n. 3, p. 189–198, 1975. ISSN 0022-3956. Disponível em: <<https://www.sciencedirect.com/science/article/pii/0022395675900266>>. 25

GABRIÉ, M. et al. *Neural networks: from the perceptron to deep nets*. 2023. Disponível em: <<https://arxiv.org/abs/2304.06636>>. 27

GARCIA, N.; VOGIATZIS, G. *How to Read Paintings: Semantic Art Understanding with Multi-Modal Retrieval*. 2018. Disponível em: <<https://arxiv.org/abs/1810.09617>>. 59, 60, 70

GOLLIN, D. et al. Improvement of functional status in alzheimer's disease patients after cognitive activation therapy (cat). *Non-Pharmacological Therapies in Dementia*, v. 3, p. 23–34, 2012. 21

GOODFELLOW, I. J. et al. *Generative Adversarial Networks*. 2014. Disponível em: <<https://arxiv.org/abs/1406.2661>>. 32

GOOGLE. *Gemini 2 Family Expands with Flash and Pro Models*. 2025. Accessed: 2025-04-07. Disponível em: <<https://developers.googleblog.com/en/gemini-2-family-expands/>>. 139, 140

GORDO, A.; LARLUS, D. Beyond instance-level image retrieval: Leveraging captions to learn a global visual representation for semantic retrieval. 2017. Disponível em: <http://openaccess.thecvf.com/content_cvpr_2017/html/Gordo_Beyond_Instance-Level_Image_CVPR_2017_paper.html>. 35, 36

GOYAL, Y. et al. Making the V in VQA matter: Elevating the role of image understanding in Visual Question Answering. In: *Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2017. 48

HENCOVÁ, M.; KOTRADYOVÁ, V. Colour in the environment for older adults. *Architecture Papers of the Faculty of Architecture and Design STU*, v. 28, n. 4, p. 15–23, 2023. Disponível em: <<https://doi.org/10.2478/alfa-2023-0021>>. 106

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. *Neural Computation*, v. 9, n. 8, p. 1735–1780, 11 1997. ISSN 0899-7667. Disponível em: <<https://doi.org/10.1162/neco.1997.9.8.1735>>. 28, 30

- HUANG, D. et al. From large language models to large multimodal models: A literature review. *Applied Sciences*, v. 14, n. 12, 2024. ISSN 2076-3417. Disponível em: <https://www.mdpi.com/2076-3417/14/12/5068>. 27, 30, 31, 32
- Hugging Face. *MTEB Leaderboard*. <https://huggingface.co/spaces/mteb/leaderboard>. Accessed: 2025-05-20. 51, 141
- HUNTLEY, J. D. et al. Adaptive working memory strategy training in early alzheimer's disease: randomised controlled trial. *The British Journal of Psychiatry*, The Royal College of Psychiatrists, v. 210, n. 1, p. 61–66, January 2017. ISSN 1472-1465. Epub 2016 Oct 6. Disponível em: <https://doi.org/10.1192/bjp.bp.116.182048>. 21
- HÖTTING, K.; RÖDER, B. Beneficial effects of physical exercise on neuroplasticity and cognition. *Neuroscience Biobehavioral Reviews*, v. 37, n. 9, Part B, p. 2243–2257, 2013. ISSN 0149-7634. CNTRICS: Modeling psychosis related cognition in animal systems to enhance translational research + Life-Span Plasticity of Brain and Behavior: A Cognitive Neuroscience Perspective. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0149763413001012>. 21
- ILHARCO, G. et al. *OpenCLIP*. Zenodo, 2021. Disponível em: <https://doi.org/10.5281/zenodo.5143773>. 50, 51, 141
- JOHNSON, J.; DOUZE, M.; JÉGOU, H. *Billion-scale similarity search with GPUs*. 2017. Disponível em: <https://arxiv.org/abs/1702.08734>. 36
- JOHNSON, J.; DOUZE, M.; JÉGOU, H. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, IEEE, v. 7, n. 3, p. 535–547, 2019. 38
- KAKARLA, S.; VENKATA, G. S. B.; GADDAM, A. How does a multilingual lm handle multiple languages? In: *Proceedings of the Workshop on Analyzing and Interpreting Neural Networks for NLP (BlackboxNLP)*. Toronto, Canada: Association for Computational Linguistics, 2023. p. 435–445. Disponível em: <https://aclanthology.org/2023.blackboxnlp-1.39>. 35
- LanguageTool Contributors. *LanguageTool: Open Source Grammar and Style Checker*. 2024. <https://languagetool.org/>. Accessed: 2025-05-17. 83
- LARSON, E. B. et al. Survival after initial diagnosis of alzheimer disease. *Annals of Internal Medicine*, v. 140, n. 7, p. 501–509, 2004. PMID: 15068977. Disponível em: <https://www.acpjournals.org/doi/abs/10.7326/0003-4819-140-7-200404060-00008>. 20
- LEWIS, M. et al. *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension*. 2019. Disponível em: <https://arxiv.org/abs/1910.13461>. 31
- LI, J. et al. *BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models*. 2023. Disponível em: <https://arxiv.org/abs/2301.12597>. 32, 35, 36
- LI, J. et al. *A Survey on Benchmarks of Multimodal Large Language Models*. 2024. Disponível em: <https://arxiv.org/abs/2408.08632>. 32

- LI, J. et al. Comprehending and ordering semantics for image captioning. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. [s.n.], 2022. p. 17963–17972. Disponível em: https://openaccess.thecvf.com/content/CVPR2022/papers/Li_Comprehending_and_Ordering_Semantics_for_Image_Captioning_CVPR_2022_paper.pdf. 36
- LI, K. et al. Image-text embedding learning via visual and textual semantic reasoning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022. Disponível em: <https://ieeexplore.ieee.org/abstract/document/9706340>. 35
- LI, X. et al. Non-drug therapies for alzheimer’s disease: A review. *Neurology and Therapy*, v. 12, n. 1, p. 39–72, 2023. ISSN 2193-6536. Disponível em: <https://doi.org/10.1007/s40120-022-00416-x>. 20, 21
- LI, Z. et al. *Towards General Text Embeddings with Multi-stage Contrastive Learning*. 2023. Disponível em: <https://arxiv.org/abs/2308.03281>. 50, 141
- LIU, H. et al. *Improved Baselines with Visual Instruction Tuning*. [S.l.]: arXiv:2310.03744, 2023. 49, 50, 139, 140
- LIU, H. et al. *LLaVA-NeXT: Improved reasoning, OCR, and world knowledge*. 2024. Disponível em: <https://llava-vl.github.io/blog/2024-01-30-llava-next/>. 49, 50, 139, 140
- LIU, Z. et al. *SpinQuant: LLM quantization with learned rotations*. 2025. Disponível em: <https://arxiv.org/abs/2405.16406>. 49, 50, 52, 140, 142
- LIVINGSTON, G. et al. Dementia prevention, intervention, and care: 2020 report of the *Lancet* commission. *The Lancet*, Elsevier, v. 396, n. 10248, p. 413–446, ago. 2020. ISSN 0140-6736. Disponível em: [https://doi.org/10.1016/S0140-6736\(20\)30367-6](https://doi.org/10.1016/S0140-6736(20)30367-6). 21
- LU, P. et al. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. *arXiv preprint arXiv:2310.02255*, 2023. Disponível em: <https://arxiv.org/abs/2310.02255>. 48
- LU, S. et al. Ovis: Structural embedding alignment for multimodal large language model. *arXiv:2405.20797*, 2024. 49, 50, 139, 140
- MALKOV, Y. A.; YASHUNIN, D. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence*. [s.n.], 2018. Disponível em: <https://arxiv.org/abs/1603.09320>. 37, 38, 39
- MELIS, G.; DYER, C.; BLUNSOM, P. On the state of the art of evaluation in neural language models. *arXiv preprint arXiv:1707.05589*, 2017. [Submitted on 18 Jul 2017 (v1), last revised 20 Nov 2017 (this version, v2)]. 35
- MERCHANT, R.; LI, X.; HOCKENMAIER, J. Grammatical information in bert sentence embeddings as two-dimensional arrays. *arXiv preprint arXiv:2010.00793*, 2020. 35, 44
- Meta AI . *Llama 4: Multimodal Intelligence at Scale*. 2024. Accessed: 2025-04-07. Disponível em: <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>. 49, 50, 139, 140

- Meta AI. *Introducing Llama 3.2 with Vision Capabilities*. 2024. Accessed: 2025-04-07. Disponível em: <<https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>>. 49, 50, 139, 140
- MICROSOFT et al. *Phi-4-Mini Technical Report: Compact yet Powerful Multimodal Language Models via Mixture-of-LoRAs*. 2025. Disponível em: <<https://arxiv.org/abs/2503.01743>>. 51, 52, 142
- MIELKE, S. J. et al. *Between words and characters: A Brief History of Open-Vocabulary Modeling and Tokenization in NLP*. 2021. Disponível em: <<https://arxiv.org/abs/2112.10508>>. 29, 30
- MIKOLOV, T. et al. *Efficient Estimation of Word Representations in Vector Space*. 2013. Disponível em: <<https://arxiv.org/abs/1301.3781>>. 34
- MUENNIGHOFF, N. et al. *MTEB: Massive Text Embedding Benchmark*. 2023. Disponível em: <<https://arxiv.org/abs/2210.07316>>. 48
- NASREDDINE, Z. S. et al. The montreal cognitive assessment, moca: A brief screening tool for mild cognitive impairment. *Journal of the American Geriatrics Society*, v. 53, n. 4, p. 695–699, 2005. Disponível em: <<https://agsjournals.onlinelibrary.wiley.com/doi/abs/10.1111/j.1532-5415.2005.53221.x>>. 25
- NEMCIKOVA, M.; KATRENIKOVA, Z.; NAGYOVA, I. Social support, positive caregiving experience, and caregiver burden in informal caregivers of older adults with dementia. *Frontiers in Public Health*, v. 11, 2023. ISSN 2296-2565. Disponível em: <<https://www.frontiersin.org/journals/public-health/articles/10.3389/fpubh.2023.1104250>>. 22
- NETO, H. G. R. et al. Mental state examination and its procedures—narrative review of brazilian descriptive psychopathology. *Frontiers in Psychiatry*, Volume 10 - 2019, 2019. ISSN 1664-0640. Disponível em: <<https://www.frontiersin.org/journals/psychiatry/articles/10.3389/fpsyt.2019.00077>>. 23
- OMELIANCHUK, P. et al. Grammatical error correction: A survey with deep learning perspective. *arXiv preprint arXiv:2101.09362*, 2021. Disponível em: <<https://arxiv.org/abs/2101.09362>>. 83
- OMURA, J. D. et al. Modifiable risk factors for alzheimer disease and related dementias among adults aged 45 years — united states, 2019. *Morbidity and Mortality Weekly Report (MMWR)*, Centers for Disease Control and Prevention, v. 71, n. 20, p. 680–685, 2022. Accessed: 2025-01-17. Disponível em: <<https://www.cdc.gov/mmwr/volumes/71/wr/pdfs/mm7120a2-H.pdf>>. 20
- OPENAI. *Introducing GPT-4o: OpenAI's Most Capable Multimodal Model*. 2024. Accessed: 2025-04-07. Disponível em: <<https://openai.com/index/hello-gpt-4o/>>. 34, 139, 140
- PARDO-MORENO, T. et al. Therapeutic approach to alzheimer's disease: Current treatments and new perspectives. *Pharmaceutics*, v. 14, n. 6, 2022. ISSN 1999-4923. Disponível em: <<https://www.mdpi.com/1999-4923/14/6/1117>>. 15, 20
- PATIL, R. et al. A survey of text representation and embedding techniques in nlp. *IEEE Access*, v. 11, p. 36120–36146, 2023. 33, 34

- RADFORD, A. et al. *Learning Transferable Visual Models From Natural Language Supervision*. 2021. Disponível em: <<https://arxiv.org/abs/2103.00020>>. 31, 34, 36, 50, 141
- RADFORD, A. et al. Language models are unsupervised multitask learners. *OpenAI Blog*, v. 1, n. 8, p. 9, 2019. 31
- RAFFEL, C. et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. 2023. Disponível em: <<https://arxiv.org/abs/1910.10683>>. 30
- React Team. *Learn React*. 2025. <<https://react.dev/learn>>. Acesso em: 2 de dezembro de 2025. 118
- Redux Team. *Redux: A Predictable State Container for JS Apps*. 2025. Accessed: 2025-12-06. Disponível em: <<https://redux.js.org/>>. 93
- ROSENBLATT, F. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, v. 65, n. 6, p. 386–408, 1958. 27
- SCHUHMANN, C. et al. LAION-5b: An open large-scale dataset for training next generation image-text models. In: *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. [s.n.], 2022. Disponível em: <<https://openreview.net/forum?id=M3Y74vmsMcY>>. 50, 141
- SHERSTINSKY, A. Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network. *Physica D: Nonlinear Phenomena*, v. 404, p. 132306, 2020. ISSN 0167-2789. Disponível em: <<https://www.sciencedirect.com/science/article/pii/S0167278919305974>>. 28
- SILVA, A. B. V.; BEJJANI, M. *ArtEvoke: AI-powered platform to trigger memories in Alzheimer's patients using art imagery*. Dissertação (Master's Thesis) — UCLouvain, Ecole Polytechnique de Louvain (EPL), Louvain-la-Neuve, Belgium, 2025. Disponível em: <<https://hdl.handle.net/2078.2/43752>>. 7, 16
- SINGHAL, K. et al. Large language models encode clinical knowledge. *Nature*, v. 620, n. 7972, p. 172–180, 2023. ISSN 1476-4687. Disponível em: <<https://doi.org/10.1038/s41586-023-06291-2>>. 31
- SNYDERMAN, D.; ROVNER, B. Mental status exam in primary care: a review. *American Family Physician*, v. 80, n. 8, p. 809–814, October 15 2009. ISSN 0002-838X. 23
- SUN, Y. et al. Survival after the diagnosis of mild-to-moderate alzheimer's disease dementia: A 15-year national cohort study in taiwan. *International Journal of Geriatric Psychiatry*, v. 39, n. 9, p. e6152, 2024. Disponível em: <<https://onlinelibrary.wiley.com/doi/abs/10.1002/gps.6152>>. 20
- SUPRIYONO et al. Advancements in natural language processing: Implications, challenges, and future directions. *Telematics and Informatics Reports*, v. 16, p. 100173, 2024. ISSN 2772-5030. Disponível em: <<https://www.sciencedirect.com/science/article/pii/S2772503024000598>>. 29, 30
- TAN, W. R. et al. Improved artgan for conditional synthesis of natural image and artwork. *IEEE Transactions on Image Processing*, v. 28, n. 1, p. 394–409, 2019. Disponível em: <<https://doi.org/10.1109/TIP.2018.2866698>>. 61

- TEAM, G. et al. *Gemma 3 Technical Report*. 2025. Disponível em: <<https://arxiv.org/abs/2503.19786>>. 51, 52, 142
- TEAM, Q. *Qdrant Use Cases*. 2023. <<https://qdrant.tech/documentation/use-cases/>>. 39
- TREDER, M. S.; LEE, S.; TSVETANOV, K. A. Introduction to large language models (llms) for dementia care and research. *Frontiers in Dementia*, v. 3, 2024. ISSN 2813-3919. Disponível em: <<https://www.frontiersin.org/journals/dementia/articles/10.3389/frdem.2024.1385303>>. 23
- VASWANI, A. et al. *Attention Is All You Need*. 2023. Disponível em: <<https://arxiv.org/abs/1706.03762>>. 28, 29, 30
- W3C. *Web Content Accessibility Guidelines (WCAG) 2.1*. 2018. <<https://www.w3.org/TR/WCAG21/>>. Accessed: 2025-04-24. 105
- WANG, L. et al. *Text Embeddings by Weakly-Supervised Contrastive Pre-training*. 2024. Disponível em: <<https://arxiv.org/abs/2212.03533>>. 50, 141
- WANG, Y. et al. *MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark*. 2024. Disponível em: <<https://arxiv.org/abs/2406.01574>>. 49
- WANG, Z.-Y.; CHO, J. Y. Older adults' response to color visibility in indoor residential environment using eye-tracking technology. *Sensors*, v. 22, n. 22, p. 8766, 2022. Disponível em: <<https://www.mdpi.com/1424-8220/22/22/8766>>. 106
- WEIZENBAUM, J. Eliza—a computer program for the study of natural language communication between man and machine. *Commun. ACM*, Association for Computing Machinery, New York, NY, USA, v. 9, n. 1, p. 36–45, jan. 1966. ISSN 0001-0782. Disponível em: <<https://doi.org/10.1145/365153.365168>>. 22, 29
- WikiArt. *WikiArt.org - Visual Art Encyclopedia*. 2024. Accessed: 2025-05-16. Disponível em: <<https://www.wikiart.org/en/>>. 61
- Wolters Kluwer Health: Lippincott Williams & Wilkins. *Color vision problems become more common with age, study shows*. 2014. Accessed: 2025-05-12. Disponível em: <<https://www.sciencedaily.com/releases/2014/02/140220102614.htm>>. 106
- WOODS, B. et al. Cognitive stimulation to improve cognitive functioning in people with dementia. *Cochrane Database of Systematic Reviews*, John Wiley & Sons, Ltd., v. 1, n. 1, p. CD005562, January 31 2023. ISSN 1469-493X. Epub 2023 Jan 31. Disponível em: <<https://doi.org/10.1002/14651858.CD005562.pub3>>. 15, 21
- World Health Organization. *Dementia: number of people affected to triple in next 30 years*. 2017. Accessed: 2025-05-11. Disponível em: <<https://www.who.int/news/item/07-12-2017-dementia-number-of-people-affected-to-triple-in-next-30-years>>. 15
- World Health Organization. *Dementia: Facts in pictures*. 2021. <<https://www.who.int/news-room/facts-in-pictures/detail/dementia>>. Accessed: 2025-01-17. 19
- World Health Organization. *Dementia: fact sheets*. 2023. <<https://www.who.int/news-room/fact-sheets/detail/dementia>>. Accessed: 2025-01-15. 19

- XU, C. et al. *Benchmark Data Contamination of Large Language Models: A Survey*. 2024. Disponível em: <<https://arxiv.org/abs/2406.04244>>. 33
- XU, S. A guide to open-source embedding models. *BentoML Blog*, Mar 2025. Disponível em: <<https://www.bentoml.com/blog/a-guide-to-open-source-embedding-models>>. 35
- YANG, A. et al. *Qwen3 Technical Report*. 2025. Disponível em: <<https://arxiv.org/abs/2505.09388>>. 51, 52, 142
- YUE, X. et al. *MMMU: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark for Expert AGI*. 2024. Disponível em: <<https://arxiv.org/abs/2311.16502>>. 48
- ZAIDEL, D. W. Art and brain: insights from neuropsychology, biology and evolution. *Journal of Anatomy*, Wiley, v. 216, n. 2, p. 177–183, February 2010. ISSN 1469-7580, 0021-8782. Disponível em: <<https://doi.org/10.1111/j.1469-7580.2009.01099.x>>. 21, 43
- ZHANG, P. et al. *Long Context Compression with Activation Beacon*. 2024. Disponível em: <<https://arxiv.org/abs/2401.03462>>. 141
- ZHANG, W. et al. Artificial intelligence technology in alzheimer’s disease research. *Intractable Rare Diseases Research*, v. 12, n. 4, p. 208–212, 2023. 22
- ZHANG, Y. et al. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025. 50
- ZOUHAR, V. et al. *A Formal Perspective on Byte-Pair Encoding*. 2024. Disponível em: <<https://arxiv.org/abs/2306.16837>>. 29

A Models Overview and Benchmarks

The following tables compile data from official model documentation and benchmark papers. A dash (–) indicates that no official result was found.

A.1 LMM Model Specifications

Team	Name	Parameters (B)	Release Date	Context tokens (K)	Max output tokens	Weights Access
AIDC-AI	Ovis2	8	03/2025	32	Not specified	Open-source
	Ovis2	16	03/2025			
	Ovis2	34	03/2025			
Anthropic	Claude 3.5 Haiku	–	07/2024	200	8192	Closed
	Claude 3.5 Sonnet	–	04/2024		8192	
	Claude 3.7 Sonnet	–	11/2024		8.192 / 64.000	
Google	Gemini 2.0 Flash-Lite	–	02/2025	1.048	8.192	Closed
	Gemini 2.0 Flash	–	02/2025	1.048	8.192	
	Gemini 2.0 Flash Thinking	–	02/2025	1.048	65.536	
	Gemini 2.0 Pro	–	02/2025	2.097	8.192	
LLaVA	1.5	7	09/2023	4.096	Not specified	Open-source
	1.5	13	09/2023			
	NeXT Vicuna	7	01/2024			
	NeXT Vicuna	13	12/2023			
	NeXT Mistral	7	12/2023			
	NeXT	34	01/2024			
Llama	3.2 Vision	11	09/2024	128	2.048	Restricted
	3.2 Vision	90	09/2024	128	2.048	
	4 Scout	~109	04/2025	10.000	Not specified	
	4 Maverick	~400	04/2025	10.000	Not specified	
	4 Behemoth	2.000	04/2025	10.000	Not specified	
OpenAI	GPT4.5 Preview	–	02/2025	128	16.384	Closed
	GPT-4o	–	08/2024			
Qwen	2.5-VL	3	02/2025	32	8.192	Open-source
		7	02/2025			
		32	03/2025			

Table 23 – Overview of evaluated multimodal SoTA models with support for image inputs.
Source: (LU et al., 2024; LIU et al., 2023; LIU et al., 2024; BAI et al., 2025; Anthropic, 2024; Anthropic , 2024; GOOGLE, 2025; Meta AI, 2024; Meta AI , 2024; OPENAI, 2024)

A.2 Benchmark Results

Team	Name	Parameters (B)	MMMU (0-shot)	VQAv2	MathVista
AIDC-AI	Ovis2	8	59%	–	71.40%
	Ovis2	16	59.60%	–	74.90%
	Ovis2	34	65.60%	–	77%

Team	Name	Parameters (B)	MMMU (0-shot)	VQAv2	MathVista
Anthropic	Claude 3.5 Haiku	-	60.50%	-	61.60%
	Claude 3.5 Sonnet	-	70.40%	-	65.40%
	Claude 3.7 Sonnet	-	71.60%	-	-
Google	Gemini 2.0 Flash-Lite	-	68%	-	57.60%
	Gemini 2.0 Flash	-	71.70%	-	-
	Gemini 2.0 Flash Thinking	-	75.40%	-	-
	Gemini 2.0 Pro	-	72.70%	-	-
LLaVA	1.5	7	-	78.50%	-
	1.5	13	36.40%	80%	-
	NeXT Vicuna	7	35.80%	81.80%	34.60%
	NeXT Vicuna	13	36.20%	82.80%	35.60%
	NeXT Mistral	7	35.30%	82.20%	37.70%
	NeXT	34	51.10%	83.70%	46.50%
Llama	3.2 Vision	11	50.70%	75.20%	51.50%
	3.2 Vision	90	60.30%	78.10%	57.30%
	4 Scout	~109	69.40%	-	70.70%
	4 Maverick	~400	73.40%	-	73.70%
	4 Behemoth	-	76.10%	-	-
OpenAI	GPT4.5 Preview	-	74.40%	-	-
	GPT-4o	-	69.10%	-	63.80%
Qwen	2.5-VL	3	53.10%	-	62.30%
	2.5-VL	7	58.60%	-	68.20%
	2.5-VL	32	70%	-	74.70%

Table 24 – Performance of multimodal models on benchmark. Results are reported for MMMU (zero-shot), VQAv2 (visual question answering), and MathVista (math + vision tasks). Source: (LU et al., 2024; LIU et al., 2023; LIU et al., 2024; BAI et al., 2025; Anthropic, 2024; Anthropic , 2024; GOOGLE, 2025; Meta AI, 2024; LIU et al., 2025; Meta AI , 2024; OPENAI, 2024).

A.3 Embedding Models

Team	Name	Image size	Embedding Dimension	Parameters (M)	Release Date	Modality	Weights Available
OpenCLIP	ViT-B/32	256px	512	151	Jan 2021	Image-Text	Open-source
	ViT-B/16	224px	512	150	Jul 2021		
	ViT-L/14	224px	768	428	Jan 2022		
	ViT-L/14	336px	768	428	April 2022		
	ViT-G/14	224px	1024	601	July 2023		
	ConvNeXt-XXLarge	224px	1024	848	Jan 2023		
	ConvNeXt-Large @320px	320px	1024	690	Jan 2023		
Google	ViT-SO400M/14 (SigLIP)	224px	768	632	Aug 2023	Image-Text	Open-source
	ViT-L/14 (DFN)	224px	768	428			
	ViT-SO400M-14 (SigLIP)	384px	768	632			
	ViT-H/14-quickgelu (DFN)	224px	1024	632			
	ViT-H-14-378-quickgelu (DFN)	378px	1024	632			
BAAI	BGE-M3	-	1024	569	Jan 2024	Text	Restrictive
	bge-large-en-v1.5	-	1024	335			
	bge-base-en-v1.5	-	768	110			
Sentence Transformers	all-MiniLM-L6-v2	-	384	22,7	Apr 2019	Text	Restrictive
DAMO Academy	GTE-large	-	1024	335	Aug 2023	Text	Open-source
intfloat	E5-large-v2	-	1024	335	Dec 2022	Text	Open-source
Qwen3	Embedding 0.6B	-	1024	600	Jun 2025	Text	Open-source
Qwen3	Embedding 4B	-	2560	4000	Jun 2025	Text	Open-source

Table 25 – Overview of evaluated embeddings models. Source: (ILHARCO et al., 2021; RADFORD et al., 2021; SCHUHMAN et al., 2022; CHEN et al., 2024b; ZHANG et al., 2024; LI et al., 2023b; WANG et al., 2024a)

A.4 Embeddings Benchmark

Team	Name	MTEB (Average)	ImageNet (Zero-shot)
OpenCLIP	ViT-B/32	–	72.8%
	ViT-B/16	–	73.5%
	ViT-L/14 - 224	–	75.3%
	ViT-L/14 - 336	–	79.2%
	ViT-G/14	–	80.1%
	ConvNeXt-XXLarge	–	79.5%
	ConvNeXt-Large@320px	–	76.9%
Google	ViT-SO400M/14 (SigLIP)	–	82.0%
	ViT-L/14 (DFN)	–	82.2%
	ViT-SO400M-14 (SigLIP)	–	83.1%
	ViT-H/14-quickgelu (DFN)	–	83.4%
	ViT-H-14-378-quickgelu (DFN)	–	84.4%
BAAI	BGE-M3	59.56%	N/A
	bge-large-en-v1.5	45.08%	N/A
	bge-base-en-v1.5	–	N/A
Sentence Transformers	all-MiniLM-L6-v2	56.26%	N/A
DAMO Academy	GTE-large	63.13%	N/A
intfloat	E5-large-v2	62.25%	N/A
Qwen3	Embedding 0.6B	70.70%	N/A
Qwen3	Embedding 4B	74.60%	N/A

Table 26 – List of evaluated embedding models and their benchmark scores. Source: (ILHARCO et al., 2021; Hugging Face,)

A.5 LLM Models Specifications

Team	Name	Parameters (B)	Release Date	Context tokens (K)	Max output tokens	Weights Access
Qwen	3	0.6 1.7 4	04/2024	32	8.192	Open-source
Microsoft	Phi-4-mini	3.8	02/2025	200	Not specified	Open-source
Google	Gemma 3	1 4	03/2025	32 128	8192	Open-source
Meta	Llama 3.2	1 3	09/2024	128	Not specified	Open-source
Maritaca AI	Sabiá-3 Sabiazinho-3	- -	09/2024 02/2025	32	Not specified	Closed

Table 27 – Lightweight LLMs (<4B parameters) for story generation from text-only input.

Source: (YANG et al., 2025; MICROSOFT et al., 2025; TEAM et al., 2025; LIU et al., 2025; ABONIZIO et al., 2025)

A.6 LLM Models Benchmarks

Team	Name	Parameters (B)	MMLU-Pro
Qwen	3	0.6 1.7 4	24.74 36.76 50.58
Microsoft	Phi-4-mini	3.8	52.8
Google	Gemma 3	1 4	14.7 43.6
Meta	Llama 3.2	1 3	13.5 34.3
Maritaca AI	Sabiá-3 Sabiazinho-3	- -	- -

Table 28 – Lightweight LLMs (<4B parameters) benchmarks. Source: (YANG et al., 2025; MICROSOFT et al., 2025; TEAM et al., 2025; LIU et al., 2025)

B Datasets

B.1 Sampled 2000 images from SemArt

In Figure 67, *Others* (123) aggregates timeframes with fewer than 100 artworks, including 1401–1450 (84), 1301–1350 (66), and 1351–1400 (37), as well as earlier periods with very few artworks.

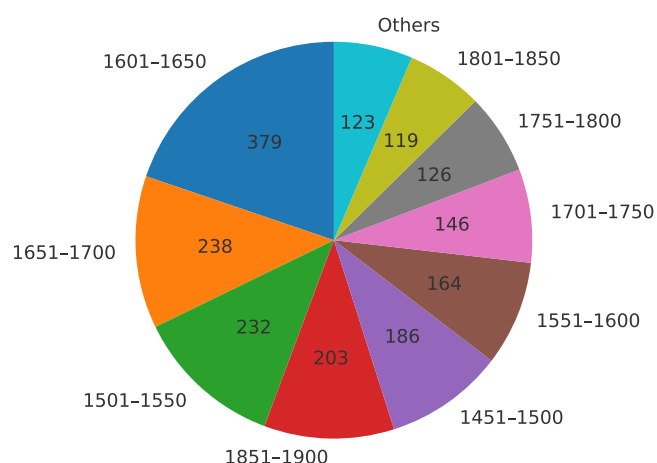


Figure 67 – Distribution of sampled artworks by timeframe.

In Figure 68, *Others* (279) includes all artistic schools with fewer than 100 artworks, such as English (54), Netherlandish (26), Austrian (15), Hungarian (14), and several others with smaller representation.

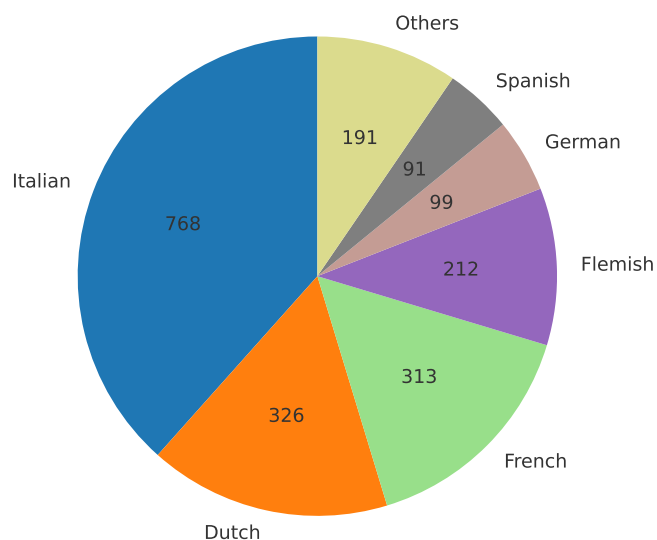


Figure 68 – Distribution of sampled artworks by artistic school/origin.

B.2 Sampled 15,570 images from SemArt

In Figure 69, *Others* (621) aggregates timeframes with fewer than 400 artworks, including 1351–1400 (378), 1251–1300 (132), 1201–1250 (27), and 1151–1200 (22).

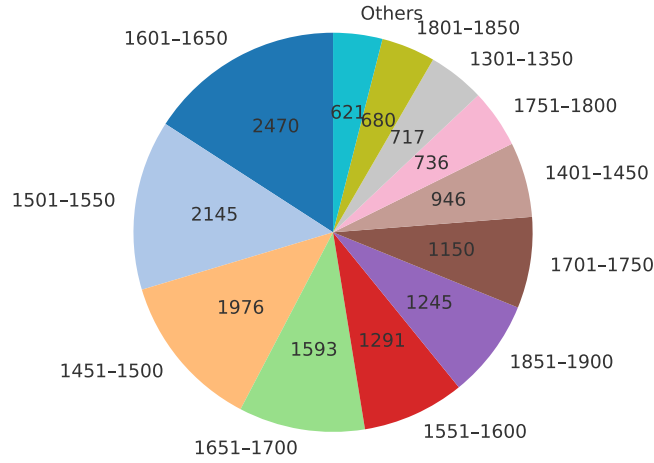


Figure 69 – Distribution of sampled artworks by timeframe.

In Figure 70, *Others* (1223) includes all artistic schools with fewer than 400 artworks, such as Netherlandish (220), Austrian (168), Hungarian (145), American (81), Danish (75), Swiss (59), Russian (47), and others with smaller representation.

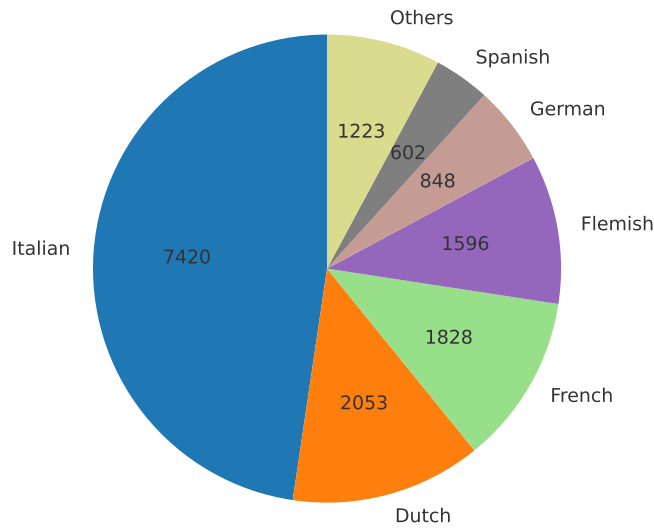


Figure 70 – Distribution of sampled artworks by artistic school/origin.

C Similarity scores

C.1 Statistical information between models

This appendix presents a detailed comparison of similarity scores between various text embedding models and state-of-the-art vision-language models.

Embedding	Model	Mean	Std	Min	Max
MiniLM-L6-v2	llava-v1.5-7b	0.2984	0.1394	-0.1046	0.7332
	llava-v1.5-13b	0.3013	0.1367	-0.0957	0.7221
	llava-v1.6-vicuna-7b	0.3609	0.1375	-0.0415	0.7876
	llava-v1.6-vicuna-13b	0.3679	0.1387	-0.0791	0.7561
	llava-v1.6-mistral-7b	0.3613	0.1414	-0.0966	0.8112
	Llama-3.2-11B-Vision-Instruct	0.3737	0.1417	-0.0435	0.7474
	Qwen2.5-VL-3B-Instruct	0.3755	0.1456	-0.0533	0.8427
	Qwen2.5-VL-7B-Instruct	0.3994	0.1435	-0.0672	0.8191
bge-large-en-v1.5	llava-v1.5-7b	0.5529	0.0813	0.3359	0.8191
	llava-v1.5-13b	0.5526	0.0797	0.3253	0.8541
	llava-v1.6-vicuna-7b	0.5997	0.0791	0.3696	0.8496
	llava-v1.6-vicuna-13b	0.6053	0.0788	0.3713	0.8718
	llava-v1.6-mistral-7b	0.5981	0.0787	0.3698	0.8599
	Llama-3.2-11B-Vision-Instruct	0.6084	0.0797	0.3699	0.8447
	Qwen2.5-VL-3B-Instruct	0.6007	0.0803	0.3861	0.8572
	Qwen2.5-VL-7B-Instruct	0.6219	0.0789	0.3839	0.8800
e5-large-v2	llava-v1.5-7b	0.7559	0.0390	0.6640	0.9035
	llava-v1.5-13b	0.7564	0.0377	0.6720	0.8931
	llava-v1.6-vicuna-7b	0.7689	0.0378	0.6682	0.8850
	llava-v1.6-vicuna-13b	0.7697	0.0380	0.6557	0.9100
	llava-v1.6-mistral-7b	0.7667	0.0376	0.6780	0.9116
	Llama-3.2-11B-Vision-Instruct	0.7708	0.0390	0.6497	0.8997
	Qwen2.5-VL-3B-Instruct	0.7700	0.0405	0.6595	0.9161
	Qwen2.5-VL-7B-Instruct	0.7725	0.0407	0.6603	0.9134
gte-large	llava-v1.5-7b	0.8137	0.0370	0.7134	0.9331
	llava-v1.5-13b	0.8145	0.0361	0.7169	0.9437
	llava-v1.6-vicuna-7b	0.8349	0.0356	0.7328	0.9425
	llava-v1.6-vicuna-13b	0.8375	0.0349	0.7284	0.9477
	llava-v1.6-mistral-7b	0.8328	0.0356	0.7277	0.9338
	Llama-3.2-11B-Vision-Instruct	0.8372	0.0363	0.7192	0.9390
	Qwen2.5-VL-3B-Instruct	0.8366	0.0363	0.7376	0.9474
	Qwen2.5-VL-7B-Instruct	0.8434	0.0359	0.7415	0.9637

Table 29 – Updated similarity statistics (mean, std, min, max) between text embeddings and vision-language models.

C.2 Statistical information between pixel max sizes

This appendix presents a detailed comparison of similarity scores between pixel max sizes in the model chosen.

Embedding	Model	Mean	Std	Min	Max
MiniLM-L6-v2	Qwen2.5-VL-7B-Instruct (512×512)	0.3994	0.1435	-0.0672	0.8191
	Qwen2.5-VL-7B-Instruct (512×640)	0.3975	0.1432	-0.0902	0.7890
	Qwen2.5-VL-7B-Instruct (512×896)	0.3978	0.1443	-0.0662	0.7904
	Qwen2.5-VL-7B-Instruct (512×1024)	0.3973	0.1456	-0.0819	0.8300
	Qwen2.5-VL-7B-Instruct (1024×1024)	0.3976	0.1422	-0.0841	0.8397
bge-large-en-v1.5	Qwen2.5-VL-7B-Instruct (512×512)	0.6219	0.0789	0.3839	0.8800
	Qwen2.5-VL-7B-Instruct (512×640)	0.6226	0.0795	0.3807	0.8413
	Qwen2.5-VL-7B-Instruct (512×896)	0.6223	0.0787	0.3736	0.8529
	Qwen2.5-VL-7B-Instruct (512×1024)	0.6221	0.0777	0.3821	0.8488
	Qwen2.5-VL-7B-Instruct (1024×1024)	0.6214	0.0764	0.3792	0.8557
e5-large-v2	Qwen2.5-VL-7B-Instruct (512×512)	0.7725	0.0407	0.6603	0.9134
	Qwen2.5-VL-7B-Instruct (512×640)	0.7734	0.0402	0.6615	0.9199
	Qwen2.5-VL-7B-Instruct (512×896)	0.7732	0.0407	0.6606	0.9233
	Qwen2.5-VL-7B-Instruct (512×1024)	0.7736	0.0402	0.6566	0.9076
	Qwen2.5-VL-7B-Instruct (1024×1024)	0.7729	0.0403	0.6760	0.9118
gte-large	Qwen2.5-VL-7B-Instruct (512×512)	0.8434	0.0359	0.7415	0.9637
	Qwen2.5-VL-7B-Instruct (512×640)	0.8434	0.0362	0.7285	0.9517
	Qwen2.5-VL-7B-Instruct (512×896)	0.8436	0.0362	0.7313	0.9495
	Qwen2.5-VL-7B-Instruct (512×1024)	0.8433	0.0359	0.7432	0.9438
	Qwen2.5-VL-7B-Instruct (1024×1024)	0.8433	0.0355	0.7371	0.9511

Table 30 – Summary of the similarity statistics for Qwen2.5-VL-7B-Instruct evaluated at various image resolutions.

D Survey answers

The table below provides supplementary data for the 18 image comparison sets used in the user survey. Negative values indicate a preference for the model using GTE-Large, while positive values indicate a preference for the alternative model, ConvNeXt.

Phrase	Comparison	Mean	Std	Min	Max
Phrase 1	1	-0.52	0.94	-1.66	1.59
	2	-0.17	1.03	-2.27	1.57
	3	-0.44	0.81	-1.75	1.87
Phrase 2	1	-0.13	1.00	-1.66	1.74
	2	-0.34	1.17	-2.10	2.18
	3	0.19	0.83	-1.39	2.02
Phrase 3	1	0.29	1.05	-1.93	2.02
	2	0.09	1.00	-1.80	1.62
	3	-0.13	1.25	-1.93	2.53
Phrase 4	1	0.21	1.03	-1.56	2.19
	2	0.41	0.84	-1.39	1.62
	3	-0.66	1.05	-2.10	1.51
Phrase 5	1	0.13	1.03	-2.02	1.62
	2	-0.51	0.84	-1.67	2.07
	3	-0.16	0.67	-1.28	1.42
Phrase 6	1	-0.67	0.93	-1.93	2.18
	2	-0.06	0.70	-1.39	1.45
	3	0.88	0.60	-0.20	1.92

Table 31 – Summary of participant preferences across phrases and comparisons. Ratings range from 1 (strong preference for left) to 10 (strong preference for right), centered around 0.

E FAISS configurations

Table 32 summarizes the specific FAISS index configurations used in the CPU and GPU benchmarking experiments. Parameters such as the number of clusters (nlist), the number of subquantizers (m_pq), and the number of bits per subquantizer (nbits_pq) were selected based on dataset size and hardware constraints to ensure training and fair comparison.

# Samples	Index Type	nlist	m_pq	nbits_pq (CPU)	nbits_pq (GPU)
100	IndexIVF	4	–	–	–
100	IndexIVFPQ	4	4	4	–
1000	IndexIVF	25	–	–	–
1000	IndexIVFPQ	25	8	6	8
5000	IndexIVF	50	–	–	–
5000	IndexIVFPQ	50	16	6	8
10000	IndexIVF	100	–	–	–
10000	IndexIVFPQ	100	16	6	8
15000	IndexIVF	100	–	–	–
15000	IndexIVFPQ	100	16	6	8

Table 32 – Index configurations used for IndexIVF and IndexIVFPQ in CPU benchmarking.

F Database Schema

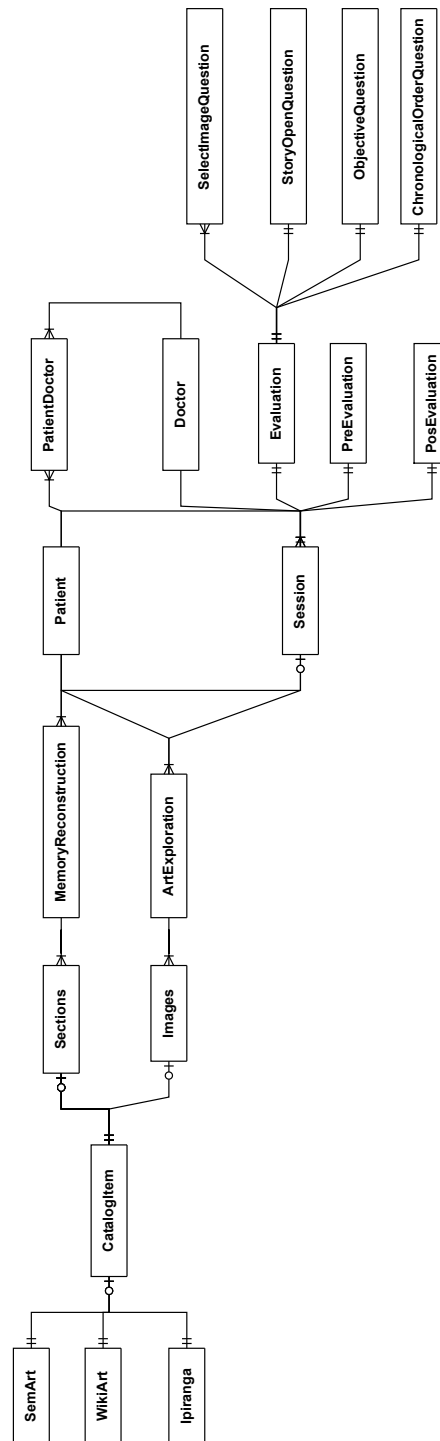


Figure 71 – Sideways representation of the full database schema used in the platform.